

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP.HCM



**TRIỂN KHAI, GIÁM SÁT VÀ TỐI ƯU HÓA HỆ THỐNG PHẦN
MỀM HƯỚNG KIẾN TRÚC MICROSERVICES DỰA TRÊN
DOCKER SWARM ORCHESTRATION.**

Nhóm 2 - Sinh viên thực hiện

1. Trần Đức Vũ – 21013481
 2. Nguyễn Ngọc Thanh Nguyên – 21116601
 3. Ngô Hoàng Khải – 21065691
 4. Hồ Quốc Thái – 21066101
 5. Bùi Ngọc Minh - 21078101
 6. Nguyễn Thế Lực - 21046661
 7. Trịnh Nam Trung – 21001205
 8. Phan Thị Như Quỳnh - 21076261
 9. Nguyễn Mai Đức Thắng - 21029381
- Đặng Bảo Chánh -21071141

Giáo viên hướng dẫn: Huỳnh Nam

CHƯƠNG 1: GIỚI THIỆU	1
1.1 Tổng quan	1
1.2 Mục tiêu đề tài	1
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	3
2.1 Các chế độ mạng trên môi trường Virtual Box.....	3
2.2 Docker Swarm	4
2.2.1 Docker swarm là gì ?.....	4
2.2.2 Thành phần và kiến trúc của Docker Swarm:	4
2.2.3 Tính năng nổi bật của Docker Swarm	5
2.3 ELK stack.	5
2.3.1 ELK Stack là gì ? Lợi ích của ELK Stack:.....	5
2.3.2 Cách mà ELK Stack hoạt động	6
2.3.3 Elasticsearch.....	6
2.3.4 Logstash.....	6
2.4 Kibana	7
2.4.1 Các chức năng của Kibana	7
2.5 Grafana và Prometheus.....	8
2.5.1 Grafana	8
2.5.2 Prometheus	8
2.6 Reverse proxy.	9
2.6.1 Reverse proxy là gì?	9
2.6.2 Tại sao nên sử dụng Reverse proxy.....	9
2.6.3 Ứng dụng của Reverse proxy có thể xảy ra những nguy cơ gì?.....	10
2.6.4 Chức năng và ứng dụng.....	11
2.7 Giới thiệu các metric được thu thập trên toàn bộ các node	12
CHƯƠNG 3: SƠ ĐỒ MẠNG VÀ KIẾN TRÚC HỆ THỐNG.....	13
3.7 Sơ đồ mạng	13
3.8 Kiến trúc hệ thống.....	13
CHƯƠNG 4: HIỆN THỰC.....	14

4.1 Xây dựng LAN network trên môi trường virtual box	14
4.1.1 Xây dựng được mạng LAN trên môi trường Virtual Box đúng phiên bản các nodes theo yêu cầu	14
4.2 Docker swarm.	23
4.2.1 Cấu hình Docker Swarm	23
4.2.2 Kiểm tra CPU / memory và I/O Usage của một node	29
4.3 Services va high availability.	32
4.3.1 Scale Services	32
4.3.2 High Avaibability	35
4.4 Giám sát và quản trị hệ thống	35
4.4.1 Kiểm tra xem các dịch vụ đang hoạt động trên các node nào	35
4.4.2 Kiểm tra CPU/ memory và I/O Usage của cụm swarm (Sử dụng grafana + prAometheus)	39
4.4.3 Đo latency và response time của hasher và rng	44
4.4.4 Sử dụng câu lệnh để đo:	45
sudo httping -c <số lần request> <địa chỉ dịch vụ (URL hoặc IP:port)>	45
4.5 Logstash chạy trên swarm với 2 replicas	46
4.6 Test log của logstash đọc logs từ 4 services: rng,hasher,worker,webui	47
4.7 Logstash xử lý lấy dữ liệu log theo filter với các thông tin quan trọng	53
4.8 Elasticsearch chạy trên swarm	53
4.9 Truy vấn dữ liệu trong elasticsearch	53
4.10 Kibana chạy trên swarm	53
4.11 Kibana kết nối lấy dữ liệu thành công từ ElasticSearch	53
4.11.1 Truy cập Kibana: Kibana chạy trên port 5601 . Mở trình duyệt và truy cập: http://192.168.139.101:5601	53
4.12 Thao tác Kibana tìm kiếm logs, trực quan, phân tích dữ liệu logs	55
4.12.1 Tìm kiếm logs	55
4.12.2 Trực quan	56
4.12.3 Phân tích dữ liệu logs	57
4.13 Hiện thị các thông tin metric trên cụm như mục trên bằng lệnh CLI	59
4.13.1 CPU, RAM (active or inactive memory), disk usage	59

4.13.2 Tất cả các process và state của chúng	60
4.13.3 Số lượng file đang mở, sockets và trạng thái của chúng.....	60
4.13.4 Các hoạt động của I/O (disk, network), trên thao tác hoặc dung lượng (volume)	61
4.14 Trình diễn cơ chế tạo pipeline thu thập và lưu trữ các metric với Prometheus với CLI.....	62
4.15 Đồng bộ hóa dữ liệu từ Prometheus với InfluxDB	65
4.16 Truy vấn dữ liệu metrics được thu thập được lưu trữ trong InfluxDB chạy trên swarm.....	68
4.17 Tạo Dashboard sử dụng Prometheus/Grafana để phân tích , giám sát hiệu suất swarm.....	71
4.18 Cấu hình URL routing trong reverse proxy với nginx trên cụm swarm cho các dịch vụ.....	78
4.19 Cấu hình URL routing trong reverse proxy với nginx trên cụm swarm cho các dịch vụ.....	82
CHƯƠNG 5: : KẾT LUẬN	85
5.1 Kết quả đạt được	85
5.1.1 Kết quả triển khai hệ thống Microservices trên Docker Swarm	85
5.1.2 Kết quả giám sát hệ thống với Prometheus-Grafana-InfluxDB	85
5.1.3 Kết quả giám sát log tập trung với ELK Stack (Elasticsearch, Logstash, Kibana)	86
5.1.4 Kết quả tối ưu hóa hệ thống	86
5.2 Hướng phát triển	86
5.2.1 Mở Rộng Quy Mô Hệ Thống (Scaling & High Availability)	86
5.2.2 Tăng tính bảo mật hệ thống:.....	87
5.2.3 CI/CD & DevOps Automation	87
5.2.4 Phân tích log nâng cao:	87
5.2.5 Triển khai thực tế trên nền tảng Cloud:.....	87
CHƯƠNG 6: TÀI LIỆU THAM KHẢO	88

CHƯƠNG 1: GIỚI THIỆU

1.1 Tổng quan

- Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, các hệ thống phần mềm ngày càng trở nên phức tạp và yêu cầu khả năng mở rộng, linh hoạt, cũng như dễ bảo trì. Kiến trúc microservices đã nổi lên như một giải pháp hiệu quả nhằm chia nhỏ các thành phần chức năng của hệ thống thành các dịch vụ độc lập, giúp tăng tính linh hoạt, dễ dàng triển khai và nâng cấp từng phần mà không ảnh hưởng đến toàn bộ hệ thống.
- Tuy nhiên, với sự gia tăng số lượng dịch vụ, việc quản lý, triển khai và giám sát trở nên phức tạp hơn. Để giải quyết vấn đề này, các công cụ điều phối container như **Docker Swarm** đã được sử dụng rộng rãi nhằm tự động hóa việc triển khai, mở rộng và quản lý các container trong môi trường phân tán. Đồng thời, hệ thống cần được tích hợp các công cụ giám sát hiệu quả như **Prometheus, Grafana, ELK Stack...** để theo dõi hiệu năng, phát hiện sớm sự cố và tối ưu hóa tài nguyên.
- Đề tài này tập trung vào việc xây dựng một hệ thống phần mềm theo kiến trúc microservices, triển khai trên nền tảng Docker Swarm, kết hợp với các công cụ giám sát nhằm đảm bảo hệ thống vận hành ổn định, hiệu quả và dễ mở rộng trong tương lai.

1.2 Mục tiêu đề tài

- Xây dựng một hệ thống phần mềm mẫu theo kiến trúc microservices, với các dịch vụ hoạt động độc lập và giao tiếp thông qua API .
- Triển khai hệ thống trên Docker Swarm, tận dụng các tính năng như phân phối tải, tự động scale, high availability...
- Tích hợp hệ thống giám sát sử dụng các công cụ như Prometheus, Grafana, ELK Stack để theo dõi hiệu năng, log và tình trạng hoạt động của các dịch vụ.
- Phân tích và tối ưu hóa hiệu suất hệ thống, từ đó đề xuất các giải pháp cải thiện khả năng mở rộng và độ tin cậy.

- Tạo nền tảng thực nghiệm để nghiên cứu và ứng dụng kiến thức vào các hệ thống thực tế trong môi trường doanh nghiệp.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Các chế độ mạng trên môi trường Virtual Box.

- **NAT:** Là chế độ mặc định VirtualBox cấp cho mỗi máy ảo khi được cấu hình lần đầu. Ở chế độ này, các máy ảo sẽ sử dụng 1 loại IP riêng để giao tiếp với Internet. Khi có thiết bị muốn sử dụng Internet, NAT sẽ thay đổi địa chỉ IP đó thành địa chỉ IP công cộng (thường là IP của máy thật) để giao tiếp với Internet và ngược lại. Lưu ý rằng ta không thể truy cập ngược từ host hoặc máy khác tới VirtualBox nếu không dùng port forwarding.
- + Khi sử dụng mạng NAT, VirtualBox sẽ tự động gán địa chỉ 10.0.2.15 mặc định cho máy. Ngoài ra nó cũng tự động tạo ra 1 mạng loopback là 127.0.0.1 cho máy.
- **NAT Network:** Chế độ tương tự NAT, nhưng cho phép nhiều máy ảo giao tiếp với nhau trong cùng một mạng NAT, phù hợp cho việc thiết lập môi trường mạng ảo với nhiều máy ảo cần giao tiếp nội bộ và truy cập Internet.
- + Ngoài ra, để sử dụng NAT Network, ta cần phải tạo một Network để chứa các máy để giao tiếp với nhau.
- **Bridged Adapter:** Là chế độ mà máy ảo kết nối trực tiếp vào mạng vật lý của máy chủ, nhận IP từ DHCP server trong mạng vật lý. Đây là chế độ thích hợp nhất nếu ta chỉ muốn máy ảo truy cập được Internet và không cần phải thiết lập địa chỉ mạng.
- **Host-only-adapter:** Là chế độ dùng để thiết lập một mạng lưới máy chủ và máy ảo, cho phép chúng giao tiếp với nhau nhưng không truy cập Internet. Chế độ này phù hợp cho các máy ảo muốn giao tiếp với máy thật, cần một mạng riêng tư, bảo mật cao.
- + Khi sử dụng Host-only-adapter, tương tự như NAT Network, ta cũng phải gán máy ảo này vào một mạng Host-only-adapter (Ta cũng phải tạo trước một mạng Host-only tương tự như NAT Network).

- + Khi sử dụng mạng này, trên máy thật cũng sẽ tự động tạo ra một mạng Ethernet ảo tượng trưng cho mạng Host-only-network.
- **Internal network:** Là chế độ mà các máy ảo có thể giao tiếp với nhau qua mạng nội bộ riêng, không kết nối với máy chủ hoặc Internet. Phù hợp cho các dự án cần thử nghiệm một môi trường để các máy giao tiếp với nhau trong một môi trường cô lập, tách biệt khỏi các mạng và máy vật lý khác, nâng cao bảo mật.
- + Các máy muốn gia nhập vào mạng chỉ cần điền đúng tên mạng, không cần phải tự thiết lập một network giống như NAT Network hay Host-only-adapter.
- **Generic Driver:** Là chế độ mạng trừu tượng mà VirtualBox cung cấp, cho phép ta kết nối máy ảo với các loại mạng ảo hóa khác nhau mà VirtualBox hỗ trợ. Cấu hình này rất phức tạp và dành cho các người dùng nâng cao.

2.2 Docker Swarm

2.2.1 Docker swarm là gì ?

Docker Swarm là một công cụ quản lý và triển khai các ứng dụng dựa trên Docker trên một cụm máy chủ (cluster). Nó là một phần của hệ sinh thái Docker và cho phép bạn tạo và quản lý các môi trường ứng dụng phân tán trên nhiều máy chủ.

2.2.2 Thành phần và kiến trúc của Docker Swarm:

Swarm Manager: Là thành phần quản lý cụm trong Docker Swarm. Nó là trung tâm điều khiển của cụm và chịu trách nhiệm quản lý và điều phối các nút và dịch vụ trong cụm. Swarm Manager duy trì trạng thái của cụm, xử lý yêu cầu triển khai và quản lý các tác vụ như cân bằng tải và khám phá dịch vụ.

Swarm Node: Là các máy chủ trong cụm Docker Swarm. Có hai loại node:

- **Manager Node:** Đóng vai trò điều phối và quản lý cụm.
- **Worker Node:** Nhận và thực thi các tác vụ được chỉ định bởi Manager.

Service: là đơn vị triển khai ứng dụng trong Docker Swarm. Mỗi service có thể bao gồm một hoặc nhiều bản sao container (replica). Dịch vụ có thể được định nghĩa bằng dòng lệnh hoặc tệp cấu hình YAML (Docker Compose). Swarm Manager chịu trách nhiệm quản lý và phân phối các bản sao này trên các node.

Overlay Network: là mạng ảo kết nối các container trên các node khác nhau trong cùng một cụm. Mạng này cho phép các container giao tiếp với nhau qua tên dịch vụ thay vì địa chỉ IP cụ thể, giúp hệ thống triển khai linh hoạt và dễ mở rộng.

Swarm Secrets: là cơ chế giúp quản lý và bảo vệ các thông tin nhạy cảm như mật khẩu, khóa API, và chứng chỉ SSL. Các secrets được mã hóa và chỉ phân phối đến container khi thực sự cần thiết, tăng cường tính bảo mật cho cụm.

Load Balancing: Docker Swarm hỗ trợ cân bằng tải tự động giữa các container trong một service. Swarm Manager sẽ phân phối các yêu cầu đến các container một cách đồng đều, tối ưu hiệu suất và đảm bảo tính sẵn sàng của ứng dụng.

Rolling Updates: cho phép cập nhật dịch vụ một cách tuần tự và an toàn. Khi có phiên bản mới của container, Swarm sẽ cập nhật lần lượt từng container mà không làm gián đoạn toàn bộ dịch vụ, đảm bảo ứng dụng vẫn hoạt động liên tục trong quá trình cập nhật.

2.2.3 Tính năng nổi bật của Docker Swarm

- **Tự động phục hồi:** Khi container hoặc node gặp sự cố, Swarm sẽ tự khởi động lại hoặc phân phối task sang node khác.
- **Quản lý dễ dàng:** Các công cụ CLI quen thuộc của Docker vẫn có thể dùng được trong Swarm.
- **Bảo mật:** Swarm sử dụng mã hóa TLS và chứng thực tự động giữa các node.
- **Khả năng mở rộng:** Có thể tăng hoặc giảm số lượng bản sao service (scaling) chỉ với một dòng lệnh.

2.3 ELK stack.

2.3.1 ELK Stack là gì ? Lợi ích của ELK Stack:

ELK Stack là từ viết tắt được sử dụng để mô tả bộ dịch vụ bao gồm ba dự án phổ biến: Elasticsearch, Logstash và Kibana. Thường được gọi là Elasticsearch, ELK Stack mang tới cho bạn khả năng tổng hợp nhật ký từ tất cả các hệ thống và ứng dụng của bạn, phân tích những nhật ký này, hiển thị dữ liệu để giám sát ứng dụng và cơ sở hạ tầng, khắc phục sự cố nhanh hơn, phân tích bảo mật, v.v

ELK Stack được sử dụng để giải quyết nhiều vấn đề, bao gồm phân tích bản ghi, tìm kiếm tài liệu, quản lý sự kiện và thông tin bảo mật (SIEM) cùng khả năng quan sát. ELK Stack cung cấp công cụ tìm kiếm và phân tích, tải nhập dữ liệu và trực quan hóa.

ELK Stack đáp ứng một nhu cầu trong lĩnh vực phân tích bản ghi. Khi ngày càng nhiều cơ sở hạ tầng CNTT của bạn chuyển sang các đám mây công cộng, bạn cần một giải pháp quản lý và phân tích nhật ký để giám sát cơ sở hạ tầng này và xử lý mọi nhật ký máy chủ, nhật ký ứng dụng và đường dẫn nhấp chuột. ELK Stack cung cấp một giải pháp phân tích bản ghi tuy đơn giản nhưng lại vô cùng hiệu quả cho các nhà phát triển và kỹ sư DevOps để giúp họ thu thập những thông tin chuyên sâu giá trị về chẩn đoán lỗi, hiệu suất ứng dụng và giám sát cơ sở hạ tầng - với mức giá vô cùng phải chăng.

2.3.2 Cách mà ELK Stack hoạt động

- Logstash tải nhập, chuyển đổi và gửi dữ liệu đến đúng điểm đích.
- Elasticsearch lập chỉ mục, phân tích và tìm kiếm dữ liệu đã tải nhập.
- Kibana hiển thị kết quả phân tích.

2.3.3 Elasticsearch

Elasticsearch là công cụ tìm kiếm và phân tích phân tán được xây dựng trên Apache Lucene. Khả năng hỗ trợ đa dạng ngôn ngữ, hiệu suất cao và tài liệu JSON không lược đồ khiến Elasticsearch trở thành một lựa chọn lý tưởng cho nhiều trường hợp sử dụng tìm kiếm và phân tích nhật ký khác nhau.

2.3.4 Logstash

Logstash là một công cụ tải nhập dữ liệu nguồn mở cho phép bạn thu thập dữ liệu từ các nguồn khác nhau, chuyển đổi dữ liệu và gửi dữ liệu tới điểm đích bạn muốn. Với các bộ lọc được tạo sẵn và hỗ trợ hơn 200 phần bổ trợ, Logstash cho phép người dùng dễ dàng tải nhập dữ liệu đến từ bất kỳ nguồn dữ liệu hay thuộc loại dữ liệu nào.

Logstash là một quy trình xử lý dữ liệu phía máy chủ, nguồn mở, gọn nhẹ cho phép bạn thu nhập dữ liệu từ các nguồn khác nhau, chuyển đổi dữ liệu nhanh chóng

và gửi dữ liệu tới điểm đích bạn muốn. Logstash thường được sử dụng như một đường ống dữ liệu cho Elasticsearch, một công cụ phân tích và tìm kiếm nguồn mở. Bởi vì được tích hợp chặt chẽ với Elasticsearch, khả năng xử lý bản ghi mạnh mẽ và hơn 200 phần bổ trợ nguồn mở được tạo sẵn có thể giúp bạn dễ dàng lập chỉ mục cho dữ liệu của mình, Logstash là một lựa chọn phổ biến cho hoạt động tải dữ liệu vào Elasticsearch.

2.4 Kibana

Kibana là một công cụ hiển thị trực quan và khám phá dữ liệu được sử dụng trong những trường hợp phân tích nhật ký và chuỗi thời gian, giám sát ứng dụng và thông tin kinh doanh. Công cụ này cung cấp những tính năng mạnh mẽ, dễ sử dụng như biểu đồ tần suất, biểu đồ đường, biểu đồ tròn, biểu đồ nhiệt và hỗ trợ không gian địa lý được tích hợp sẵn. Ngoài ra, công cụ này còn cung cấp khả năng tích hợp chặt chẽ với Elasticsearch, một công cụ phân tích và tìm kiếm phổ biến, khiến Kibana trở thành lựa chọn hàng đầu cho hoạt động hiển thị trực quan dữ liệu được lưu trữ trong Elasticsearch.

Bạn có thể chạy Kibana tại chỗ trên Đám mây điện toán linh hoạt của Amazon (Amazon EC2) hoặc trên các phiên bản được cấp phép của Apache 2.0 (lên đến phiên bản 7.10.2) của Dịch vụ OpenSearch của Amazon. OpenSearch Dashboards là một nguồn mở thay thế cho Kibana, cũng cho phép người dùng tự quản lý. Công cụ này bắt nguồn từ phiên bản nguồn mở cuối cùng của Kibana (7.10.2). Nó chứa nhiều cải tiến và được hỗ trợ tốt thông qua Dự án OpenSearch. Với hình thức triển khai tại chỗ hoặc trên Amazon EC2, bạn chịu trách nhiệm cài đặt phần mềm Kibana hoặc OpenSearch Dashboards, cung cấp và quản lý cơ sở hạ tầng. Với Dịch vụ OpenSearch, Kibana hoặc OpenSearch Dashboards được triển khai tự động với miền của bạn như một dịch vụ được quản lý hoàn toàn, tự động xử lý tất cả các tác vụ nặng để quản lý cụm.

2.4.1 Các chức năng của Kibana

Biểu đồ tương tác: Kibana cung cấp các biểu đồ và báo cáo trực quan mà bạn có thể sử dụng để điều hướng tương tác giữa một lượng lớn dữ liệu nhật ký. Bạn có

thể linh hoạt kéo các cửa sổ thời gian, phóng to/thu nhỏ những tập dữ liệu con cụ thể và xem chi tiết các báo cáo để trích xuất thông tin hữu ích từ dữ liệu của mình.

Hỗ trợ ánh xạ: Kibana đi kèm với khả năng hỗ trợ không gian địa lý mạnh mẽ để bạn có thể xếp lớp thông tin địa lý một cách liền mạch lên trên dữ liệu của mình và hiển thị trực quan kết quả trên bản đồ.

Bộ lọc và dữ liệu tổng hợp được tạo sẵn: Sử dụng bộ lọc và dữ liệu tổng hợp được tạo sẵn của Kibana, bạn có thể chạy các phân tích khác nhau như biểu đồ tần suất, truy vấn top-N và xu hướng chỉ trong vài bước.

Bảng thông tin dễ dàng truy cập: Bạn có thể dễ dàng thiết lập bảng thông tin và báo cáo cũng như chia sẻ chúng với những người khác. Bạn chỉ cần có một trình duyệt để xem và khám phá dữ liệu.

2.5 Grafana và Prometheus.

2.5.1 Grafana

Grafana là một nền tảng mã nguồn mở giúp bạn truy vấn, hiển thị, và cảnh báo dữ liệu từ nhiều nguồn khác nhau. Bạn có thể sử dụng Grafana để theo dõi thông tin theo thời gian thực từ nhiều loại dữ liệu khác nhau, như cơ sở dữ liệu thời gian thực (Time Series Database), SQL, NoSQL, hoặc các dịch vụ cloud thông qua các bảng điều khiển (dashboard) có thể tùy chỉnh.

Grafana trong môi trường Docker Swarm là một công cụ trực quan hóa dữ liệu, thường được sử dụng để giám sát hệ thống, phân tích dữ liệu và xây dựng dashboard thời gian thực. Khi triển khai trong môi trường Docker Swarm, Grafana trở thành một phần rất quan trọng trong hệ sinh thái giám sát, đặc biệt khi kết hợp với các công cụ như Prometheus, InfluxDB, hoặc Loki.

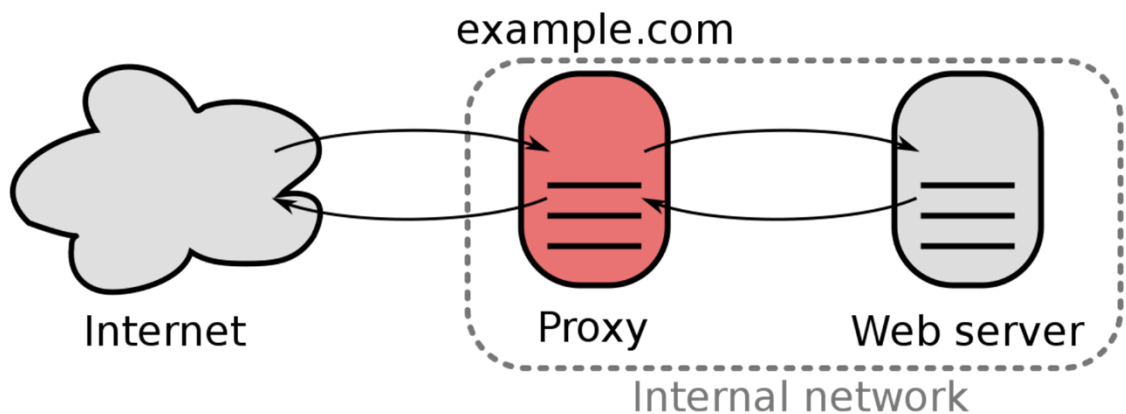
2.5.2 Prometheus

Prometheus là một hệ thống giám sát và cảnh báo mã nguồn mở, được thiết kế để thu thập và xử lý các chỉ số (metrics, không phải logs) từ ứng dụng và hạ tầng. Prometheus hoạt động theo mô hình pull, tức là nó định kỳ gửi yêu cầu HTTP để thu thập dữ liệu từ các mục tiêu được cấu hình.

2.6 .Reverse proxy.

2.6.1 Reverse proxy là gì?

Reverse proxy là một loại proxy server đặt ở phía máy chủ (server) và hoạt động ngược với forward proxy. Khi một người dùng truy cập vào một trang web, yêu cầu của người dùng được gửi đến reverse proxy trước khi được chuyển tiếp đến máy chủ cuối cùng.



2.6.2 Tại sao nên sử dụng Reverse proxy

Sử dụng reverse proxy mang lại nhiều lợi ích cho hệ thống và môi trường web.

Bảo mật: Reverse proxy đóng vai trò làm lớp trung gian giữa người dùng và máy chủ cuối cùng. Nó ẩn địa chỉ IP và thông tin định danh của máy chủ thực tế, giúp bảo vệ máy chủ khỏi các cuộc tấn công trực tiếp và che giấu cấu trúc hệ thống bên trong. Điều này giúp ngăn chặn các cuộc tấn công DDoS, SQL injection, cross-site scripting và nhiều hình thức tấn công khác.

Phân phối tải: Reverse proxy có thể được cấu hình để phân phối tải (load balancing) giữa nhiều máy chủ. Khi có nhiều yêu cầu từ người dùng, reverse proxy sẽ chuyển tiếp yêu cầu đến các máy chủ khác nhau để chia sẻ tải công việc. Điều này giúp cải thiện hiệu suất và khả năng mở rộng của hệ thống, đồng thời giảm nguy cơ quá tải máy chủ.

Caching: Reverse proxy có khả năng lưu trữ (cache) các nội dung phổ biến trên máy chủ. Khi người dùng yêu cầu nội dung đã được lưu trữ trong cache, reverse proxy

có thể phục vụ lại nhanh chóng mà không cần truy cập tới máy chủ cuối cùng. Điều này giúp giảm thời gian tải trang và tăng tốc độ truy cập cho người dùng.

Quản lý và kiểm soát: Reverse proxy cung cấp khả năng quản lý và kiểm soát truy cập vào máy chủ cuối cùng. Bằng cách áp dụng quy tắc chặn truy cập, xác thực người dùng, kiểm soát băng thông và theo dõi hoạt động mạng, reverse proxy giúp hạn chế truy cập trái phép và đảm bảo an ninh hệ thống.

SSL/TLS Offloading: Reverse proxy có thể xử lý quá trình mã hóa và giải mã SSL/TLS. Khi sử dụng reverse proxy, máy chủ cuối cùng không cần phải xử lý việc này, giúp tăng hiệu suất của máy chủ và giảm tải cho nó.

Chuyển hướng (URL routing): Reverse proxy có thể được cấu hình để chuyển hướng yêu cầu từ người dùng đến các máy chủ khác nhau dựa trên các quy tắc cấu hình. Điều này hữu ích khi cần phân chia yêu cầu vào các ứng dụng, dịch vụ hoặc máy chủ riêng biệt dựa trên đường dẫn URL, tên miền hoặc các yêu cầu cụ thể khác.

Tổng quát, sử dụng reverse proxy giúp tăng cường bảo mật, cải thiện hiệu suất, tăng khả năng mở rộng và cung cấp khả năng quản lý truy cập cho hệ thống web.

2.6.3 Ứng dụng của Reverse proxy có thể xảy ra những nguy cơ gì?

Mặc dù reverse proxy mang lại nhiều lợi ích cho hệ thống web, nhưng cũng có một số nguy cơ cần được quan tâm khi triển khai và sử dụng reverse proxy. Dưới đây là một số nguy cơ phổ biến:

Single point of failure:

Nếu reverse proxy gặp sự cố hoặc gặp vấn đề, nó có thể làm gián đoạn hoạt động của toàn bộ hệ thống. Nếu không có các biện pháp dự phòng, điều này có thể dẫn đến gián đoạn truy cập và mất mát dữ liệu.

– Yếu tố hiệu suất:

Reverse proxy có thể làm tăng thời gian đáp ứng và trễ mạng. Mỗi yêu cầu từ người dùng phải đi qua một lớp trung gian thêm, và việc xử lý và chuyển tiếp yêu cầu

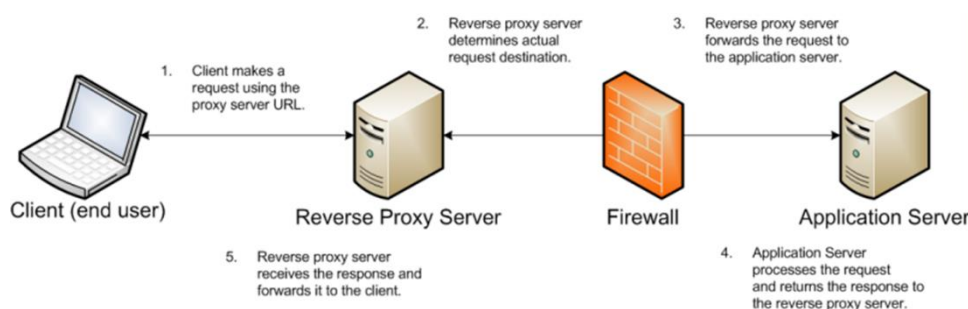
có thể tạo ra một chút trễ thời gian. Điều này có thể ảnh hưởng đến hiệu suất tổng thể của hệ thống.

- Bảo mật và rủi ro:

Sử dụng reverse proxy đặt ra một số vấn đề bảo mật tiềm ẩn. Nếu không được cấu hình và quản lý đúng cách, reverse proxy có thể trở thành điểm yếu của hệ thống và bị tấn công. Cần chú ý đến việc bảo vệ reverse proxy khỏi các cuộc tấn công như DDoS, injection attacks, hoặc các lỗ hổng bảo mật khác.

- Quản lý và khó khăn triển khai:

Triển khai và quản lý reverse proxy có thể phức tạp và đòi hỏi kiến thức kỹ thuật. Cấu hình và duy trì reverse proxy yêu cầu sự hiểu biết về mạng, giao thức và quy tắc cấu hình. Điều này có thể tạo ra thách thức cho các nhà quản lý hệ thống.



- Sự không tương thích:

Một số ứng dụng web hoặc giao thức có thể không tương thích hoặc không hoạt động tốt khi chạy qua reverse proxy. Điều này có thể đòi hỏi phải tùy chỉnh cấu hình hoặc xử lý đặc biệt để đảm bảo tính tương thích và khả năng hoạt động ổn định.

Để giảm thiểu các nguy cơ này, cần thực hiện triển khai và cấu hình reverse proxy một cách cẩn thận, đảm bảo bảo mật, hiệu suất và sẵn sàng của hệ thống. Ngoài ra, việc đảm bảo sự dự phòng và giám sát liên tục của reverse proxy cũng rất quan trọng để đảm bảo sự ổn định và hiệu quả của hệ thống.

2.6.4 Chức năng và ứng dụng

Reverse proxy có nhiều chức năng quan trọng trong một hệ thống. Một trong số đó là load balancing, nghĩa là phân phối tải truy cập từ người dùng đến nhiều máy

chủ để cân bằng tải công việc và tăng cường hiệu suất. Reverse proxy cũng có khả năng caching, tức là lưu trữ các nội dung phổ biến từ máy chủ cuối cùng để phục vụ lại cho các yêu cầu tương tự, giảm thời gian tải trang và tăng tốc độ truy cập. Ngoài ra, reverse proxy cung cấp lớp bảo vệ bổ sung cho máy chủ cuối cùng bằng cách che giấu thông tin và bảo vệ khỏi các cuộc tấn công trực tiếp. Nó cũng cho phép quản lý truy cập và kiểm soát truy cập vào máy chủ cuối cùng.

2.7 Giới thiệu các metric được thu thập trên toàn bộ các node

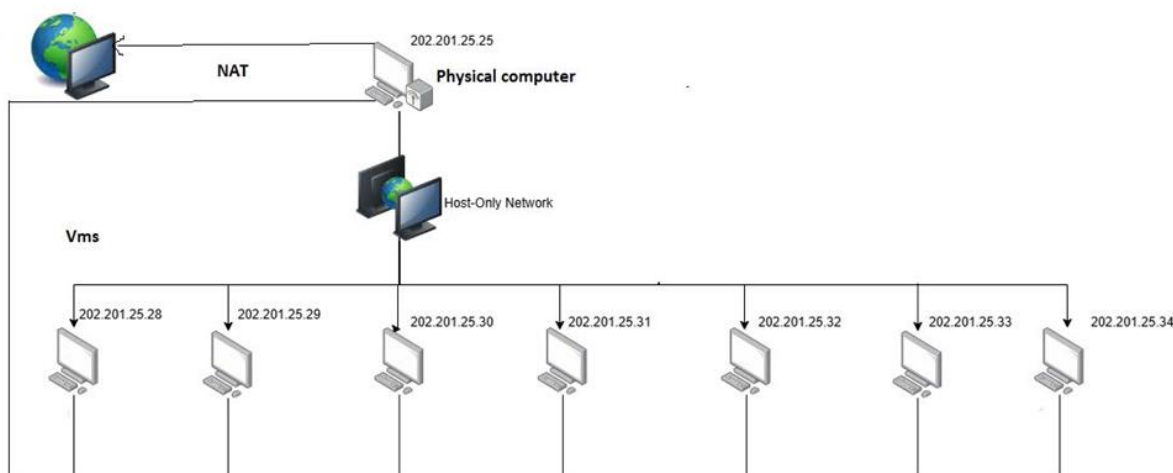
Dưới đây là các metric chính được thu thập từ toàn bộ các node trong hệ thống để giám sát hiệu suất và tài nguyên:

- **CPU Usage:** Theo dõi tỷ lệ sử dụng CPU, bao gồm các tác vụ người dùng và hệ thống.
- **RAM Usage (Active/Inactive Memory):** Giám sát bộ nhớ đã sử dụng và còn lại, bao gồm bộ nhớ active và inactive.
- **Disk Usage:** Dung lượng ổ đĩa đã sử dụng và còn lại, bao gồm các phân vùng và các hoạt động đọc/ghi.
- **Processes and State:** Thông tin về các tiến trình đang chạy và trạng thái của chúng (running, sleeping, etc.).
- **Open Files:** Số lượng file đang mở trên hệ thống.
- **Sockets and States:** Trạng thái của các kết nối mạng (TCP/UDP).
- **I/O Activity:** Các hoạt động I/O của ổ đĩa và mạng, bao gồm số byte đọc/ghi và gửi/nhận qua mạng.
- **Hardware Metrics:** Thông tin về phần cứng vật lý như tốc độ quạt và nhiệt độ CPU.

Các công cụ phổ biến để thu thập các metric này bao gồm: Prometheus, Grafana, node_exporter, cAdvisor.

CHƯƠNG 3: SƠ ĐỒ MẠNG VÀ KIẾN TRÚC HỆ THỐNG

3.7 Sơ đồ mạng



Mô tả sơ đồ : Máy master có thể giao tiếp với các máy worker khác thông qua mạng Host-only Network (các máy worker cũng có thể giao tiếp với nhau thông qua địa chỉ IP của từng máy đã được tạo trong mạng ảo) bằng cách ping qua lại giữa các IP từ các máy ảo . Các máy ảo trong hệ thống cũng có thể truy cập ra ngoài Internet (dùng để tải các gói phần mềm , container ,...) thông qua mạng NAT của một máy chủ vật lý (máy chính) .

3.8 Kiến trúc hệ thống

Trong các hệ thống phân tán hiện đại, việc đảm bảo hiệu suất cao, khả năng mở rộng và độ tin cậy là yếu tố then chốt đối với các ứng dụng, đặc biệt là những ứng dụng đòi hỏi tài nguyên lớn như cryptocurrency mining hoặc xử lý dữ liệu thời gian thực. Kiến trúc này được xây dựng để tập trung vào giải quyết các vấn đề hiệu suất gặp phải trong quá trình triển khai ứng dụng gồm năm service (rng, hasher, worker, Redis, v.v.) trên một node duy nhất. Các vấn đề bao gồm CPU load cao, memory saturation, khả năng mở rộng hạn chế và rủi ro single point of failure. Để giải quyết, chúng em đề xuất sử dụng **Docker Swarm**, một công cụ native orchestration của Docker, nhằm tạo môi trường cluster giúp tăng cường performance, scalability và reliability.

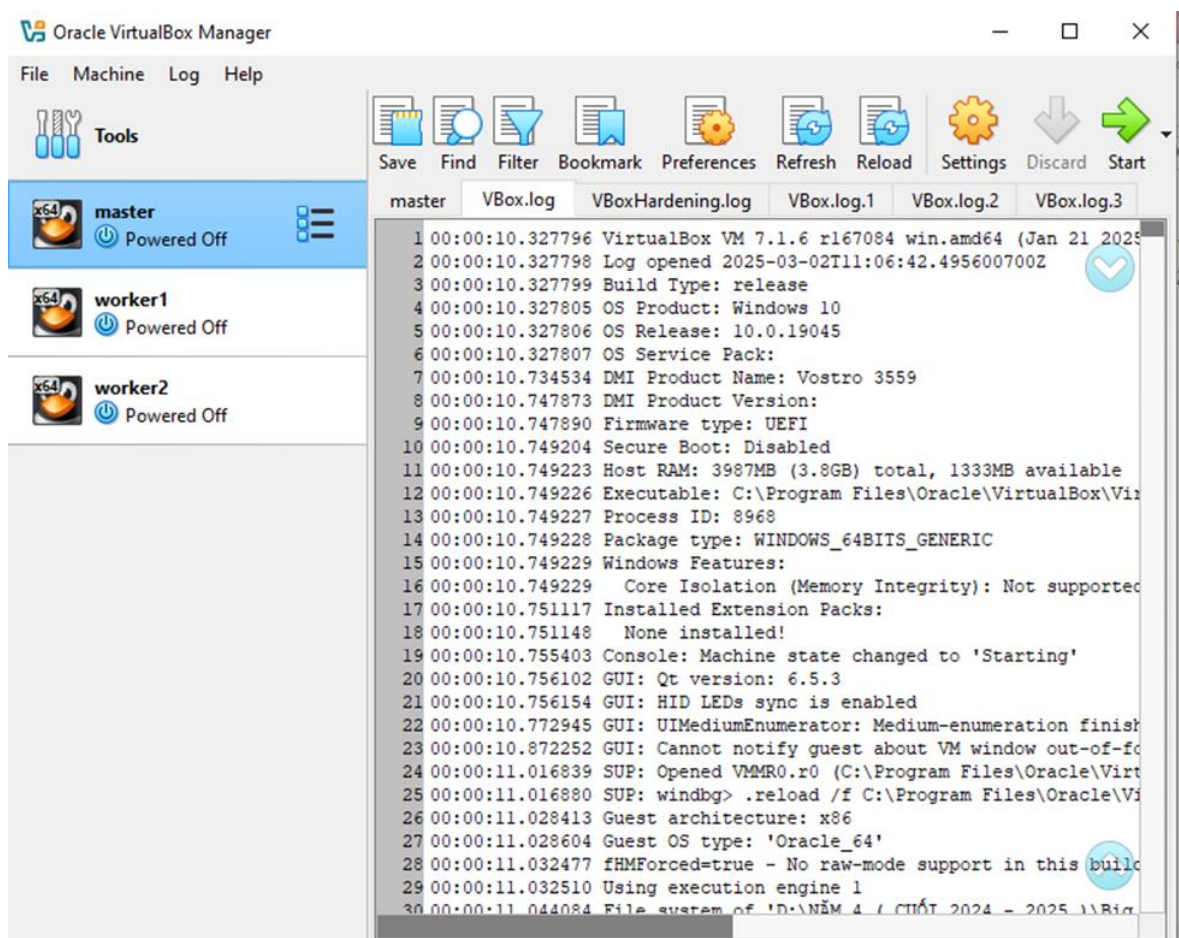
Những thách thức này nhấn mạnh nhu cầu về một kiến trúc phân tán, scalable và fault-tolerant để đảm bảo performance ứng dụng mạnh mẽ.

CHƯƠNG 4: HIỆN THỰC

4.1 Xây dựng LAN network trên môi trường virtual box

4.1.1 Xây dựng được mạng LAN trên môi trường Virtual Box đúng phiên bản các nodes theo yêu cầu

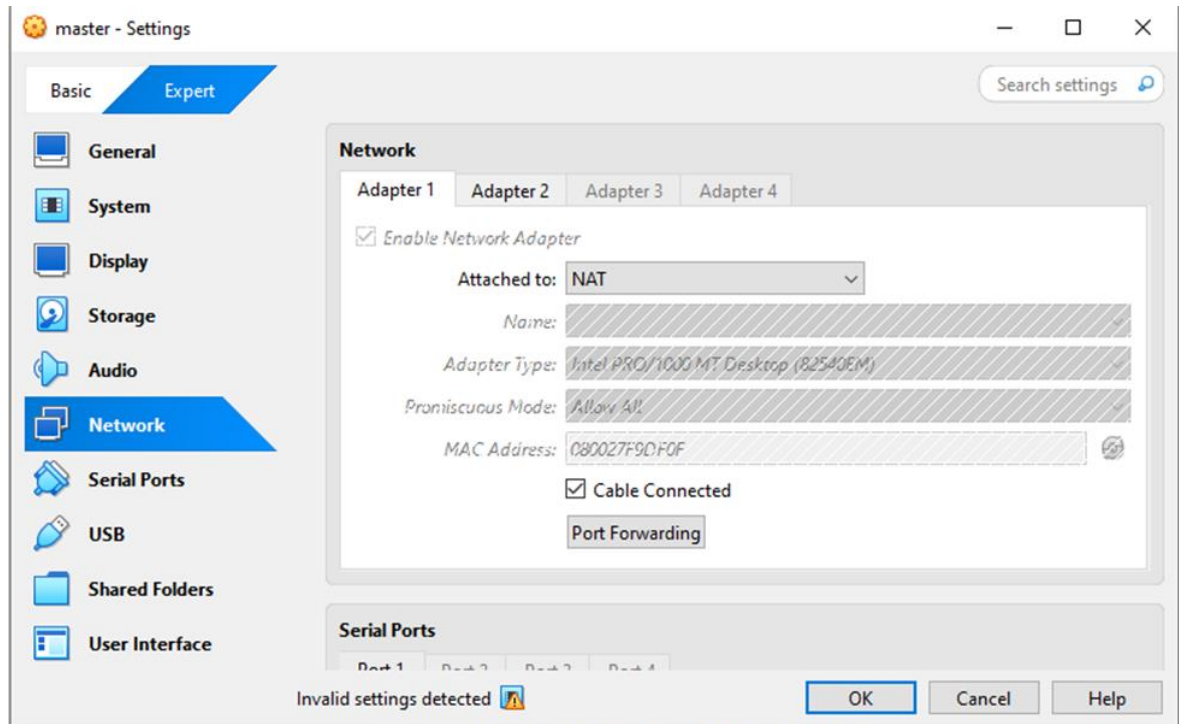
Đầu tiên xây dựng hệ thống máy tính gồm : 1 máy master và 4 máy worker (tượng trưng 1 máy master và 2 máy worker)



Khởi động máy ảo và hoàn thành các cài đặt ssh và docker engine (máy master ở trên đã cài đặt xong và generate MAC ra 2 máy worker 1 và 2)

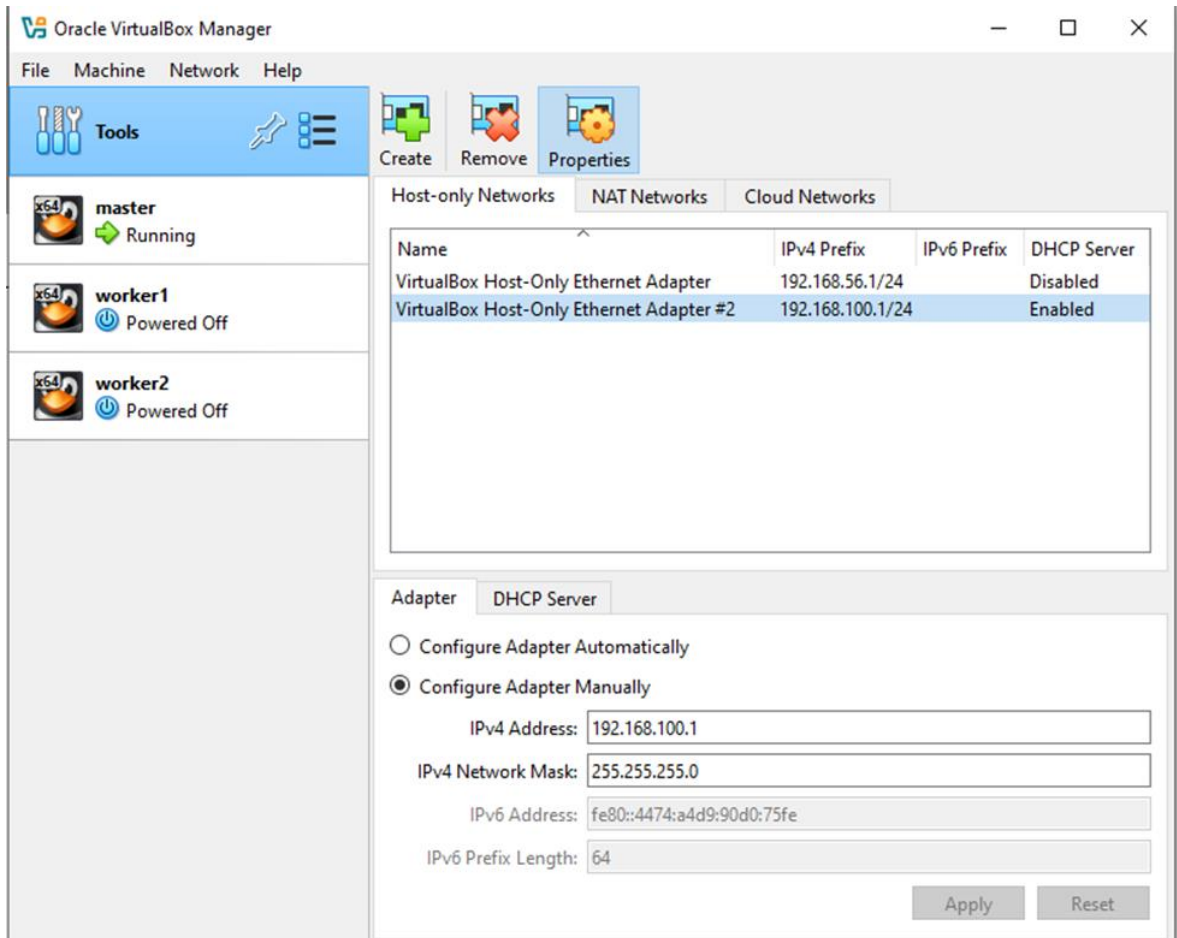
Ở máy master và 2 máy worker vào phần setting để chỉnh lại cài đặt mạng ở adapter 1 là : NAT (Máy ảo nhận chế độ network là NAT (Network Address Translation) thì các máy ảo sẽ có cùng 1 cái ip và có thể kết nối ra ngoài internet thông qua máy

host window (máy thật) thông qua gateway cố định là 10.0.2.2. Tuy nhiên thì từ máy window không thể truy cập vào từng máy ảo vì các máy ảo đã được translate network dẫn đến ip của các máy ảo đều giống nhau nên không thể xác định được và



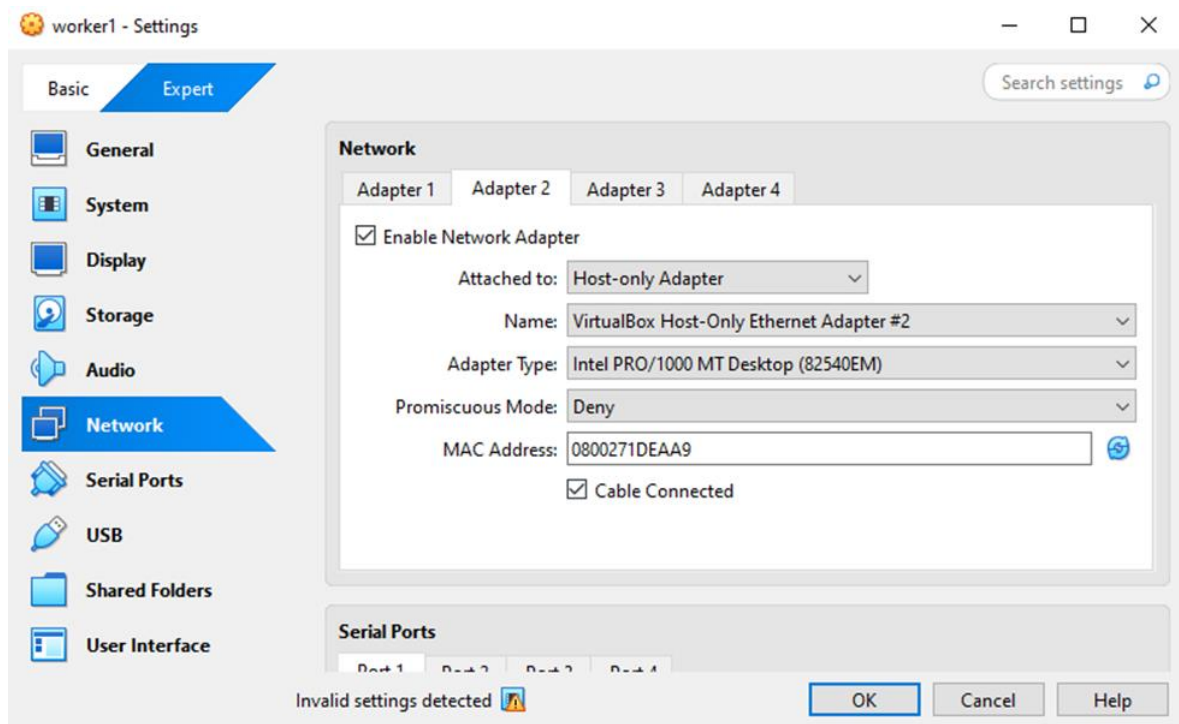
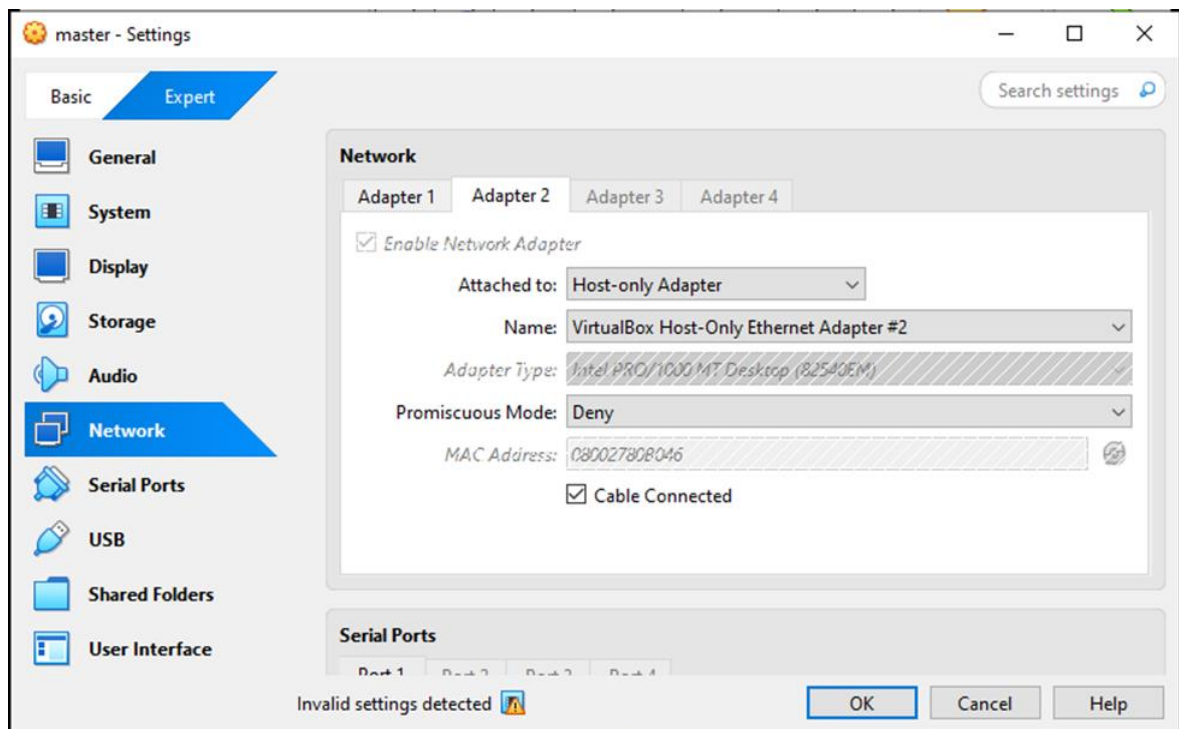
các máy ảo cũng không thể giao tiếp qua lại với nhau (do không có ip riêng biệt))

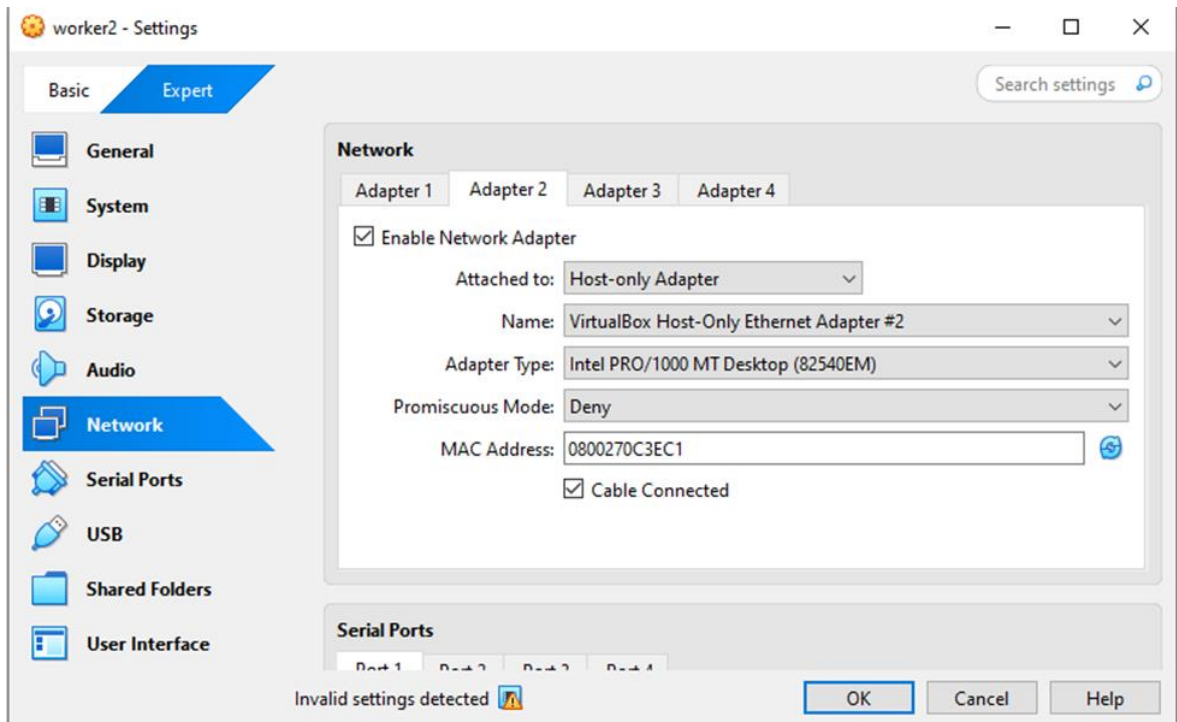
Tạo Host-only network tự động để cài đặt IP theo cấu hình của nhóm : Vào tools



Ở đây có 2 mạng Host-Only network : IP 192.168.56.1 là IP mặc định có sẵn , còn IP 192.168.100.1 là IP có thể tạo tùy theo cấu hình . (Ở đây chọn cấu hình IP là 192.168.100.1)

Tiếp theo ở các máy : master , worker1 , worker2 sẽ cấu hình ở adapter 2 là Virtual host-only Ethernet Adapter #2 (do tạo ra một mạng khác nên có #2)





Khởi động máy master và thực hiện các câu lệnh trở về thư mục gốc :

```

master [running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Ubuntu 24.10 osboxes tty1
osboxes login: osboxes
Password:
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-18-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sun Mar 2 03:07:53 PM UTC 2025

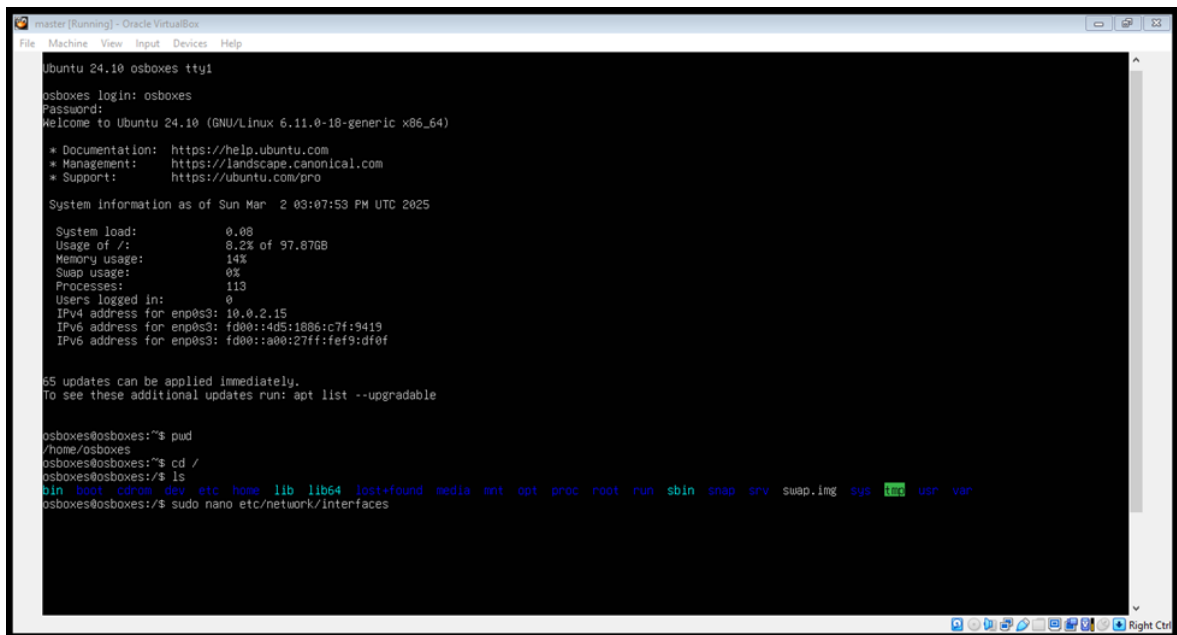
System load:  0.00
Usage of /:   8.2% of 97.87GB
Memory usage: 14%
Swap usage:   0%
Processes:   113
Users logged in: 0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00:14d5:1886:c7f:9419
IPv6 address for enp0s3: fd00:a00:27ff:fef9:df0f

65 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

osboxes@osboxes:~$ pwd
/home/osboxes
osboxes@osboxes:~$ cd /
osboxes@osboxes:/$ ls
bin boot cgroup dev etc home lib lib64 lost+found media mnt opt proc root run/sbin snap srv swap.img sys usr var
osboxes@osboxes:/$ _

```

Dùng lệnh `sudo nano /etc/network/interfaces` để tiến hành thiết lập IP host-only network là 192.168.100.101. Còn các mạng loop back và NAT thì để auto cấp IP bởi DHCP



```
master [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

Ubuntu 24.10 osboxes tty1
osboxes login: osboxes
Password:
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-18-generic x86_64)

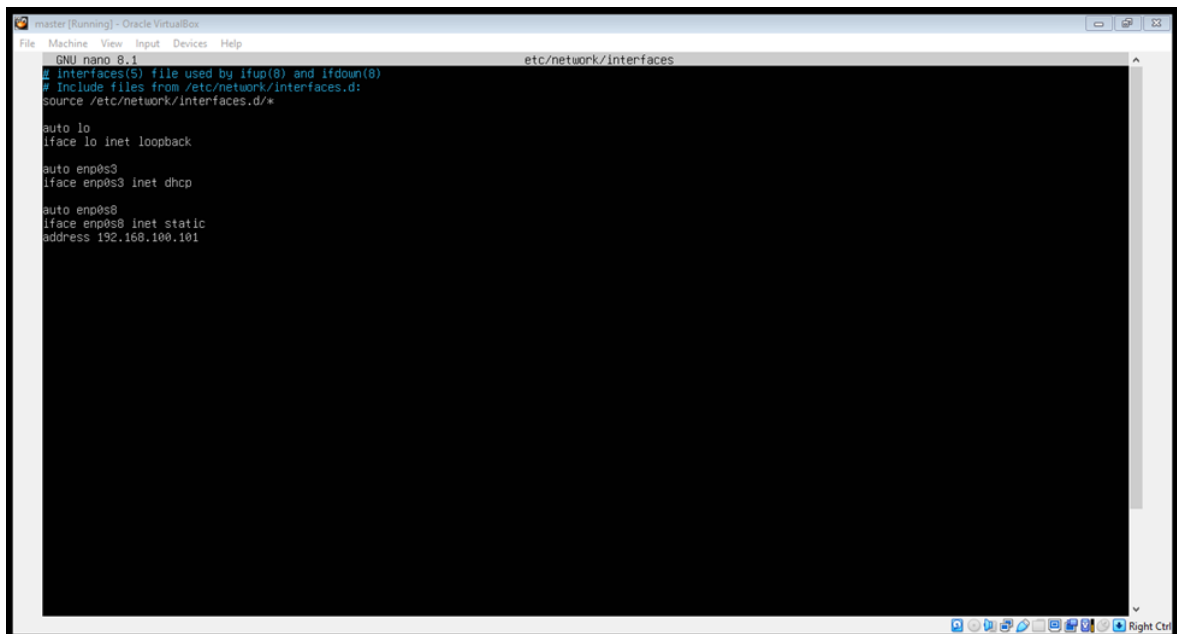
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Mar 2 03:07:53 PM UTC 2025

System load:          0.00
Usage of /:           0.2% of 97.87GB
Memory usage:        14%
Swap usage:          0%
Processes:           113
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00::4d5:1006:c7f:9419
IPv6 address for enp0s3: fd00::a00:27ff:fef9:df0f

65 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

osboxes@osboxes:~$ pwd
/home/osboxes
osboxes@osboxes:~$ cd /
osboxes@osboxes:/$ ls
bin boot cdrom dev etc home lib lib64 lost+found media mnt opt proc root run sbin snap srv swap.img sys usr var
osboxes@osboxes:/$ sudo nano /etc/network/interfaces
```



```
master [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

GNU nano 8.1 /etc/network/interfaces
# interfaces(5) file used by ifup(8) and ifdown(8)
# Include files from /etc/network/interfaces.d:
source /etc/network/interfaces.d/*

auto lo
iface lo inet loopback

auto enp0s3
iface enp0s3 inet dhcp

auto enp0s8
iface enp0s8 inet static
address 192.168.100.101
```

Sau đó Ctrl O -> Enter -> Ctrl X

Sau đó dùng lệnh sudo reboot để khởi động lại máy ảo và cài đặt lại IP

Dùng lệnh để kiểm tra lại IP sau khi đã cài đặt : ip a

```
master [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

osboxes@osboxes:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f9:df:0f brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86261sec preferred_lft 75461sec
    inet6 fd00::4d5:1886:c7f:9419/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86351sec preferred_lft 14351sec
    inet6 fd00::a00:27ff:fe9:df0f/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86292sec preferred_lft 14292sec
    inet6 fe80::a00:27ff:fe9:df0f/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:80:b0:46 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.101/24 brd 192.168.100.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe9:b046/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 06:5e:23:0a:9c:cf brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
5: br-a20ac261e180: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 22:f0:fc:88:ac:f0 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-a20ac261e180
        valid_lft forever preferred_lft forever
osboxes@osboxes:~$
```

Có thể thấy IP sau khi cài đặt là : 192.168.100.101

Làm tương tự với các máy worker1 , worker2 là : 192.168.100.102 và 192.168.100.103

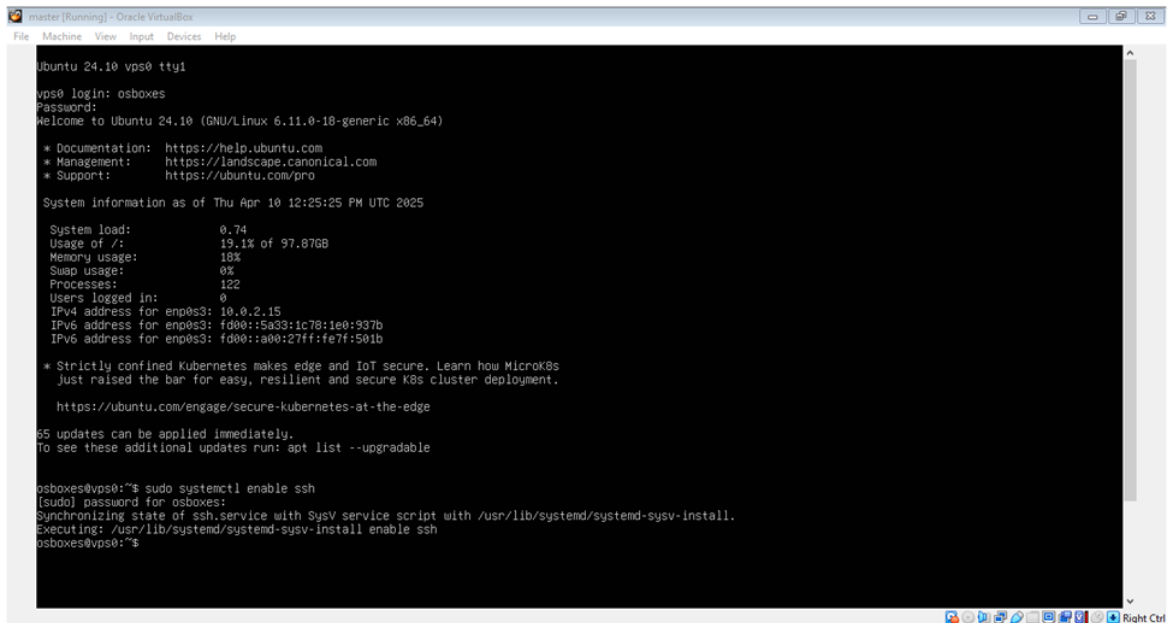
Có thể kiểm tra mạng NAT bằng cách : ping google.com (nếu phản hồi thì là thành công)

```
master [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

osboxes@osboxes:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f9:df:0f brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86261sec preferred_lft 75461sec
    inet6 fd00::4d5:1886:c7f:9419/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86351sec preferred_lft 14351sec
    inet6 fd00::a00:27ff:fe9:df0f/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86292sec preferred_lft 14292sec
    inet6 fe80::a00:27ff:fe9:df0f/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:80:b0:46 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.101/24 brd 192.168.100.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe9:b046/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 06:5e:23:0a:9c:cf brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
5: br-a20ac261e180: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 22:f0:fc:88:ac:f0 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-a20ac261e180
        valid_lft forever preferred_lft forever
osboxes@osboxes:~$ ping google.com
PING google.com (217.160.0.201) 56(84) bytes of data:
64 bytes from 217.160.0.201: icmp_seq=1 ttl=255 time=240 ms
64 bytes from 217.160.0.201: icmp_seq=2 ttl=255 time=257 ms
64 bytes from 217.160.0.201: icmp_seq=3 ttl=255 time=239 ms
64 bytes from 217.160.0.201: icmp_seq=4 ttl=255 time=237 ms
64 bytes from 217.160.0.201: icmp_seq=5 ttl=255 time=239 ms
64 bytes from 217.160.0.201: icmp_seq=6 ttl=255 time=282 ms
```

Tiếp theo kích hoạt ssh bằng lệnh : sudo systemctl enable ssh

Khởi chạy ssh trên server máy ảo : sudo systemctl start ssh



```
master [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

Ubuntu 24.10 vps0 tty1

vps0 login: osboxes
Password:
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-18-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Thu Apr 10 12:25:25 PM UTC 2025

System load:          0.74
Usage of /:            19.1% of 97.87GB
Memory usage:          10%
Swap usage:            0%
Processes:             122
Users logged in:       0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00::5a33:1c78:1e0:937b
IPv6 address for enp0s3: fd00::a00:27ff:fe7f:501b

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.
   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

65 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

osboxes@vps0:~$ sudo systemctl enable ssh
[sudo] password for osboxes:
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
osboxes@vps0:~$
```

Kiểm tra trạng thái ssh : `sudo systemctl status ssh` (hiển thị enable là đã kích hoạt)

```
osboxes@vps0:~$ sudo systemctl status ssh
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-04-12 16:16:35 UTC; 4min 38s ago
   Invocation: 615c2a1321d1497eac087c7825256f7b
   TriggeredBy: • ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 941 (sshd)
     Tasks: 1 (limit: 1886)
    Memory: 144K (peak: 2.5M swap: 1.1M swap peak: 1.1M)
       CPU: 68ms
    CGroup: /system.slice/ssh.service
            └─941 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Apr 12 16:16:33 vps0 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Apr 12 16:16:35 vps0 sshd[941]: Server listening on 0.0.0.0 port 22.
Apr 12 16:16:35 vps0 sshd[941]: Server listening on :: port 22.
Apr 12 16:16:35 vps0 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

Tiếp theo mở cmd (run as administrator)

Gõ lệnh để kiểm tra mạng Host-Only Network : `ping 192.168.100.101`

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>ping 192.168.100.101

Pinging 192.168.100.101 with 32 bytes of data:
Reply from 192.168.100.101: bytes=32 time=1ms TTL=64
Reply from 192.168.100.101: bytes=32 time<1ms TTL=64
Reply from 192.168.100.101: bytes=32 time<1ms TTL=64
Reply from 192.168.100.101: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.100.101:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Windows\system32>
```

Tiến hành ssh vào máy ảo master : ssh osboxes@192.168.100.101

```
osboxes@osboxes: ~
C:\Windows\system32>ssh osboxes@192.168.100.101
osboxes@192.168.100.101's password:
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-18-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Mar  2 04:14:09 PM UTC 2025

System load:          0.16
Usage of /:            8.3% of 97.87GB
Memory usage:         14%
Swap usage:           0%
Processes:            112
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00::4d5:1886:c7f:9419
IPv6 address for enp0s3: fd00::a00:27ff:fef9:df0f

65 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Last login: Sun Mar  2 15:07:54 2025 from 192.168.100.1
osboxes@osboxes:~$
```

Có thể ping qua lại giữa máy master và worker1, worker2

```
worker1 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:a3:24:d0 brd ff:ff:ff:ff:ff:ff
inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
    valid_lft 86279sec preferred_lft 75479sec
inet6 fd00::9f27:36b5:1a22:984a/64 scope global dynamic mngtmpaddr noprefixroute
    valid_lft 86278sec preferred_lft 14278sec
inet6 fd00::a00:27ff:fea3:24d0/64 scope global dynamic mngtmpaddr noprefixroute
    valid_lft 86278sec preferred_lft 14278sec
inet6 fe80::a00:27ff:fea3:24d0/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:99:5d:40 brd ff:ff:ff:ff:ff:ff
inet 192.168.100.102/24 brd 192.168.100.255 scope global enp0s8
    valid_lft forever preferred_lft forever
inet6 fe80::a00:27ff:fe99:5d40/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
4: dockero: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
link/ether 6a:a2:34:13:0d:70 brd ff:ff:ff:ff:ff:ff
inet 172.17.0.1/16 brd 172.17.255.255 scope global dockero
    valid_lft forever preferred_lft forever
5: br-a20ac261e100: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
link/ether 4a:3d:38:65:36:fd brd ff:ff:ff:ff:ff:ff
inet 172.18.0.1/16 brd 172.18.255.255 scope global br-a20ac261e100
    valid_lft forever preferred_lft forever
osboxes@osboxes:~$ ping 192.168.100.101
PING 192.168.100.101 (192.168.100.101) 56(84) bytes of data:
64 bytes from 192.168.100.101: icmp_seq=1 ttl=64 time=2.72 ms
64 bytes from 192.168.100.101: icmp_seq=2 ttl=64 time=0.387 ms
64 bytes from 192.168.100.101: icmp_seq=3 ttl=64 time=0.366 ms
64 bytes from 192.168.100.101: icmp_seq=4 ttl=64 time=0.396 ms
64 bytes from 192.168.100.101: icmp_seq=5 ttl=64 time=0.414 ms
64 bytes from 192.168.100.101: icmp_seq=6 ttl=64 time=0.373 ms
64 bytes from 192.168.100.101: icmp_seq=7 ttl=64 time=0.412 ms
64 bytes from 192.168.100.101: icmp_seq=8 ttl=64 time=0.476 ms
64 bytes from 192.168.100.101: icmp_seq=9 ttl=64 time=0.527 ms
64 bytes from 192.168.100.101: icmp_seq=10 ttl=64 time=0.560 ms
64 bytes from 192.168.100.101: icmp_seq=11 ttl=64 time=0.434 ms
64 bytes from 192.168.100.101: icmp_seq=12 ttl=64 time=0.383 ms
64 bytes from 192.168.100.101: icmp_seq=13 ttl=64 time=0.449 ms
64 bytes from 192.168.100.101: icmp_seq=14 ttl=64 time=0.409 ms
```

SSH vào worker1 và ping vào máy master

```
osboxes@osboxes: ~
IPv6 address for enp0s3: fd00::9f27:36b5:1a22:984a
IPv6 address for enp0s3: fd00::a00:27ff:fea3:24d0

65 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Last login: Sun Mar  2 16:14:11 2025 from 192.168.100.1
osboxes@osboxes:~$ ping 192.168.100.101
PING 192.168.100.101 (192.168.100.101) 56(84) bytes of data:
64 bytes from 192.168.100.101: icmp_seq=1 ttl=64 time=0.497 ms
64 bytes from 192.168.100.101: icmp_seq=2 ttl=64 time=0.403 ms
64 bytes from 192.168.100.101: icmp_seq=3 ttl=64 time=0.416 ms
64 bytes from 192.168.100.101: icmp_seq=4 ttl=64 time=0.387 ms
64 bytes from 192.168.100.101: icmp_seq=5 ttl=64 time=0.420 ms
64 bytes from 192.168.100.101: icmp_seq=6 ttl=64 time=0.567 ms
64 bytes from 192.168.100.101: icmp_seq=7 ttl=64 time=0.533 ms
^C
--- 192.168.100.101 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6154ms
rtt min/avg/max/mdev = 0.387/0.460/0.567/0.065 ms
```

Vậy là đã hoàn thành xây dựng mạng LAN theo yêu cầu

4.2 Docker swarm.

4.2.1 Cấu hình Docker Swarm

- Khi chạy lệnh “sudo docker swarm”. Docker đang báo rằng chưa cung cấp đầy đủ **subcommand** (lệnh con). Tức là đang gọi docker swarm nhưng chưa nói rõ muốn làm gì với nó. Vì vậy, nó liệt kê ra các lệnh con (subcommands) có thể sử dụng với docker swarm.

```

osboxes@vps0:~$ sudo docker swarm
[sudo] password for osboxes:
Usage:  docker swarm COMMAND

Manage Swarm

Commands:
  ca           Display and rotate the root CA
  init         Initialize a swarm
  join         Join a swarm as a node and/or manager
  join-token   Manage join tokens
  leave        Leave the swarm
  unlock       Unlock swarm
  unlock-key   Manage the unlock key
  update       Update the swarm

Run 'docker swarm COMMAND --help' for more information on a command.

```

Lệnh con	Mô tả
init	Khởi tạo swarm (biến máy hiện tại thành manager đầu tiên trong swarm).
join	Cho máy hiện tại tham gia vào một swarm đã có sẵn (làm worker hoặc manager).
join-token	Hiển thị hoặc tạo lại token để các node khác tham gia swarm.
leave	Rời khỏi swarm.
ca	Hiển thị hoặc xoay vòng (rotate) chứng chỉ CA (Certificate Authority).
unlock	Mở khóa swarm (nếu swarm được bảo vệ bằng mật khẩu).
unlock-key	Quản lý khóa mở khóa swarm.
update	Cập nhật cấu hình swarm.

- “sudo docker node” :

```

osboxes@vps0:~$ sudo docker node
Usage:  docker node COMMAND

Manage Swarm nodes

Commands:
  demote    Demote one or more nodes from manager in the swarm
  inspect   Display detailed information on one or more nodes
  ls        List nodes in the swarm
  promote   Promote one or more nodes to manager in the swarm
  ps        List tasks running on one or more nodes, defaults to current node
  rm        Remove one or more nodes from the swarm
  update    Update a node

Run 'docker node COMMAND --help' for more information on a command.

```

Lệnh	Mô tả
ls	Liệt kê tất cả các node trong swarm.
inspect	Xem thông tin chi tiết của 1 hoặc nhiều node.
ps	Xem các container đang chạy (nhiệm vụ) trên một node.
promote	Nâng cấp node từ worker → manager.
demote	Hạ cấp node từ manager → worker.
rm	Xóa node khỏi swarm.
update	Cập nhật thông tin của node (như gán nhãn, gán role...).

- “sudo docker service”:

```
osboxes@vps0:~$ sudo docker service
Usage:  docker service COMMAND

Manage Swarm services

Commands:
  create      Create a new service
  inspect     Display detailed information on one or more services
  logs       Fetch the logs of a service or task
  ls         List services
  ps         List the tasks of one or more services
  rm         Remove one or more services
  rollback   Revert changes to a service's configuration
  scale      Scale one or multiple replicated services
  update     Update a service
```

Lệnh	Mô tả
create	Tạo một service mới.
ls	Liệt kê tất cả các service.
inspect	Xem thông tin chi tiết về một hoặc nhiều service.
ps	Xem danh sách các container (task) đang chạy trong service.
logs	Xem log của một service hoặc task.
update	Cập nhật service (thay đổi số lượng replicas, image, command...).
scale	Tăng hoặc giảm số lượng bản sao (replicas) của service.
rollback	Quay lại cấu hình cũ nếu update thất bại hoặc không mong muốn.
rm	Xóa service.

- Khi chạy lệnh “sudo docker node ls” thì hệ thống báo lỗi do máy hiện tại chưa tham gia swarm hoặc không phải là manager node.

```
osboxes@osboxes:~$ sudo docker node ls
Error response from daemon: This node is not a swarm manager. Use "docker swarm init" or "docker swarm join" to connect this node to swarm and try again.
```

- Khi chạy “sudo docker swarm init” hệ thống báo có nhiều địa chỉ IP (mạng NAT và Host-Only trong VirtualBox), nên Docker không biết dùng địa chỉ IP nào để quảng bá (advertise) tới các node khác trong swarm.

```
osboxes@osboxes:~$ sudo docker swarm init
Error response from daemon: could not choose an IP address to advertise since this system has multiple
addresses on different interfaces (10.0.2.15 on enp0s3 and 192.168.56.101 on enp0s8) - specify one with
--advertise-addr
```

- Không nên sử dụng mạng NAT để xây dựng cụm Docker Swarm là vì NAT không hỗ trợ kết nối trực tiếp giữa các máy ảo, Docker Swarm yêu cầu các node phải giao tiếp được trực tiếp với nhau qua địa chỉ IP.

- Bước tiếp theo, ta chọn mạng Host-only-network để xây dựng swarm cluster với cổng mặc định trên từng node dùng giao tiếp giữa các node trong swarm là 2377. Với câu lệnh bên dưới “sudo docker swarm init --advertise-addr 192.168.56.101:2377” thì đã tạo ra docker swarm với máy master 01 là máy có vai trò quản lý(master hay leader) trong cụm docker swarm.

```
osboxes@osboxes:~$ sudo docker swarm init --advertise-addr 192.168.56.101:2377
Swarm initialized: current node (m8we3e4b19ua0uia4u75nurpe) is now a manager.

To add a worker to this swarm, run the following command:

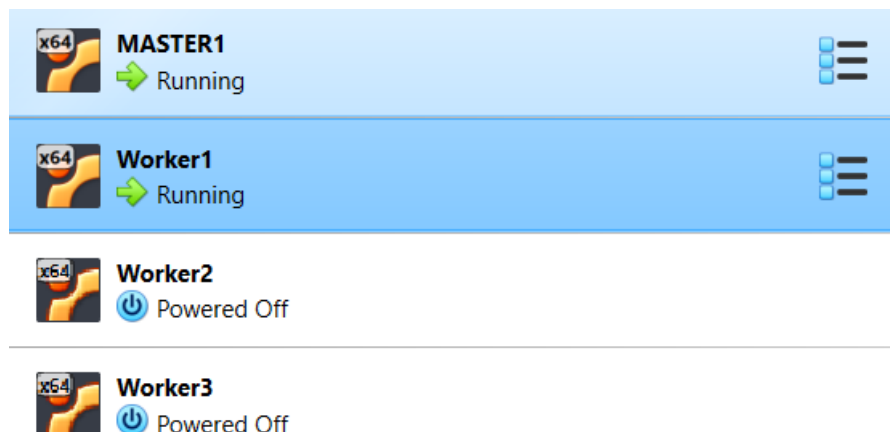
    docker swarm join --token SWMTKN-1-67kk2eectclaxvertysk84q0wd30d5kquvvkn8trfe4qa92rt-ct05v0uspn66o1y5wr8zrbyjh 192.168.56.101:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

- Ta phải lưu đoạn “docker swarm join –token...” để sau này thêm các node vào cụm thì cần đến chúng.

- Để thêm các node khác vào cụm docker swarm thì:

+ Chúng ta bật các máy cần thêm



+ Kiểm tra đã có dịch vụ ssh chưa

+ Sau đó dùng câu lệnh để thêm lần lượt các máy vào cụm docker swarm: “docker swarm join –token (token ở đây là lúc khởi tạo máy leader có xuất hiện đoạn token, lưu ý phải lưu lại để thêm các máy khác vào cụm cần token này)”

```
osboxes@osboxes:~$ sudo docker swarm join --token SWMTKN-1-67kk2eectuciaxvertysk84q0wd30d5kquvvkn8tr
fe4qa92rt-ct05v0uspn66o1y5wr8zrbyjh 192.168.56.101:2377
This node joined a swarm as a worker.
```

- Sau khi lần lượt thêm các máy vào cụm docker swarm xong, ta gõ lệnh “sudo docker info” để xem thông tin chi tiết về cụm:

```
ClusterID: ixcnwkdcvvgwn8op81a4gqzyk
Managers: 3
Nodes: 5
Default Address Pool: 10.0.0.0/8
```

- + Hình này cho thấy, trong cụm đã được thêm tổng cộng 5 node, trong đó có 3 node đóng vai trò là manager, 2 node còn lại đóng vai trò worker.
- Lượt xuống dưới, ta thấy địa chỉ của node đang thực thi lệnh là 192.168.56.101, các địa chỉ của manager lần lượt là 192.168.56.101, 192.168.56.102, 192.168.56.103 với cổng 2377 như đã nói ở trên.

```
Node Address: 192.168.56.101
Manager Addresses:
192.168.56.101:2377
192.168.56.102:2377
192.168.56.103:2377
```

- Tiếp tục gõ lệnh “sudo docker node ls” để xem thông tin các node:

```
osboxes@vps0:~$ sudo docker node ls
ID                HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
qpacmz0925otnvj3ae33gednq * vps0            Ready     Drain            Leader             28.0.2
9x0ojbf0mnrvt8hyy7pw9by3 vps1            Ready     Active           Reachable          28.0.1
b89cr88cci7vvr1on20h0z7av vps2            Down      Active           Unreachable        28.0.1
xximnhkt9w2m7midabcf7d6 vps3            Down      Active           28.0.1
sl654s5vrqqbazn7lri0senb6 vps4            Down      Active           28.0.1
```

Cột	Ý nghĩa
HOSTNAME	Tên định danh của node trong Swarm (tên máy).
STATUS	Trạng thái hiện tại của node: - Ready: node đang kết nối tốt với Swarm. - Down: node không phản hồi, có thể tắt máy.
AVAILABILITY	Mức độ sẵn sàng của node để nhận task: - Active: node sẵn sàng nhận task (container).

	- Pause: node tạm dừng nhận task mới. - Drain: node không nhận task, các task hiện có sẽ chuyển sang node khác.
MANAGER STATUS	Chỉ hiển thị nếu node là manager: - Leader: node manager chính. - Reachable: manager phụ, hoạt động tốt. - Unreachable: manager phụ đang mất kết nối. - (trống): node là worker, không có vai trò quản lý.
ENGINE VERSION	Phiên bản Docker Engine đang chạy trên node đó.

- Như vậy, với các bước ở trên, ta đã tự tạo được cụm docker swarm gồm 3 node manager và 2 node worker. Để quản trị các node trên swarm, ta thường dùng các câu lệnh:

- + Node worker muốn rời khỏi swarm : “docker swarm leave”(làm trên máy worker) hoặc “sudo docker node rm (tên của máy woker muốn xóa)” lưu ý lệnh này chỉ làm = được trên một node đóng vai tròn manager.
- + Node manager muốn rời khỏi swarm : sudo docker swarm leave -force

4.2.2 Kiểm tra CPU / memory và I/O Usage của một node

- Đầu tiên dùng lệnh top để xem tổng dung lượng, % cpu của từng node

```

osboxes@vps0: ~
top - 14:58:54 up 2:11, 3 users, load average: 0.00, 0.01, 0.00
Tasks: 107 total, 2 running, 101 sleeping, 4 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.3 sy, 0.0 ni, 99.3 id, 0.0 wa, 0.0 hi, 0.3 si, 0.0 st
MiB Mem : 3403.8 total, 2558.9 free, 499.1 used, 560.1 buff/cache
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used. 2904.8 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 351 root        -11   0 287336 25508 6948 S   0.3   0.7   0:00.69 multipathd
 980 root        20   0 1957460 109980 63712 S   0.3   3.2   0:46.72 dockerd
3363 root        20   0      0      0      0 I   0.3   0.0   0:00.44 kworker/0:3-events
    1 root        20   0 22544 14360 10264 S   0.0   0.4   0:02.25 systemd
    2 root        20   0      0      0      0 S   0.0   0.0   0:00.02 kthreadd
    3 root        20   0      0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root         0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root         0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
    6 root         0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushng

```

- Sau đó trên máy manager có thể dùng lệnh sudo docker node inspect để xem CPU, Memory của nodes khác bằng cách dùng node ls trước để kiểm tra id

```
[5]+ Stopped top
osboxes@vps0:~$ sudo docker node ls
[sudo] password for osboxes:
ID                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
vc2swvyga9nq72e5cb4n5bdng    vp1        Ready    Active
s4evgb9qvmldr6mdad7g0ifrw *  vps0      Ready    Drain          Leader           27.5.1
r6fnhfo9yk2k5jr1up3njrmbv    vps2      Ready    Active          Reachable        27.5.1
m8djdgm6tmzlp0ss5cgu8au     vps3      Down     Active          Reachable        27.5.1
osboxes@vps0:~$
```

- Sau đó dùng lệnh `sudo docker node inspect r6fnhfo9yk2k5jr1up3njrmbv --pretty` của máy vps2

```
osboxes@vps0:~$ sudo docker node inspect r6fnhfo9yk2k5jr1up3njrmbv --pretty
ID: r6fnhfo9yk2k5jr1up3njrmbv
Hostname: vps2
Joined at: 2025-03-02 14:08:48.044816454 +0000 utc
Status:
  State: Ready
  Availability: Active
  Address: 192.168.56.103
Manager Status:
  Address: 192.168.56.103:2377
  Raft Status: Reachable
  Leader: No
Platform:
  Operating System: linux
  Architecture: x86_64
Resources:
  CPUs: 1
  Memory: 961.1MiB
Plugins:
  Log: awslogs, fluentd, gcplogs, gelf, journald, json-file, local, splunk, syslog
  Network: bridge, host, ipvlan, macvlan, null, overlay
  Volume: local
Engine Version: 27.5.1
TLS Info:
TrustRoot:
```

- Lệnh này giúp theo dõi mức sử dụng CPU, RAM, I/O của các container đang chạy trên node hiện tại:

```

. Please run this command on a manager node or promote the current node to a manager.
osboxes@vp1:~$ top
top - 13:39:44 up 52 min,  3 users,  load average: 0.00, 0.04, 0.09
Tasks: 127 total,   1 running, 114 sleeping, 12 stopped,   0 zombie
%Cpu(s):  0.3 us,   0.3 sy,   0.0 ni, 99.3 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  961.1 total,   142.2 free,   492.0 used,   474.2 buff/cache
CONTAINER ID   NAME                                CPU %     MEM USAGE / LIMIT     MEM %     NET I/O     BLOCK I/O
PIDS
a75d501a309b   worker.4.dkaf13a4se9x3bb914ap4c1cr 0.00%     19.09MiB / 961.1MiB    1.99%     4.27kB / 252B    5.23MB /
8.7MB 1
e82cc2f9c026   rng.1.jso5bdao1nv3id42ytpymjagt     0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
cc3c78348c1b   worker.6.ncwocz5okn3zmqco1jvtpdh1j 0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
f120a79435ff   webui.1.yiyo4omz6e1g1rqlwg4fqtd3   0.00%     28.36MiB / 961.1MiB    2.95%     2.04kB / 378B    45.7MB /
1.37MB 10
431f5bcbeebc   hasher.1.sdc2tv8tt4u9tzucaxpbkjs0j 0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
10fd75fce89a   worker.9.z2sq4uk5uxzuzcfb37ako7x   0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
7f81b45024c9   worker.10.yhmf4jlao6854cich7ajivi1f 0.00%     20.24MiB / 961.1MiB    2.11%     2.78kB / 252B    1.25MB /
4.43MB 1
e2540ee826a1   worker.8.c36gzyzu4l6aw6f0ihrqvqr7 0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
37804fa54c3c   worker.3.u276l914zmqqaakjiafskf9tf 0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
318d4aee0a81   redis.1.rnyhia7c8p7qbnhh4w0ljklva 0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
528781d499c5   worker.7.jig4qna31pbfcqo2h86a9zdhm 0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
2ab05551eb32   worker.2.twc7n3e7vihnmap9sjiylr04w 0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B
0
bec7c8468fe4   worker.5.e1k5cs4vrhx2kknwnwtekr13h 0.00%     23.71MiB / 961.1MiB    2.47%     2.11kB / 252B    5.33MB /
2.34MB 1
d8bb51a48b26   worker.1.hsblon2sm23v99mxbmd1h069z 0.00%     0B / 0B          0.00%     0B / 0B        0B / 0B

```

- Kết luận: Các container khác có mức sử dụng tài nguyên tương tự, với mức sử dụng CPU thấp hoặc không đáng kể, sử dụng bộ nhớ khoảng 2.50% đến 2.85% của giới hạn, và có 1 tiến trình đang chạy. Dựa trên dữ liệu, các container đang hoạt động với mức sử dụng tài nguyên thấp, không có dấu hiệu quá tải hoặc sử dụng tài nguyên bất thường. Tuy nhiên, việc giám sát liên tục là cần thiết để đảm bảo hiệu suất và phát hiện sớm các vấn đề tiềm ẩn.
- Sử dụng lệnh sudo docker stats để xem tổng quan các container có chứa trong swarm cùng các chỉ số hiển thị như %CPU, Mem%, NET I/O

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
ec47cbed64e2	redis.1.xktnbbnxh5wlpd190v314zgp	0.28%	3.875MiB / 3.324GiB	0.11%	800B / 0B	28.8MB / 0B	6
c98e1b207626	logstash.1.ulsldhmps9bf0h2oro0lqsrj	31.43%	338.5MiB / 3.324GiB	9.94%	3.06kB / 1.61kB	144MB / 75.1MB	15
74f67fc39d9e	grafana.1.c3vikxsgmj0r5s7qbvrnlfcvc	0.05%	37.96MiB / 3.324GiB	1.12%	19.2kB / 3.4kB	274MB / 65.6MB	9
8535c21a99d6	logstash.2.xt67ks1tdjsju3pl47ayrddnd	31.78%	361.7MiB / 3.324GiB	10.62%	2.99kB / 1.61kB	88.2MB / 33.9MB	15
ec2e9799623f	elasticsearch.1.Shuq6csfnglue8va9hntgonyn	32.23%	1.561GiB / 3.324GiB	46.97%	964B / 0B	276MB / 367MB	39
7134f330bf31	kibana.1.cxp80ke1ke5ngbkefazdqiflp	27.33%	324.8MiB / 3.324GiB	9.54%	964B / 0B	390MB / 3.27MB	12

- CPU % càng cao nghĩa là container tiêu thụ CPU nhiều hơn.

- **MEM USAGE / LIMIT** và **MEM %** dùng để đánh giá container có thể đang thiếu bộ nhớ hay không.
- **NET I/O** cho biết dung lượng dữ liệu vào/ra mạng, giúp kiểm tra container có băng thông lớn hay không.
- **BLOCK I/O** thể hiện hoạt động đọc/ghi trên ổ cứng hoặc volume, hữu ích để giám sát mức I/O đĩa.
- **PIDS** là số process bên trong container, quan sát để tránh tràn PID hoặc container chạy quá nhiều tiến trình.

4.3 Services va high availability.

4.3.1 Scale Services

- Trước khi scale, cần đảm bảo rằng ít nhất **1 replica** của mỗi dịch vụ đang chạy bình thường. Điều này giúp tránh lỗi khi triển khai số lượng lớn replicas cùng lúc.

- Dùng lệnh docker service ls để kiểm tra

```

root@vps0:/home/osboxes# docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
1f67n4tr6mp      coinswarm-proxy     replicated          1/1                  nginx:latest        *:8080->80/tcp
wq9abx12grwn     elasticsearch       replicated          1/1                  elasticsearch:8.16.5 *:9200->9200/tcp
85dh99txeipl     grafana             replicated          1/1                  grafana/grafana:latest *:3000->3000/tcp
9fv1q7ag4m9j     hashier             replicated          2/1                  minhb3187/hashier:coinswarm *:8002->80/tcp
tejbvqtvlyi     influxdb            replicated          1/1                  influxdb:1.8         *:8086->8086/tcp
js8cffi1ni9     kibana              replicated          1/1                  kibana:8.16.5        *:5601->5601/tcp
p2lc8gex5m1q     logstash            replicated          1/1                  logstash:8.16.5      *:5000->5000/tcp
ffzbd1n68er4     node-exporter       global             2/2                  prom/node-exporter:latest *:9090->9090/tcp
e6mo7zovpwz2     prometheus          replicated          1/1                  prom/prometheus:latest *:9090->9090/tcp
ungc693ktgt4     redis               replicated          1/1                  redis:latest
refvhhxnibd4     rng                 replicated          1/1                  minhb3187/rng:coinswarm *:8001->80/tcp
111e88qu4nr9     webui               replicated          1/1                  minhb3187/webui:coinswarm *:8000->80/tcp
114pdwc513n4     worker              replicated          1/1                  minhb3187/worker:coinswarm

```

- Lệnh này liệt kê tất cả các dịch vụ đang chạy trong Docker Swarm.
- Kiểm tra xem các dịch vụ **worker**, **rng**, **hashier**, **webui**, **redis** đều có ít nhất **1 replica** ở trạng thái **Running**.

- Nếu có dịch vụ nào chưa chạy, cần khắc phục trước khi tiếp tục.
- Sau khi đảm bảo các dịch vụ cơ bản đang hoạt động, ta sẽ **tăng số lượng replicas** theo yêu cầu.
- Khi đảm bảo được thì chúng ta sẽ bắt đầu scale số lượng service lên theo đúng yêu cầu
- Đầu tiên ta scale 15 worker với lệnh sau: `docker service scale worker=15`

```

root@vps0: /home/osboxes
root@vps0:/home/osboxes# docker service scale worker=15
worker scaled to 15
overall progress: 15 out of 15 tasks
1/15: running [=====]
2/15: running [=====]
3/15: running [=====]
4/15: running [=====]
5/15: running [=====]
6/15: running [=====]
7/15: running [=====]
8/15: running [=====]
9/15: running [=====]
10/15: running [=====]
11/15: running [=====]
12/15: running [=====]
13/15: running [=====]
14/15: running [=====]
15/15: running [=====]
verify: Service worker converged
root@vps0:/home/osboxes#

```

- Sau đó ta scale 10 rng với lệnh sau: `docker service scale rng=10`

```

root@vps0: /home/osboxes
root@vps0:/home/osboxes# docker service scale rng=10
rng scaled to 10
overall progress: 10 out of 10 tasks
1/10: running [=====]
2/10: running [=====]
3/10: running [=====]
4/10: running [=====]
5/10: running [=====]
6/10: running [=====]
7/10: running [=====]
8/10: running [=====]
9/10: running [=====]
10/10: running [=====]
verify: Service rng converged
root@vps0:/home/osboxes#

```

- scale 10 hasher với lệnh sau: `docker service scale hasher=10`

```
root@vps0:/home/osboxes
root@vps0:/home/osboxes# docker service scale hasher=10
hasher scaled to 10
overall progress: 10 out of 10 tasks
1/10: running [=====]
2/10: running [=====]
3/10: running [=====]
4/10: running [=====]
5/10: running [=====]
6/10: running [=====]
7/10: running [=====]
8/10: running [=====]
9/10: running [=====]
10/10: running [=====]
verify: Service hasher converged
root@vps0:/home/osboxes#
```

- scale webui với lệnh sau: docker service scale webui=5

```
root@vps0:/home/osboxes
root@vps0:/home/osboxes# docker service scale webui=5
webui scaled to 5
overall progress: 5 out of 5 tasks
1/5: running [=====]
2/5: running [=====]
3/5: running [=====]
4/5: running [=====]
5/5: running [=====]
verify: Service webui converged
root@vps0:/home/osboxes#
```

- scale redis với lệnh sau: docker service scale redis=2

```
root@vps0:/home/osboxes# docker service scale redis=2
redis scaled to 2
overall progress: 2 out of 2 tasks
1/2: running [=====>]
2/2: running [=====>]
verify: Service redis converged
root@vps0:/home/osboxes#
```

- Sau đó kiểm tra số lượng dịch vụ

```
root@vps0:/home/osboxes# docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
1fo7r4xtr6mp      coinswarm-proxy      replicated          1/1                  nginx:latest        *:8000->80/tcp
wp9abx12grun       elasticsearch        replicated          2/1                  elasticsearch:8.16.5 *:9200->9200/tcp
65dn99txdip1       grafana              replicated          2/1                  grafana/grafana:latest *:3000->3000/tcp
qjaya322y1hs       hasher              replicated          10/10               minhnbn3107/haser:coinswarm *:8002->80/tcp
tejbvqtvlyi       influxdb            replicated          2/1                  influxdb:1.8         *:8086->8086/tcp
js8cfffni1ni0     kibana              replicated          1/1                  kibana:8.16.5        *:5601->5601/tcp
p2lc8gex5mlq       logstash            replicated          1/1                  logstash:8.16.5      *:5000->5000/tcp
zfbzdlm68er4       node-exporter        global             2/1                  prom/node-exporter:latest *:9090->9090/tcp
e6m07zovpwz2       prometheus          replicated          2/2                  prom/prometheus:latest *:9090->9090/tcp
rf3mou904uug       redis              replicated          10/10               minhnbn3107/redis:coinswarm *:8001->80/tcp
refvxxn1bd4       rng                replicated          1/1                  minhnbn3107/rng:coinswarm *:8001->80/tcp
vb5vfjcrugm8       webui              replicated          1/1                  minhnbn3107/webui:coinswarm *:8000->80/tcp
nhttpl1x6s3v       worker              replicated          15/15               minhnbn3107/worker:coinswarm
```

4.3.2 High Availability

4.4 Giám sát và quản trị hệ thống

4.4.1 Kiểm tra xem các dịch vụ đang hoạt động trên các node nào

Các câu lệnh sau chỉ thực hiện được khi node đang thực hiện đáp ứng được 1 trong 2 điều kiện:

- + Docker swarm đã được khởi tạo.
- + Máy thực hiện lệnh đang là manager.

- Đối với các máy worker nếu chạy lệnh này sẽ bị trả về dòng lỗi "This node is not a swarm manager."

```
osboxes@vps3:~$ sudo docker node ls
Error response from daemon: This node is not a swarm manager. Worker nodes can't be used to view or modify cluster state.
Please run this command on a manager node or promote the current node to a manager.
```

`sudo docker node ls`

Để kiểm tra tổng quát toàn bộ các dịch vụ đang có trong cluster, ta có thể dùng lệnh: `sudo docker node ls`.

```
osboxes@vps1:~$ sudo docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
m48r1bh41qzg	elk_elasticsearch	replicated	2/1	docker.elastic.co/elasticsearch/elasticsearch:8.16.5	*:9200->9200/tcp
re3we0go6erkl	elk_kibana	replicated	2/1	docker.elastic.co/kibana/kibana:8.16.5	*:5601->5601/tcp
x1if2zzudq3e	elk_logstash	replicated	1/1	docker.elastic.co/logstash/logstash:8.16.5	*:5000->5000/tcp
p4ezmlkla08z	hasher	replicated	17/10	moonvein/haser:coinswarm	*:8002->80/tcp
d1woxr2hwys	redis	replicated	3/2	redis:latest	
6fgd64tw8rq7	rng	replicated	17/10	moonvein/rng:coinswarm	*:8001->80/tcp
jdkyhok0w112	webui	replicated	8/5	moonvein/webui:coinswarm	*:8000->80/tcp
tzcwq0gmkfrx	worker	replicated	26/15	moonvein/worker:coinswarm	

```
osboxes@vps1:~$
```

- Giải thích chi tiết các cột:
- Cột ID: các định danh mà docker swarm tạo ra và gán cho mỗi service khi chúng được tạo ra. Mỗi ID sẽ khác nhau và có thể dùng ID này để tham chiếu thay cho tên.
- Cột NAME: Tên mà chúng ta đặt cho các service khi thực hiện deploy thông qua giá trị `--name`.
- Cột MODE: Các chế độ chạy của service. Thông thường có 2 chế độ là `replicated` và `global`.
- + Chế độ `global`: Mỗi node chỉ chạy đúng 1 bản sao của nó.
- + Chế độ `replicated`: Mỗi node có thể chạy nhiều bản sao của nó. Ví dụ 1/1 nghĩa là có 1 container đang chạy. Nếu thấy 0/1 là đang bị lỗi hoặc chưa khởi động xong. Ta có thể dùng lệnh `scale` để nâng số lượng replica lên.
- Cột IMAGE: Tên image Docker mà service dùng để khởi tạo container.
- Cột PORTS: Thông tin publish port nếu có.
- + Ví dụ: `*:8000->80/tcp` nghĩa là người dùng có thể truy cập service qua cổng 8000 của máy chủ (host), sau đó Docker sẽ chuyển tiếp đến cổng 80 trong container. Nếu không cài đặt tham số port gì thì service chỉ hoạt động nội bộ trong mạng Swarm.

2. `sudo docker service ps <tên_service>`

- Để kiểm tra chi tiết hơn về 1 dịch vụ, cụ thể là các task của service đó đang được chạy ở node nào, chạy từ bao giờ, có bị lỗi gì không... Ta sẽ sử dụng lệnh docker service ps <tên_service>. Ví dụ nếu ta muốn kiểm tra các task của dịch vụ hasher, ta sẽ gõ lệnh: sudo docker service ps hasher.

```
sshboxes@vps1:~$ sudo docker service ps hasher
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
yvdu7ebpx883	hasher.1	moonvein/hasher:coinswarm	vps1	Running	Running 3 minutes ago		
bwzseun27kxm	hasher.1	moonvein/hasher:coinswarm	vps5	Shutdown	Running 27 hours ago		
gcdxgq6q4nfx	hasher.1	moonvein/hasher:coinswarm	vps7	Shutdown	Shutdown 27 hours ago		
ysa1hmi80qtz	hasher.1	moonvein/hasher:coinswarm	vps5	Shutdown	Failed 27 hours ago	"task: non-zero exit (255)"	
xxax70zv4lvr	hasher.1	moonvein/hasher:coinswarm	vps6	Shutdown	Failed 3 days ago	"task: non-zero exit (255)"	
bb618v9c4tpd	hasher.2	moonvein/hasher:coinswarm	vps2	Running	Running 3 minutes ago		
qhwee0tkk0tz	hasher.2	moonvein/hasher:coinswarm	vps5	Shutdown	Running 27 hours ago		
qq3s905qy41	hasher.2	moonvein/hasher:coinswarm	vps1	Shutdown	Failed 27 hours ago	"task: non-zero exit (255)"	
bgpcvq1na7ab	hasher.2	moonvein/hasher:coinswarm	vps2	Shutdown	Failed 3 days ago	"task: non-zero exit (255)"	
tsw0515s1j2f	hasher.2	moonvein/hasher:coinswarm	vps2	Shutdown	Shutdown 3 days ago		
qg5mg7a66fzc	hasher.3	moonvein/hasher:coinswarm	vps2	Running	Running 3 minutes ago		
vevrdlc157ea	hasher.3	moonvein/hasher:coinswarm	vps6	Shutdown	Running 27 hours ago		
lu10q2v3vsls	hasher.3	moonvein/hasher:coinswarm	vps7	Shutdown	Shutdown 27 hours ago		
kkxwjltxmlbw	hasher.3	moonvein/hasher:coinswarm	vps0	Shutdown	Failed 3 days ago	"task: non-zero exit (255)"	
tshd05t6wzjk	hasher.3	moonvein/hasher:coinswarm	vps1	Shutdown	Shutdown 3 days ago		
tf4s10oksd1ha	hasher.4	moonvein/hasher:coinswarm	vps2	Running	Running 4 minutes ago		
0s07b7judd16	hasher.4	moonvein/hasher:coinswarm	vps2	Shutdown	Failed 4 minutes ago	"task: non-zero exit (255)"	
xnjlpl4sn8pt	hasher.4	moonvein/hasher:coinswarm	vps2	Shutdown	Failed 27 hours ago	"task: non-zero exit (255)"	
q4bfspcp1p955	hasher.4	moonvein/hasher:coinswarm	vps0	Shutdown	Shutdown 3 days ago		
00pa2kas0r1j	hasher.4	moonvein/hasher:coinswarm	vps0	Shutdown	Shutdown 3 days ago		
0v61euk6qahd	hasher.5	moonvein/hasher:coinswarm	vps1	Running	Running 3 minutes ago		

- Giải thích chi tiết các cột:
 - + Cột ID: ID của mỗi task, tương tự ID của lệnh sudo docker service ls.
 - + Cột NAME: Tên các task, gồm có: tên_service.số_thứ_tự . Ví dụ, rng.1 là bản sao thứ nhất của service rng. Tương ứng với tổng số replica đang chạy sẽ có bấy nhiêu bản sao của service đó.
 - + Cột IMAGE: Docker image được sử dụng để khởi tạo container này.
 - + Cột NODE: Tên node trong cụm docker swarm mà task này đang chạy.
 - + Cột DESIRED STATE: Trạng thái mong muốn của Docker Swarm cho task này. Có 3 trạng thái thường thấy là: Running, shutdown, ready.
 - + Trạng thái running: Swarm muốn task này phải đang chạy. Đây là trạng thái bình thường nhất.
 - + Trạng thái shutdown: Swarm đã yêu cầu dừng task này, có thể do ta xóa service, scale hoặc deploy lại.
 - + Trạng thái ready: Task đang trong quá trình chuẩn bị để chuyển sang Running. Trạng thái này thường nhanh và khá ít gặp được.
- Cột CURRENT STATE: Trạng thái hiện tại thực tế của task. Có thể kể ra 1 số trạng thái như sau:
 - + Running at ...: Container đã được tạo và đang chạy ổn định.
 - + Preparing: Swarm đang tạo task/container (chưa chạy).

- + Comeplete at ...: Task đã hoàn thành và dừng lại thành công. Cũng có trường hợp task bị lỗi trong lúc làm và bị nhầm tưởng thành task đã hoàn thành.
- + Failed at...: Task/container bị crash hoặc lỗi khi khởi động hay đang chạy. Swarm sẽ ghi lại lỗi (hiện trong cột ERROR) và có thể tạo lại task khác.
- + Exited at...: Khi một task/container đã kết thúc (vì hoàn thành công việc hoặc gặp lỗi).
- Cột ERROR: Nếu task gặp lỗi trong quá trình khởi động hoặc chạy, sẽ hiển thị thông báo ở đây. Còn trong các trường hợp chạy bình thường không có lỗi thì cột này sẽ trống không.
- Cột PORTS: Tương ứng với cột PORTS của lệnh docker service ls, hiển thị ra tham số port của task đó tương ứng với tham số port của service đó.

3. docker node ps <tên_node>

- Để kiểm tra xem các task đang chạy trên một node nào đó, ta có thể dùng lệnh: docker service <hostname || id_node>. Giả sử trong cluster của ta có 1 node có hostname là vps4 thì ta sẽ gõ lệnh: sudo docker node ps vps4.

```
nsboxes@vps1:~$ sudo docker node ps vps4
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
u9o4efsqpbqj	hasher.10	moonvein/hasher:coinswarm	vps4	Shutdown	Running 27 hours ago		
90pd5t51yych	rng.3	moonvein/rng:coinswarm	vps4	Shutdown	Running 27 hours ago		
jxhqkstm692	rng.8	moonvein/rng:coinswarm	vps4	Shutdown	Running 27 hours ago		
x2753oc3wcm	webui.2	moonvein/webui:coinswarm	vps4	Shutdown	Running 27 hours ago		
h6hent95atgh	worker.2	moonvein/worker:coinswarm	vps4	Shutdown	Failed 27 hours ago	"starting container failed: fa_"	
yc980ain29ly	worker.8	moonvein/worker:coinswarm	vps4	Shutdown	Running 27 hours ago		
gz1n6yuv6vvdq	worker.11	moonvein/worker:coinswarm	vps4	Shutdown	Shutdown 27 hours ago		
Fq51megqeo2h	worker.13	moonvein/worker:coinswarm	vps4	Shutdown	Running 27 hours ago		

```
nsboxes@vps1:~$
```

- Giải thích chi tiết các cột:
- + Các cột của lệnh sudo docker service ps <tên_node> tương đương với các cột trong lệnh sudo docker service ps <tên_service>.
- + Cột ID: ID của mỗi task, tương tự ID của lệnh sudo docker service ls. ID của task này tương đương với ID của task trong cột ID của lệnh docker service ps hasher.
- + Cột NAME: Tên các task, gồm có: tên_service.số_thứ_tự . Ví dụ, hasher.10 là bản sao thứ mười của service hasher. Tương ứng với tổng số replica đang chạy sẽ có bấy nhiêu bản sao của service đó.
- + Cột IMAGE: Docker image được sử dụng để khởi tạo container này.

- + Cột NODE: Tên node khi ta gõ lệnh.
- + Cột DESIRED STATE: Trạng thái mong muốn của Docker Swarm cho task này. Có 3 trạng thái thường thấy là: Running, shutdown, ready.
- + Trạng thái running: Swarm muốn task này phải đang chạy. Đây là trạng thái bình thường nhất.
- + Trạng thái shutdown: Swarm đã yêu cầu dừng task này, có thể do ta xóa service, scale hoặc deploy lại.
- + Trạng thái ready: Task đang trong quá trình chuẩn bị để chuyển sang Running. Trạng thái này thường nhanh và khá ít gặp được.
- Cột CURRENT STATE: Trạng thái hiện tại thực tế của task. Có thể kể ra 1 số trạng thái như sau:
 - + Running at ...: Container đã được tạo và đang chạy ổn định.
 - + Preparing: Swarm đang tạo task/container (chưa chạy).
 - + Complete at ...: Task đã hoàn thành và dừng lại thành công. Cũng có trường hợp task bị lỗi trong lúc làm và bị nhầm tưởng thành task đã hoàn thành.
 - + Failed at...: Task/container bị crash hoặc lỗi khi khởi động hay đang chạy. Swarm sẽ ghi lại lỗi (hiện trong cột ERROR) và có thể tạo lại task khác.
 - + Exited at...: Khi một task/container đã kết thúc (vì hoàn thành công việc hoặc gặp lỗi).
- Cột ERROR: Nếu task gặp lỗi trong quá trình khởi động hoặc chạy, sẽ hiển thị thông báo ở đây. Còn trong các trường hợp chạy bình thường không có lỗi thì cột này sẽ trống không.
- Cột PORTS: Tương ứng với cột PORTS của lệnh docker service ls, hiển thị ra tham số port của task đó tương ứng với tham số port của service đó.

4.4.2 Kiểm tra CPU/ memory và I/O Usage của cụm swarm (Sử dụng

grafana + prAometheus)

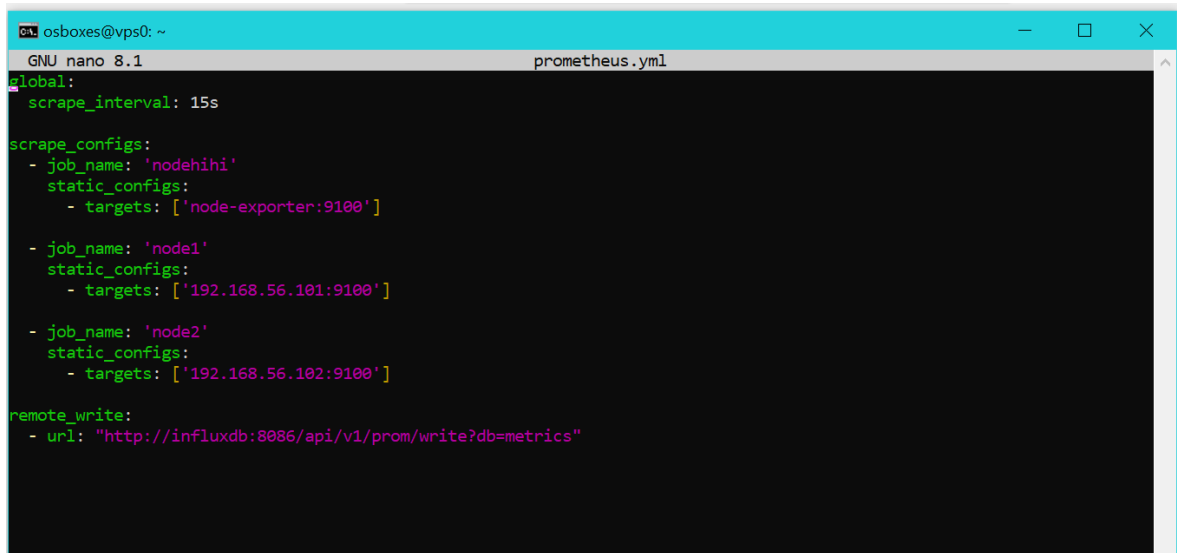
Bước 1: Tạo thư mục cấu hình

```
osboxes@vps0:~$ mkdir -p swarm-monitoring/prometheus
osboxes@vps0:~$ cd swarm-monitoring
```

Bước 2: Tạo file cấu hình prometheus/prometheus.yml

```
osboxes@vps0:~/swarm-monitoring$ mkdir -p prometheus
osboxes@vps0:~/swarm-monitoring$ nano prometheus/prometheus.yml
osboxes@vps0:~/swarm-monitoring$
```

- Dán nội dung cấu hình Prometheus vào file:



```
GNU nano 8.1 prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'nodehihi'
    static_configs:
      - targets: ['node-exporter:9100']

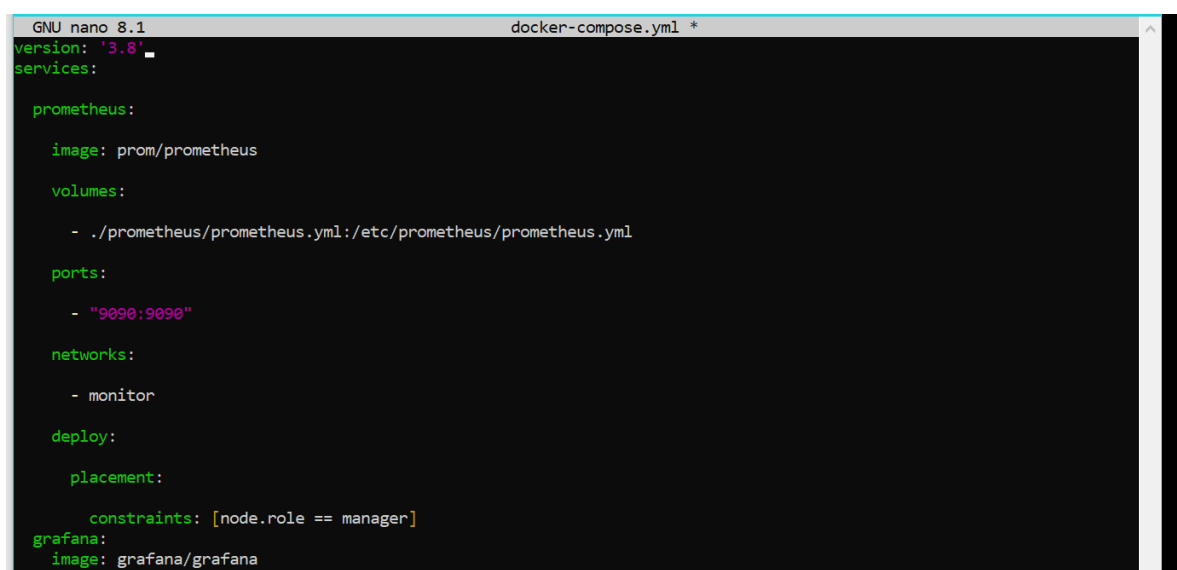
  - job_name: 'node1'
    static_configs:
      - targets: ['192.168.56.101:9100']

  - job_name: 'node2'
    static_configs:
      - targets: ['192.168.56.102:9100']

remote_write:
  - url: "http://influxdb:8086/api/v1/prom/write?db=metrics"
```

Bước 3: Tạo docker-compose.yml (sử dụng được với Docker Swarm)

- Tạo file docker-compose.yml trong thư mục swarm-monitoring/:nano docker-compose.yml
- Dán cấu hình Docker Compose vào file:



```
GNU nano 8.1 docker-compose.yml *
version: '3.8'
services:
  prometheus:
    image: prom/prometheus
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    networks:
      - monitor
    deploy:
      placement:
        constraints: [node.role == manager]
  grafana:
    image: grafana/grafana
```

Bước 4: Deploy lên cụm Swarm

docker stack deploy -c docker-compose.yml monitoring

Bước 5: Kiểm tra trạng thái dịch vụ

docker service ls

```
osboxes@vps0:~/swarm-monitoring$ sudo docker service ls
docker service ls
ID                NAME                MODE                REPLICAS    IMAGE                                  PORTS
ugcfu31jq3o0     elasticsearch        replicated          2/1         elasticsearch:8.16.5                *:9200->9200/tcp
ytx61s6upyv8     grafana              replicated          1/1         grafana/grafana:10.4.1              *:3000->3000/tcp
c8jk4yukpeac     hasher              replicated          0/1         nhuquynh15/hasher:coinswarm         *:8002->80/tcp
vpi5yodh4rhs     kibana              replicated          1/1         kibana:8.16.5                       *:5601->5601/tcp
nitjuhmvw6eu     logstash            replicated          2/2         logstash:8.16.5                     *:5000->5000/tcp
kr65avfop939     monitoring_node-exporter global             0/1         prom/node-exporter:latest
1ua9vrjld59g     monitoring_prometheus replicated         0/1         prom/prometheus:latest              *:9090->9090/tcp
3qz33ba5hkzr     redis               replicated          1/1         redis:latest
5x0pm677pd02     rng                 replicated          2/1         nhuquynh15/rng:coinswarm            *:8001->80/tcp
wvkm07hwx14w     webui               replicated          1/1         nhuquynh15/webui:coinswarm          *:8000->80/tcp
sqkn9e1xp662     worker              replicated          1/1         nhuquynh15/worker:coinswarm
```

Bước 6: Truy cập vào Giao diện web của Grafana với link <http://192.168.56.101:3000>

- Khi truy cập lần đầu, Grafana sẽ yêu cầu bạn đăng nhập.
- Mặc định tài khoản:

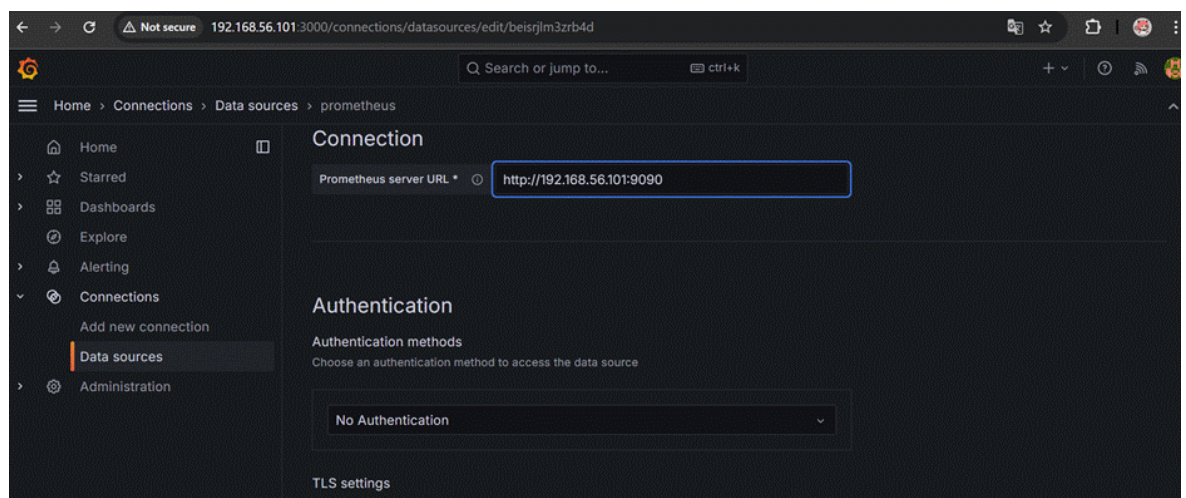
Username: admin

Password: admin

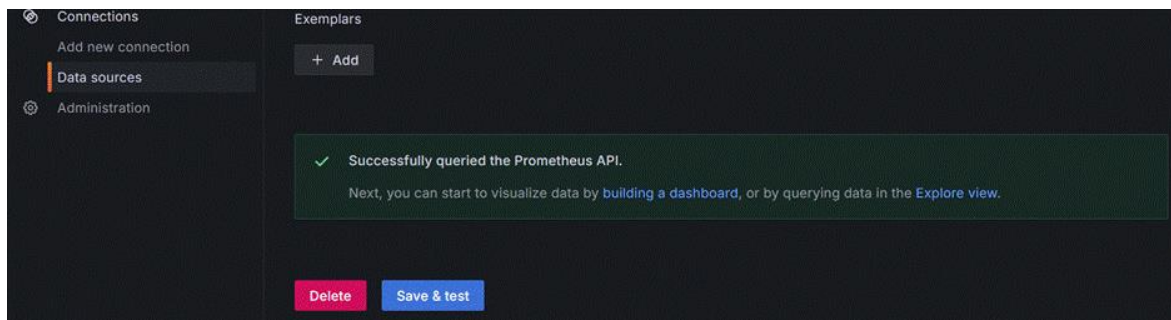
Sau khi đăng nhập, nó sẽ bắt bạn đổi mật khẩu.

Tiếp theo là Add data source

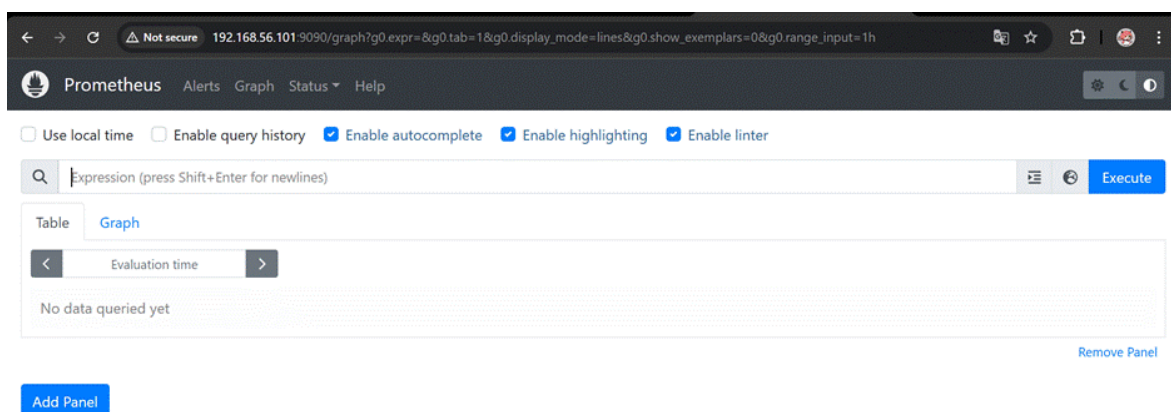
Điền vào URL: <http://192.168.56.101:9090>



Save & test thành công



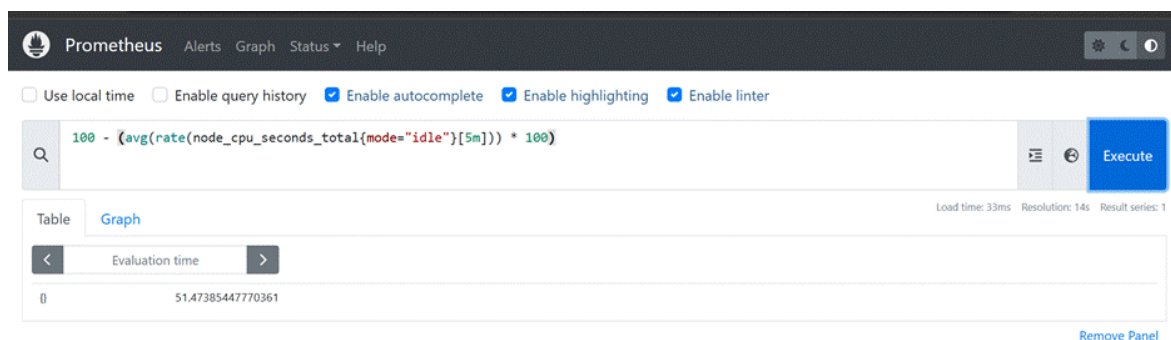
Truy cập vào trang <http://192.168.56.101:9090>, chọn add panel



Xem tổng CPU usage của cả cụm (gộp toàn bộ node)

$100 - (\text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5\text{m}])) * 100$

Nhập vào ô expression và nhấn Excute để thực thi



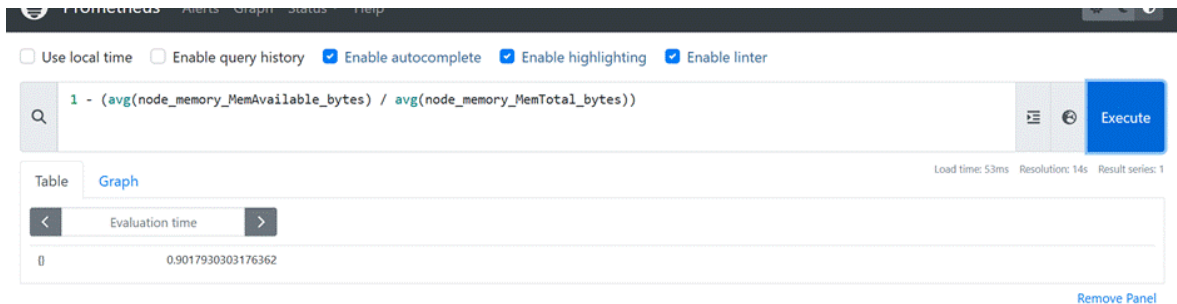
Kết quả nhận được là: 51,47

--> tổng cộng tất cả các CPU trong cụm Swarm đang sử dụng khoảng 50.47%

Xem tổng memory usage của cả cụm (gộp toàn bộ node)

1 -
$$\frac{(\text{avg}(\text{node_memory_MemAvailable_bytes}))}{\text{avg}(\text{node_memory_MemTotal_bytes}))}$$

Nhập vào ô expression và nhấn Execute để thực thi



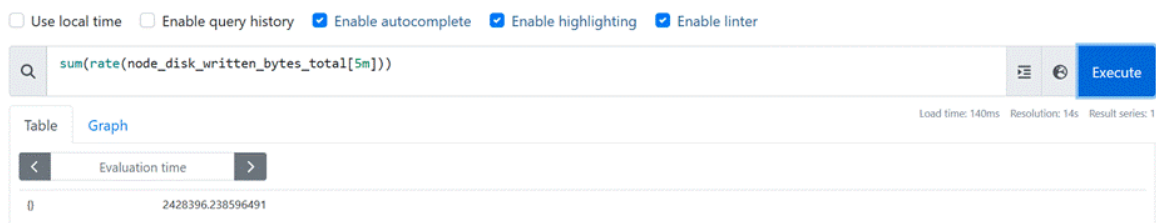
Kết quả nhận được là: 0.9018

--> 0,18% bộ nhớ đang được sử dụng trong toàn cụm Swarm, đây là con số rất cao

Xem Disk I/O Usage của cụm Swarm

Tốc độ ghi (write rate):

$$\text{sum}(\text{rate}(\text{node_disk_written_bytes_total}[5m]))$$

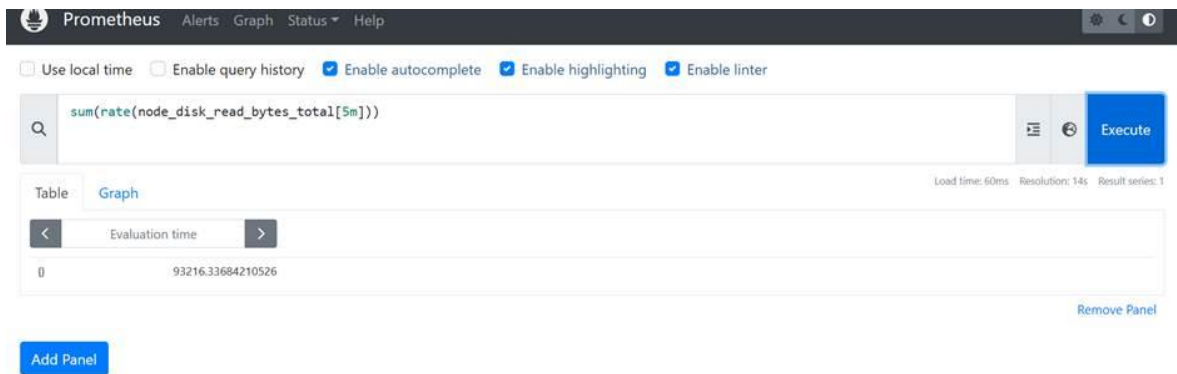


--> Kết quả trả về: 2428396.238596491 bytes/second $\approx$ 2,428,396 bytes/s $\approx$ 2.31 MB/s

--> Đây là tổng tốc độ ghi dữ liệu ra ổ đĩa (write I/O) trên toàn cụm Swarm, được tính trung bình trong 5 phút gần nhất.

Tốc độ đọc (read rate):

$$\text{sum}(\text{rate}(\text{node_disk_read_bytes_total}[5m]))$$



--> Kết quả trả về: 93216.33684210526 bytes/second $\approx$ 93.2 KB/s

-->F

4.4.3 Đo latency và response time của hasher và rng

1. Đo latency và response time của dịch vụ rng

Bước 1: Xác định máy đang chạy dịch vụ RNG

- Sử dụng câu lệnh sau để kiểm tra:

Sudo docker service ps rng

- Kiểm tra máy nào có dòng chữ running tức là máy đó đang chạy dịch vụ rng

Bước 2: Tiến hành đo

- Sử dụng câu lệnh để đo: `sudo httping -c <số lần request> <địa chỉ dịch vụ (URL hoặc IP:port)>`

Ví dụ: `sudo httping -c 10 202.201.25.28:8001`

```
osboxes@vps0:~$ sudo httping -c 10 202.201.25.28:8001
PING 202.201.25.28:8001 (/):
connected to 202.201.25.28:8001 (177 bytes), seq=0 time= 10.28 ms
connected to 202.201.25.28:8001 (177 bytes), seq=1 time= 6.66 ms
connected to 202.201.25.28:8001 (177 bytes), seq=2 time= 8.00 ms
connected to 202.201.25.28:8001 (177 bytes), seq=3 time= 31.26 ms
connected to 202.201.25.28:8001 (177 bytes), seq=4 time= 17.08 ms
connected to 202.201.25.28:8001 (177 bytes), seq=5 time= 10.32 ms
connected to 202.201.25.28:8001 (177 bytes), seq=6 time= 6.48 ms
connected to 202.201.25.28:8001 (177 bytes), seq=7 time= 5.91 ms
connected to 202.201.25.28:8001 (177 bytes), seq=8 time= 6.61 ms
connected to 202.201.25.28:8001 (177 bytes), seq=9 time= 6.16 ms
--- http://202.201.25.28:8001/ ping statistics ---
10 connects, 10 ok, 0.00% failed, time 10130ms
round-trip min/avg/max = 5.9/10.9/31.3 ms
```

Giải thích chi tiết:

- `sudo`: chạy với quyền root (do `htping` cần quyền để mở socket).
- `htping`: công cụ gửi request HTTP như ping.
- `-c 10`: gửi tổng cộng 10 yêu cầu HTTP.
- `202.201.25.28:8001`: địa chỉ IP và cổng của dịch vụ mà bạn đang kiểm tra latency.

Nhận xét:

- Đã gửi 10 yêu cầu HTTP, tất cả đều nhận được phản hồi thành công từ dịch vụ đang chạy tại `202.201.25.28:8001`.

- Không có lỗi kết nối, tổng thời gian của quá trình kiểm tra là 10.13 giây.

- Độ trễ của các phản hồi:

+ Phản hồi nhanh nhất: 5.9ms

+ Thời gian trung bình: 10.9ms

+ Phản hồi chậm nhất: 31.3ms

Tổng kết: **Dịch vụ đang hoạt động ổn định**, không có lỗi phản hồi, có một phản hồi **cao bất thường (31.3ms)**, nhưng nếu chỉ thỉnh thoảng thì vẫn có thể chấp nhận được (do mạng hoặc tải hệ thống tại thời điểm đó).

4.4.4 Sử dụng câu lệnh để đo:

`sudo htping -c <số lần request> <địa chỉ dịch vụ (URL hoặc IP:port)>`

`sudo htping -c 10 localhost:8002`

Trong đó:

- + `sudo`: chạy với quyền root (cần thiết vì `htping` mở socket).
- + `htping`: công cụ gửi HTTP request để đo độ trễ.
- + `-c 10`: gửi 10 request HTTP.
- + `localhost:8002`: địa chỉ IP và port của dịch vụ Hasher.

Phân tích kết quả:

```
osboxes@vps0:~$ sudo httping -c 10 localhost:8002
PING localhost:8002 (/):
connected to 127.0.0.1:8002 (177 bytes), seq=0 time= 6.34 ms
connected to 127.0.0.1:8002 (177 bytes), seq=1 time= 9.19 ms
connected to 127.0.0.1:8002 (177 bytes), seq=2 time= 11.12 ms
connected to 127.0.0.1:8002 (177 bytes), seq=3 time= 5.64 ms
connected to 127.0.0.1:8002 (177 bytes), seq=4 time= 6.28 ms
connected to 127.0.0.1:8002 (177 bytes), seq=5 time= 9.84 ms
connected to 127.0.0.1:8002 (177 bytes), seq=6 time= 6.91 ms
connected to 127.0.0.1:8002 (177 bytes), seq=7 time= 9.79 ms
connected to 127.0.0.1:8002 (177 bytes), seq=8 time= 5.87 ms
connected to 127.0.0.1:8002 (177 bytes), seq=9 time= 6.37 ms
--- http://localhost:8002/ ping statistics ---
10 connects, 10 ok, 0.00% failed, time 10092ms
round-trip min/avg/max = 5.6/7.7/11.1 ms
osboxes@vps0:~$
```

Giải thích chi tiết:

- + Đã gửi **10** request HTTP đến localhost:8002.
- + **Tất cả** các request đều nhận phản hồi thành công.
- + Tổng thời gian kiểm tra: khoảng 1 giây.
- + Độ trễ của các phản hồi:

Phản hồi nhanh nhất: **5.6 ms**

Thời gian trung bình: **7.1 ms**

Phản hồi chậm nhất: **11.1 ms**

Nhận xét:

- + Dịch vụ Hasher tại localhost:8002 hoạt động ổn định.
- + Không có lỗi kết nối hay mất gói.
- + Độ trễ trung bình rất thấp, cho thấy hệ thống phản hồi nhanh và ổn định
- + Mức độ dao động giữa min/max là nhỏ (5.6 – 11.1 ms), cho thấy mạng hoạt động hiệu quả, không có phản hồi bất thường.

4.5 Logstash chạy trên swarm với 2 replicas

- Để làm được việc này, ta cần chắc chắn rằng dịch vụ đang chạy tốt (không bị trường hợp như 0/1, 0/2...). Sau đó ta cũng cần chắc chắn rằng nơi thực hiện

câu lệnh phải có service logstash, nếu không ta sẽ bị lỗi “blind mount”. Cuối cùng, ta sẽ sử dụng lệnh: `sudo docker service scale logstash=2`. Kết quả sẽ ra như ảnh

```
osboxes@vps0:~$ sudo docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
tlgebpbjde8v	coinswarm-proxy	replicated	1/1	nginx:latest	*:8080->80/tcp
o4bx4gpsbxay	elasticsearch	replicated	1/1	elasticsearch:8.16.5	*:9200->9200/tcp
41aamcs277y3	grafana	replicated	2/1	grafana/grafana:latest	*:3000->3000/tcp
p4ezmlk1a08z	hasher	replicated	10/10	moonvein/hasher:coinswarm	*:8002->80/tcp
ppjodp7vz805	influxdb	replicated	1/1	influxdb:1.8	*:8086->8086/tcp
w1l396ni6nhf	kibana	replicated	1/1	kibana:8.16.5	*:5601->5601/tcp
611zurfybdsb	logstash	replicated	2/2	logstash:8.16.5	*:5000->5000/tcp
k9qf51nthxis	node-exporter	global	5/4	prom/node-exporter:latest	
3us87n5v414d	prometheus	replicated	1/1	prom/prometheus:latest	*:9090->9090/tcp
d1woxr2hwygs	redis	replicated	2/2	redis:latest	
6fgd64tw8rq7	rng	replicated	12/10	moonvein/rng:coinswarm	*:8001->80/tcp
jdkyhok8w112	webui	replicated	6/5	moonvein/webui:coinswarm	*:8000->80/tcp
8a0tp80iakhl	worker	replicated	6/5	moonvein/worker:coinswarm	

4.6 Test log của logstash đọc logs từ 4 services: rng, hasher, worker, webui

- Cách 1: Ta có thể dùng lệnh `sudo docker service logs logstash` để xem toàn bộ log được gửi từ 4 service rng, hasher, worker và webui về logstash.

```
logstash-2_7c7f6f0urmpgpb [2025-04-15T12:52:31.872Z] [WARN] [logstash.codecs.line] [main] [5566f80dbdb6f70565925500a6544040725681574445482262354637e] Received an event that has a different character encoding than you configured. (source: "stdin" \u0000\u0001\u0002\u0003\u0004\u0005\u0006\u0007\u0008\u0009\u000a\u000b\u000c\u000d\u000e\u000f\u0010\u0011\u0012\u0013\u0014\u0015\u0016\u0017\u0018\u0019\u001a\u001b\u001c\u001d\u001e\u001f\u0020\u0021\u0022\u0023\u0024\u0025\u0026\u0027\u0028\u0029\u002a\u002b\u002c\u002d\u002e\u002f\u0030\u0031\u0032\u0033\u0034\u0035\u0036\u0037\u0038\u0039\u003a\u003b\u003c\u003d\u003e\u003f\u0040\u0041\u0042\u0043\u0044\u0045\u0046\u0047\u0048\u0049\u004a\u004b\u004c\u004d\u004e\u004f\u0050\u0051\u0052\u0053\u0054\u0055\u0056\u0057\u0058\u0059\u005a\u005b\u005c\u005d\u005e\u005f\u0060\u0061\u0062\u0063\u0064\u0065\u0066\u0067\u0068\u0069\u006a\u006b\u006c\u006d\u006e\u006f\u0070\u0071\u0072\u0073\u0074\u0075\u0076\u0077\u0078\u0079\u007a\u007b\u007c\u007d\u007e\u007f\u0080\u0081\u0082\u0083\u0084\u0085\u0086\u0087\u0088\u0089\u008a\u008b\u008c\u008d\u008e\u008f\u0090\u0091\u0092\u0093\u0094\u0095\u0096\u0097\u0098\u0099\u009a\u009b\u009c\u009d\u009e\u009f\u00a0\u00a1\u00a2\u00a3\u00a4\u00a5\u00a6\u00a7\u00a8\u00a9\u00aa\u00ab\u00ac\u00ad\u00ae\u00af\u00b0\u00b1\u00b2\u00b3\u00b4\u00b5\u00b6\u00b7\u00b8\u00b9\u00ba\u00bb\u00bc\u00bd\u00be\u00bf\u00c0\u00c1\u00c2\u00c3\u00c4\u00c5\u00c6\u00c7\u00c8\u00c9\u00ca\u00cb\u00cc\u00cd\u00ce\u00cf\u00d0\u00d1\u00d2\u00d3\u00d4\u00d5\u00d6\u00d7\u00d8\u00d9\u00da\u00db\u00dc\u00dd\u00de\u00df\u00e0\u00e1\u00e2\u00e3\u00e4\u00e5\u00e6\u00e7\u00e8\u00e9\u00ea\u00eb\u00ec\u00ed\u00ie\u00ef\u00f0\u00f1\u00f2\u00f3\u00f4\u00f5\u00f6\u00f7\u00f8\u00f9\u00fa\u00fb\u00fc\u00fd\u00fe\u00ff\u2013\u2014\u2015\u2016\u2017\u2018\u2019\u201a\u201b\u201c\u201d\u201e\u201f\u2020\u2021\u2022\u2023\u2024\u2025\u2026\u2027\u2028\u2029\u202a\u202b\u202c\u202d\u202e\u202f\u2030\u2031\u2032\u2033\u2034\u2035\u2036\u2037\u2038\u2039\u203a\u203b\u203c\u203d\u203e\u203f\u2040\u2041\u2042\u2043\u2044\u2045\u2046\u2047\u2048\u2049\u204a\u204b\u204c\u204d\u204e\u204f\u2050\u2051\u2052\u2053\u2054\u2055\u2056\u2057\u2058\u2059\u205a\u205b\u205c\u205d\u205e\u205f\u2060\u2061\u2062\u2063\u2064\u2065\u2066\u2067\u2068\u2069\u206a\u206b\u206c\u206d\u206e\u206f\u2070\u2071\u2072\u2073\u2074\u2075\u2076\u2077\u2078\u2079\u207a\u207b\u207c\u207d\u207e\u207f\u2080\u2081\u2082\u2083\u2084\u2085\u2086\u2087\u2088\u2089\u208a\u208b\u208c\u208d\u208e\u208f\u2090\u2091\u2092\u2093\u2094\u2095\u2096\u2097\u2098\u2099\u209a\u209b\u209c\u209d\u209e\u209f\u20a0\u20a1\u20a2\u20a3\u20a4\u20a5\u20a6\u20a7\u20a8\u20a9\u20aa\u20ab\u20ac\u20ad\u20ae\u20af\u20b0\u20b1\u20b2\u20b3\u20b4\u20b5\u20b6\u20b7\u20b8\u20b9\u20ba\u20bb\u20bc\u20bd\u20be\u20bf\u20c0\u20c1\u20c2\u20c3\u20c4\u20c5\u20c6\u20c7\u20c8\u20c9\u20ca\u20cb\u20cc\u20cd\u20ce\u20cf\u20d0\u20d1\u20d2\u20d3\u20d4\u20d5\u20d6\u20d7\u20d8\u20d9\u20da\u20db\u20dc\u20dd\u20de\u20df\u20e0\u20e1\u20e2\u20e3\u20e4\u20e5\u20e6\u20e7\u20e8\u20e9\u20ea\u20eb\u20ec\u20ed\u20ee\u20ef\u20f0\u20f1\u20f2\u20f3\u20f4\u20f5\u20f6\u20f7\u20f8\u20f9\u20fa\u20fb\u20fc\u20fd\u20fe\u20ff\u2100\u2101\u2102\u2103\u2104\u2105\u2106\u2107\u2108\u2109\u210a\u210b\u210c\u210d\u210e\u210f\u2110\u2111\u2112\u2113\u2114\u2115\u2116\u2117\u2118\u2119\u211a\u211b\u211c\u211d\u211e\u211f\u2120\u2121\u2122\u2123\u2124\u2125\u2126\u2127\u2128\u2129\u212a\u212b\u212c\u212d\u212e\u212f\u2130\u2131\u2132\u2133\u2134\u2135\u2136\u2137\u2138\u2139\u213a\u213b\u213c\u213d\u213e\u213f\u2140\u2141\u2142\u2143\u2144\u2145\u2146\u2147\u2148\u2149\u214a\u214b\u214c\u214d\u214e\u214f\u2150\u2151\u2152\u2153\u2154\u2155\u2156\u2157\u2158\u2159\u215a\u215b\u215c\u215d\u215e\u215f\u2160\u2161\u2162\u2163\u2164\u2165\u2166\u2167\u2168\u2169\u216a\u216b\u216c\u216d\u216e\u216f\u2170\u2171\u2172\u2173\u2174\u2175\u2176\u2177\u2178\u2179\u217a\u217b\u217c\u217d\u217e\u217f\u2180\u2181\u2182\u2183\u2184\u2185\u2186\u2187\u2188\u2189\u218a\u218b\u218c\u218d\u218e\u218f\u2190\u2191\u2192\u2193\u2194\u2195\u2196\u2197\u2198\u2199\u219a\u219b\u219c\u219d\u219e\u219f\u21a0\u21a1\u21a2\u21a3\u21a4\u21a5\u21a6\u21a7\u21a8\u21a9\u21aa\u21ab\u21ac\u21ad\u21ae\u21af\u21b0\u21b1\u21b2\u21b3\u21b4\u21b5\u21b6\u21b7\u21b8\u21b9\u21ba\u21bb\u21bc\u21bd\u21be\u21bf\u21c0\u21c1\u21c2\u21c3\u21c4\u21c5\u21c6\u21c7\u21c8\u21c9\u21ca\u21cb\u21cc\u21cd\u21ce\u21cf\u21d0\u21d1\u21d2\u21d3\u21d4\u21d5\u21d6\u21d7\u21d8\u21d9\u21da\u21db\u21dc\u21dd\u21de\u21df\u21e0\u21e1\u21e2\u21e3\u21e4\u21e5\u21e6\u21e7\u21e8\u21e9\u21ea\u21eb\u21ec\u21ed\u21ee\u21ef\u21f0\u21f1\u21f2\u21f3\u21f4\u21f5\u21f6\u21f7\u21f8\u21f9\u21fa\u21fb\u21fc\u21fd\u21fe\u21ff\u2200\u2201\u2202\u2203\u2204\u2205\u2206\u2207\u2208\u2209\u220a\u220b\u220c\u220d\u220e\u220f\u2210\u2211\u2212\u2213\u2214\u2215\u2216\u2217\u2218\u2219\u221a\u221b\u221c\u221d\u221e\u221f\u2220\u2221\u2222\u2223\u2224\u2225\u2226\u2227\u2228\u2229\u222a\u222b\u222c\u222d\u222e\u222f\u2230\u2231\u2232\u2233\u2234\u2235\u2236\u2237\u2238\u2239\u223a\u223b\u223c\u223d\u223e\u223f\u2240\u2241\u2242\u2243\u2244\u2245\u2246\u2247\u2248\u2249\u224a\u224b\u224c\u224d\u224e\u224f\u2250\u2251\u2252\u2253\u2254\u2255\u2256\u2257\u2258\u2259\u225a\u225b\u225c\u225d\u225e\u225f\u2260\u2261\u2262\u2263\u2264\u2265\u2266\u2267\u2268\u2269\u226a\u226b\u226c\u226d\u226e\u226f\u2270\u2271\u2272\u2273\u2274\u2275\u2276\u2277\u2278\u2279\u227a\u227b\u227c\u227d\u227e\u227f\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u22f8\u22f9\u22fa\u22fb\u22fc\u22fd\u22fe\u22ff\u2280\u2281\u2282\u2283\u2284\u2285\u2286\u2287\u2288\u2289\u228a\u228b\u228c\u228d\u228e\u228f\u2290\u2291\u2292\u2293\u2294\u2295\u2296\u2297\u2298\u2299\u229a\u229b\u229c\u229d\u229e\u229f\u22a0\u22a1\u22a2\u22a3\u22a4\u22a5\u22a6\u22a7\u22a8\u22a9\u22aa\u22ab\u22ac\u22ad\u22ae\u22af\u22b0\u22b1\u22b2\u22b3\u22b4\u22b5\u22b6\u22b7\u22b8\u22b9\u22ba\u22bb\u22bc\u22bd\u22be\u22bf\u22c0\u22c1\u22c2\u22c3\u22c4\u22c5\u22c6\u22c7\u22c8\u22c9\u22ca\u22cb\u22cc\u22cd\u22ce\u22cf\u22d0\u22d1\u22d2\u22d3\u22d4\u22d5\u22d6\u22d7\u22d8\u22d9\u22da\u22db\u22dc\u22dd\u22de\u22df\u22e0\u22e1\u22e2\u22e3\u22e4\u22e5\u22e6\u22e7\u22e8\u22e9\u22ea\u22eb\u22ec\u22ed\u22ee\u22ef\u22f0\u22f1\u22f2\u22f3\u22f4\u22f5\u22f6\u22f7\u
```

- ```
osboxes@vps0:~$ sudo docker container ls
```
- | CONTAINER ID         | IMAGE                     | COMMAND                  |
|----------------------|---------------------------|--------------------------|
| 2b8d3f7d532e         | moonvein/hasher:coinswarm | "python hasher.py"       |
| cf776f0a7430         | moonvein/rng:coinswarm    | "python rng.py"          |
| 2ec34125684e         | moonvein/webui:coinswarm  | "node webui.js"          |
| 9662eebdc3c1         | moonvein/hasher:coinswarm | "python hasher.py"       |
| c762768fced9         | moonvein/rng:coinswarm    | "python rng.py"          |
| dc7f27870441         | logstash:8.16.5           | "/usr/local/bin/dock..." |
| 4c9b92ce89b5         | logstash:8.16.5           | "/usr/local/bin/dock..." |
| 942d753c3467         | kibana:8.16.5             | "/bin/tini -- /usr/l..." |
| 11142b7c8861         | moonvein/worker:coinswarm | "python worker.py"       |
| a1bd97f5b3a0         | prom/prometheus:latest    | "/bin/prometheus --c..." |
| e24147b5ee2a         | prom/node-exporter:latest | "/bin/node_exporter ..." |
| d0aj3a4huahxy36g8dn2 |                           |                          |

- [illegible]

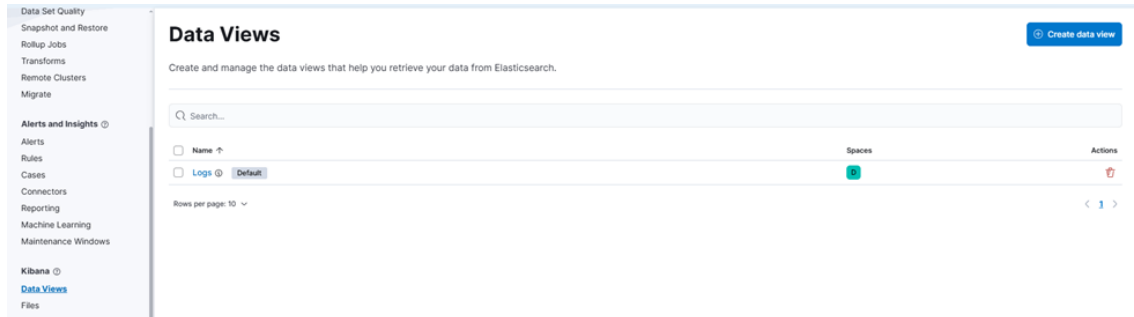
- ```

$ docker exec -it kibana kibana:root curl -X GET "202.201.25.25:9200/_cat/index?v"
health status index uuid pri rep docs.count docs.deleted store.size pri.store.size dataset.size
green open .internal.alerts-transform.health.alerts-default-000001 lgLoXce8R0q5Z5E5wqC 1 0 0 249b 249b 249b
green open .internal.alerts-observability_logs.alerts-default-000001 9-0iHlP862cSGmWFlq-0 1 0 0 249b 249b 249b
green open .internal.alerts-observability_logs.alerts-default-000001 Yl8CnHlP862cSGmWFlq-0 1 0 0 249b 249b 249b
green open .internal.alerts-ml.anomaly-detection.alerts-default-000001 mCmF8605C0iNgJQ2KwA 1 0 0 249b 249b 249b
green open .internal.alerts-observability.slo.alerts-default-000001 bcP5R83v5sXK0flFp-fFQ 1 0 0 249b 249b 249b
green open .internal.alerts-default.alerts-default-000001 0Hv0U-1-Ru9vB1C0F3dW 1 0 0 249b 249b 249b
green open .internal.alerts-observability_agn.alerts-default-000001 J1v5k5X0Q0Qy-cnTmT 1 0 0 249b 249b 249b
green open .internal.alerts-observability.metrics.alerts-default-000001 J1L-1L8K53Jap8iPhHmZhw 1 0 0 249b 249b 249b
green open .kibana-observability-ai-assistant.conversations-000001 XlCGV6fZQjYUk1Zb3sYVw 1 0 0 249b 249b 249b
green open .internal.alerts-ml.anomaly-detection.health.alerts-default-000001 J1Ck01G05vgYkUwX5f1g 1 0 0 249b 249b 249b
green open .internal.alerts-observability_threshold.alerts-default-000001 334e010Bm-1-c0IT3Rk 1 0 0 249b 249b 249b
green open .internal.alerts-security.alerts-default-000001 P9v8aadOTJebC1K87xKkg 1 0 0 249b 249b 249b
green open .kibana-observability-ai-assistant.kb-000001 T2Z0X-ppR9mN8K10uZ1w 1 0 0 249b 249b 249b
green open .internal.alerts-stack.alerts-default-000001 BG0617FRX3vFvInuIRa 1 0 0 249b 249b 249b
yellow open logs-2025.04.15 XUY65f5Tsq-pn0a0L1 1 1 448020 0 459.8mb 459.8mb 459.8mb

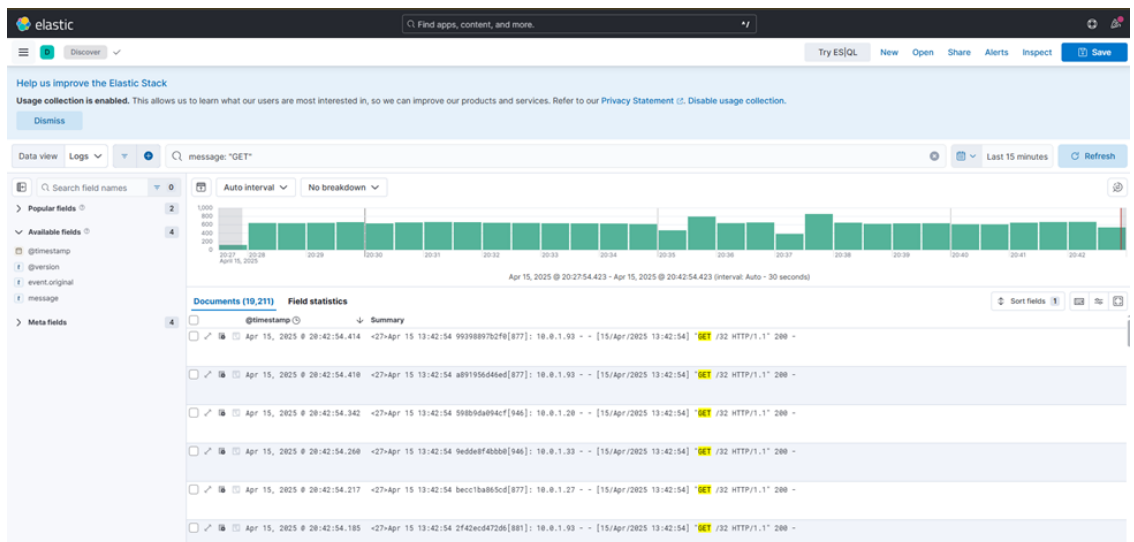
```

- | | | |
|---------|----|---------------|
| Nhóm 02 | 48 | Lớp DHHTTT17B |
|---------|----|---------------|

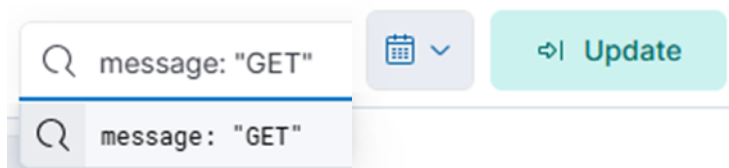
- Cách 2: Truy vấn logs trên Kibana: Để truy vấn được logs, ta cần tạo 1 data view. Ta tìm thanh công cụ nằm ở bên trái màn hình vào theo đường dẫn Stack management => Data views => Create data views.



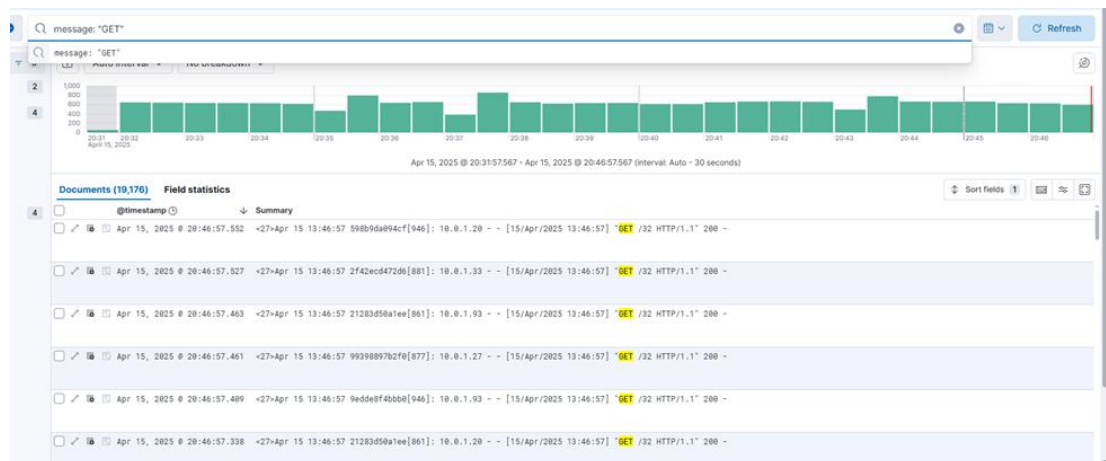
- + Sau khi tạo xong, ta ấn vào mục Discover trong tab Analytics. Giao diện sẽ như sau:



- + Nếu ta có nhiều Data views, ta có thể bấm và đổi qua Data view khác.
- + Tiếp theo, để xem các log theo mong muốn, ta có thể thực hiện truy vấn như sau:
 - 1) Sử dụng bộ lọc dữ liệu
 - Ở thanh filter, giả sử ta muốn lấy ra các log mà trong đó, thông tin message có giá trị chứa chuỗi “GET” thì ta nhập vào message: “GET” để lọc, sau đó ta ấn vào nút Update để bắt đầu quá trình lọc.

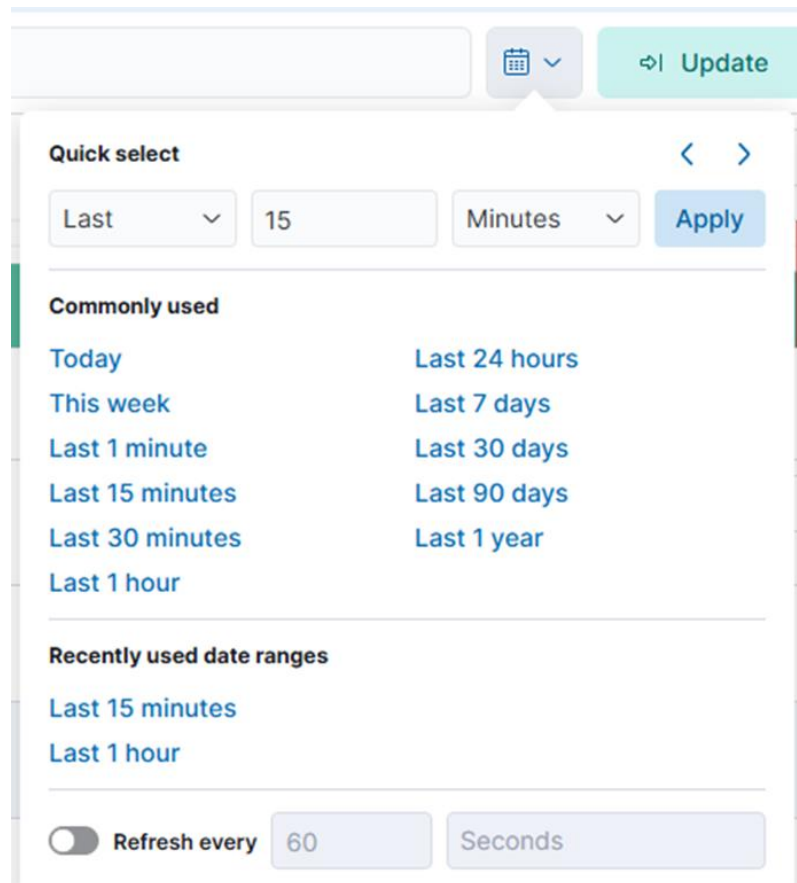


- Sau khi lọc xong, Kibana sẽ hiển thị ra các logs chỉ chứa các message có giá trị “GET”



2) chỉnh thời gian truy vấn logs

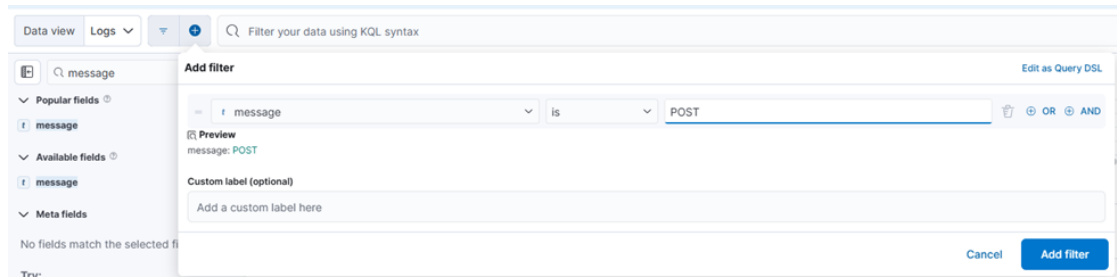
- Ở trong icon lịch, ta có thể chọn thời gian log được up lên như sau:



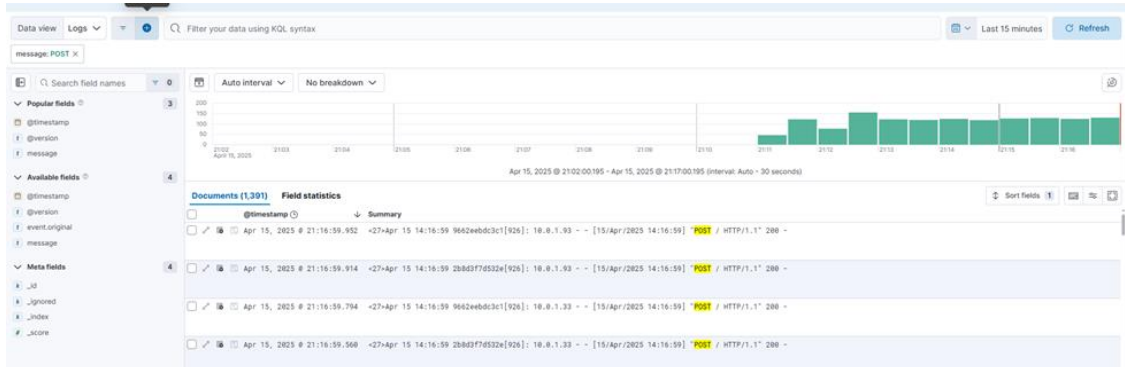
- Cụ thể:
- + Ở tab Quick select, ta có thể điều chỉnh thời gian theo ý ta. Chẳng hạn nếu ta chỉnh là Last 15 minutes thì nó sẽ lấy log từ 15 phút trước đến hiện tại.
- + Còn nếu ta sử dụng nút gạt Refresh thì Kibana sẽ tự động làm mới dữ liệu theo khoảng thời gian định sẵn (mặc định 60s – phù hợp cho việc theo dõi thời gian thực).

3) Truy vấn theo Field

- Nếu không dùng thanh tìm kiếm, ta có thể sử dụng Filter để lọc ra các giá trị. Giả sử ta muốn lọc ra các message có chứa chuỗi POST thì ta sẽ làm như sau:
- + Ta nhấn vào dấu + để Add filter. Sau đó, hộp thoại này sẽ hiện ra, tiếp theo, ta chọn field message. Ở phần operator, ta chọn is. Ở phần value, ta nhập POST và ấn Add filter.



+ Kết quả là ta sẽ thu được các message có chứa chuỗi “POST”.



4) Thay đổi dạng hiển thị dữ liệu theo Interval và Breakdown

– Ta có thể thay đổi cách trực quan biểu đồ log bằng cách điều chỉnh:

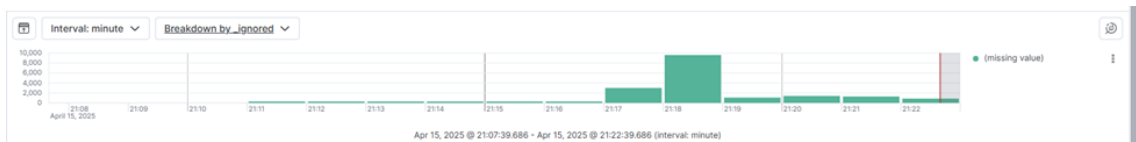
+ Interval: Chia biểu đồ theo thời gian.

+ Breakdown: Dùng để phân loại các log theo giá trị của một field nào đó.

+ Ví dụ: ta điều chỉnh Interval là minute và Breakdown là by _ignored thì có nghĩa là:

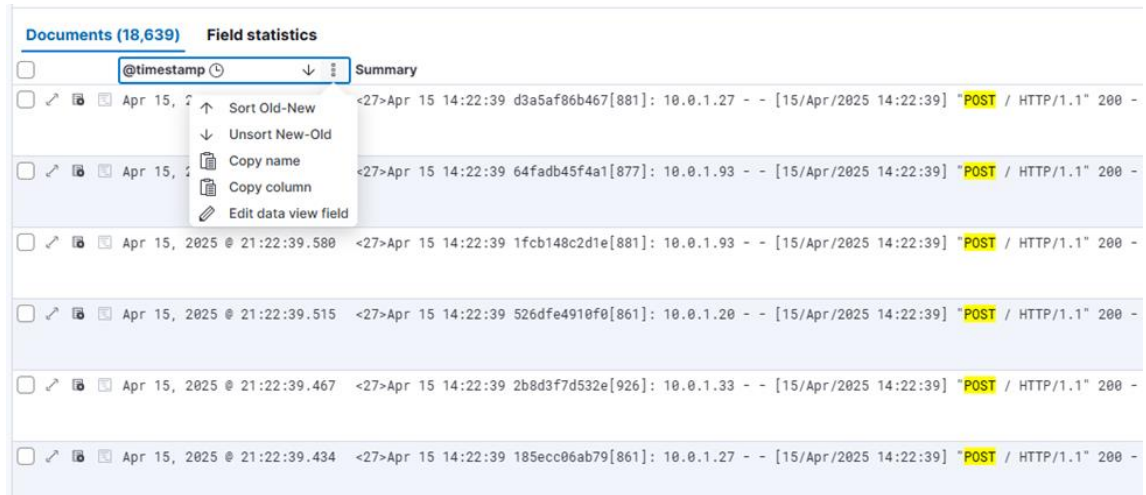
Interval: minute = Ta đang gộp các log lại theo từng phút (mỗi cột trên biểu đồ tương ứng với 1 phút).

Breakdown by _ignored = Phân nhóm các log dựa trên các field bị Elasticsearch bỏ qua (ignored) trong quá trình indexing(Elasticsearch không tạo chỉ mục index được).



5) Sắp xếp log theo thời gian hoặc theo phân loại

- Ở trên nơi hiện log đầu tiên thấy được, ta có thể chọn `@timestamp` để sắp xếp thời gian tùy theo mục ta chọn. Chẳng hạn ta chọn Sort Old-New thì các log sẽ được sắp xếp theo thời gian từ cũ nhất đến mới nhất.



4.7 Logstash xử lý lấy dữ liệu log theo filter với các thông tin quan trọng

4.8 Elasticsearch chạy trên swarm

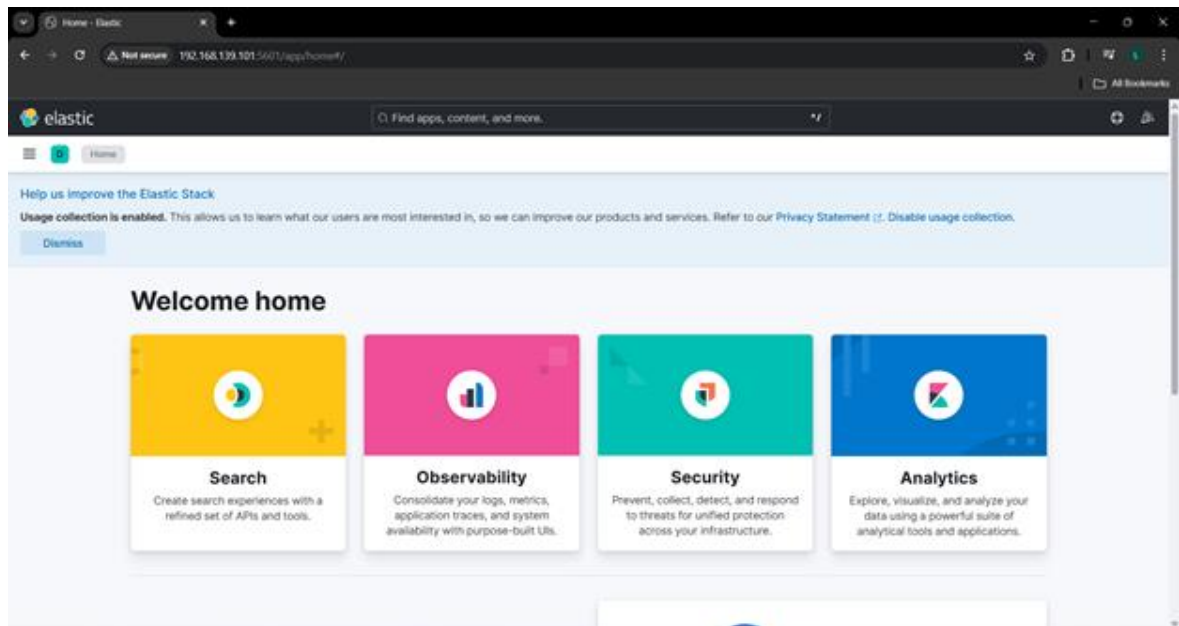
4.9 Truy vấn dữ liệu trong elasticsearch

4.10 Kibana chạy trên swarm

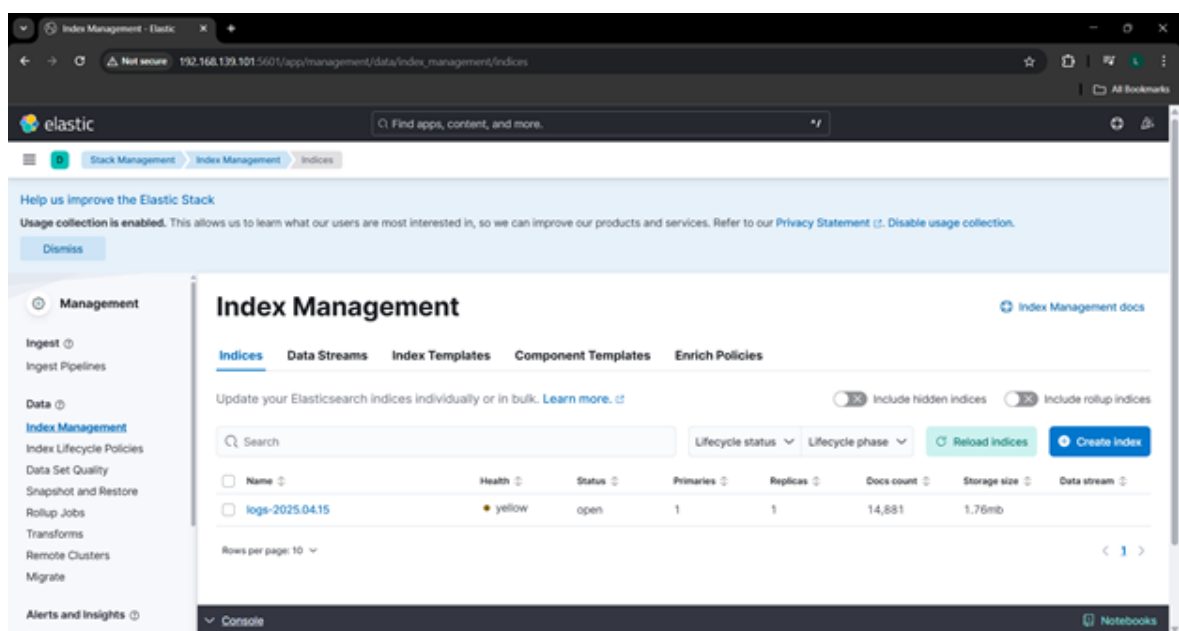
4.11 Kibana kết nối lấy dữ liệu thành công từ ElasticSearch

4.11.1 Truy cập Kibana: Kibana chạy trên port 5601 . Mở trình duyệt và truy cập: <http://192.168.139.101:5601>

- Nếu Kibana kết nối thành công với Elasticsearch, bạn sẽ thấy giao diện chính của Kibana (thường là trang **Welcome** hoặc **Discover** nếu đã thiết lập trước đó).



- **Kiểm tra trạng thái Elasticsearch từ Kibana:** Trong Kibana, vào **Stack Management > Index Management** để xem liệu Kibana có liệt kê được các index từ Elasticsearch (như logs-2025.04.15).



- Trong container Kibana chạy lệnh:
curl <http://elasticsearch:9200>

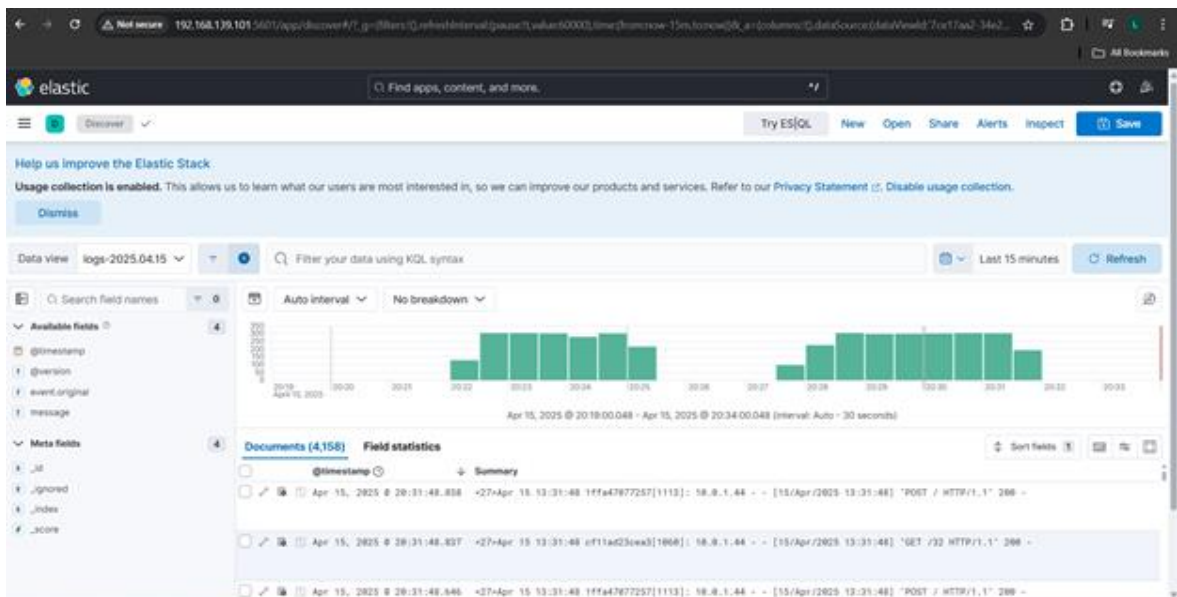
```
root@vps: /home/osboxes
kibana@2e7b13df3c4a:~$ curl http://elasticsearch:9200
{"name": "be894a3a97a0",
 "cluster_name": "docker-cluster",
 "cluster_uuid": "HqrsnradQ72hjN-W7cNjw",
 "version": {
  "number": "8.16.5",
  "build_flavor": "default",
  "build_type": "docker",
  "build_hash": "22f9957c211507e96c6295abfb74d12abff3a2ea",
  "build_date": "2025-02-27T22:08:27.576465563Z",
  "build_snapshot": false,
  "lucene_version": "9.12.0",
  "minimum_wire_compatibility_version": "7.17.0",
  "minimum_index_compatibility_version": "7.0.0"
 },
 "tagline": "You Know, for Search"
}
```

4.12 Thao tác Kibana tìm kiếm logs, trực quan, phân tích dữ liệu logs

4.12.1 Tìm kiếm logs

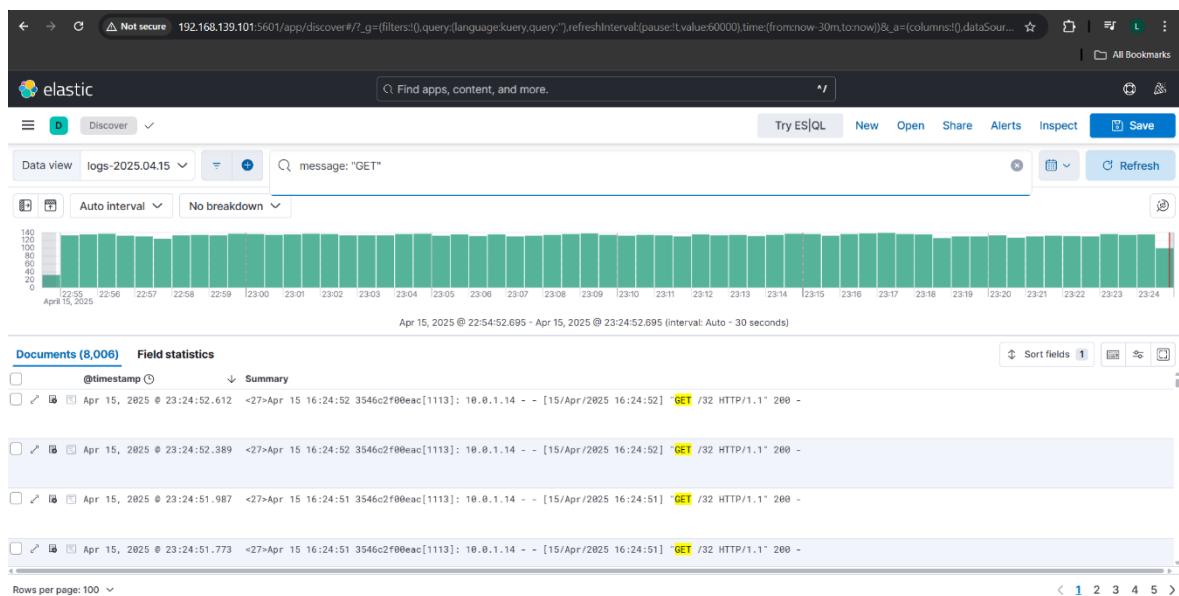
Vào Discover:

- Từ menu bên trái, chọn **Discover**.
- Chọn index pattern logs-2025.04.15 từ menu thả xuống.



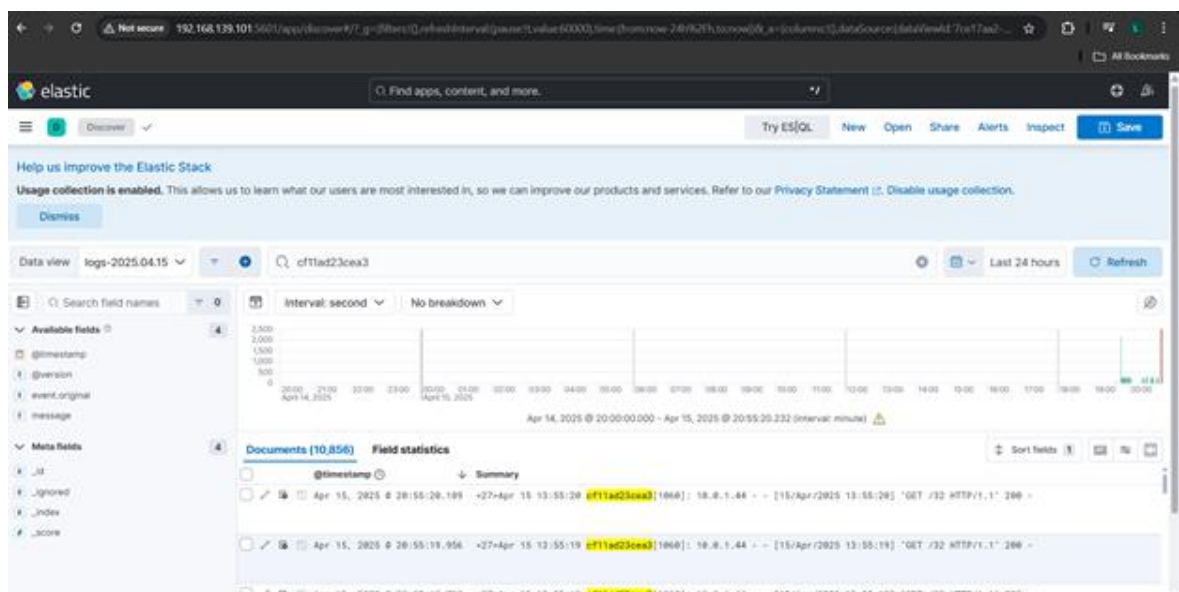
Ví dụ để lọc tất cả log có method GET:

Nhấn **Refresh** hoặc Enter để thực thi truy vấn.



Lọc theo thời gian:

Sử dụng **Time Picker** (góc trên bên phải) để chọn khoảng thời gian, ví dụ: "Last 24 hours" hoặc "Today".



4.12.2 Trực quan

Tạo Visualization:

- Từ menu bên trái, chọn Visualize Library > Create Visualization.

– Chọn loại biểu đồ, ví dụ:

+ **Vertical Bar**: Theo dõi số log theo thời gian

Ví dụ : Biểu đồ cột đếm log theo thời gian:

Chọn **Vertical Bar**.

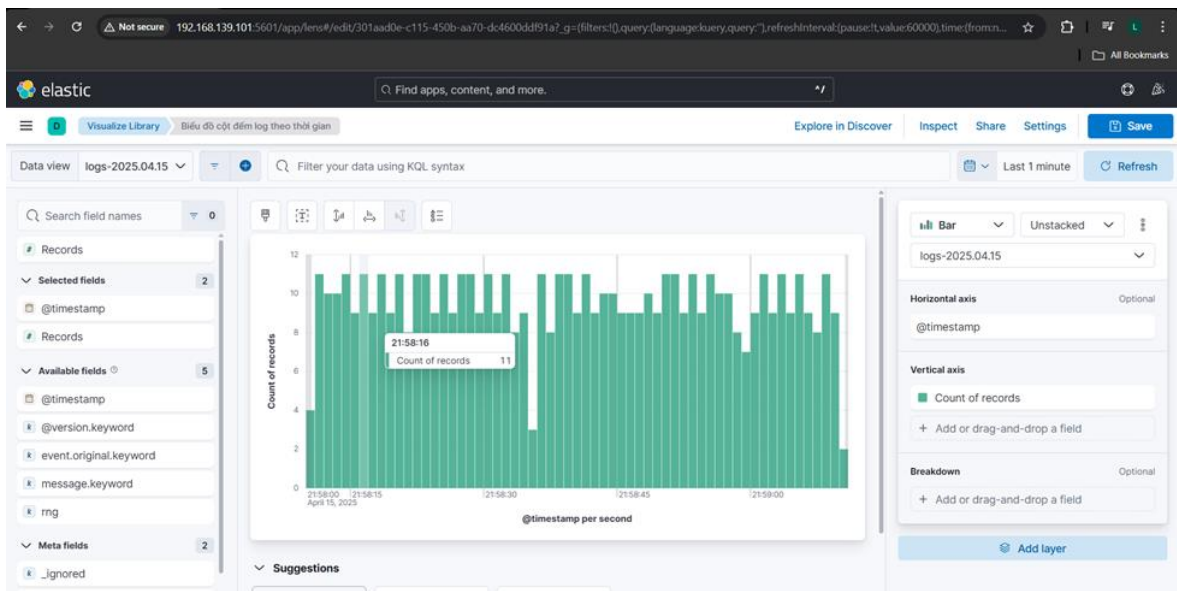
Cấu hình:

+ **Vertical axis**: Functions = Count.

+ **Horizontal axis**: Functions = Date Histogram, Field = @timestamp, Interval = Auto.

Nhấn **Apply Changes**.

Lưu với tên " Biểu đồ cột đếm log theo thời gian ".



4.12.3 Phân tích dữ liệu logs

@timestamp Thời điểm log được ghi lại

Summary: Nội dung log gồm: địa chỉ IP, thời gian, request và mã trạng thái

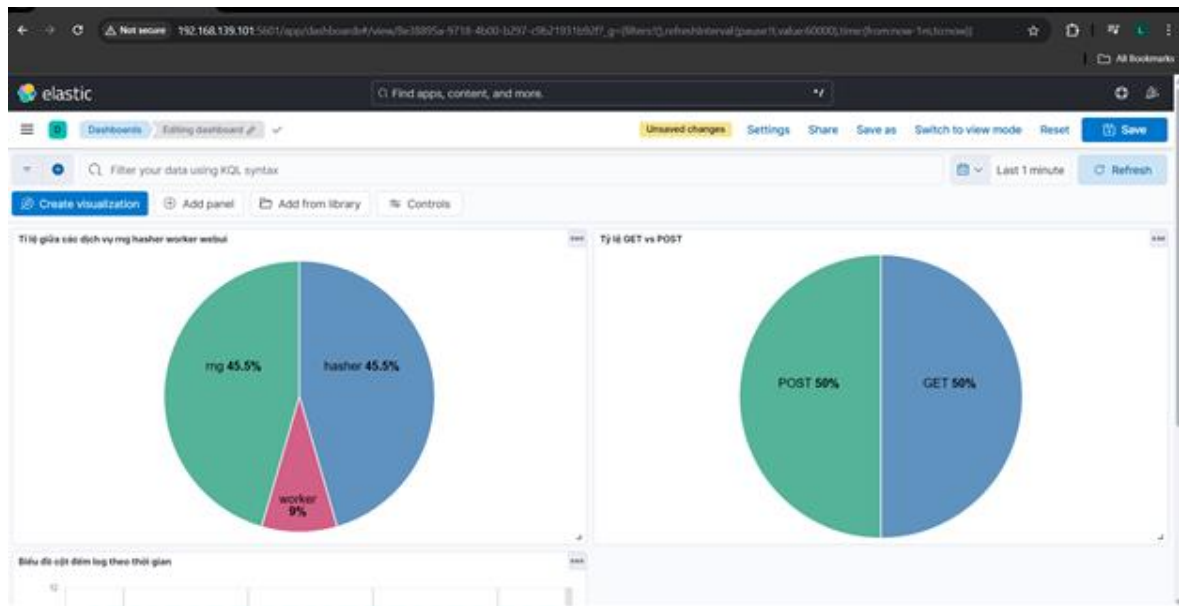
Documents (17,597)		Field statistics			Sort fields 1			
☐	@timestamp		Summary					
☐	🔍	🕒	Apr 15, 2025 @ 23:07:21.533	<27>Apr 15 16:07:21 1ffa47677257[1113]: 10.0.1.14 - - [15/Apr/2025 16:07:21] "POST / HTTP/1.1" 200 -				
☐	🔍	🕒	Apr 15, 2025 @ 23:07:21.527	<27>Apr 15 16:07:21 3546c2f00eac[1113]: 10.0.1.14 - - [15/Apr/2025 16:07:21] "GET /32 HTTP/1.1" 200 -				
☐	🔍	🕒	Apr 15, 2025 @ 23:07:21.316	<27>Apr 15 16:07:21 1ffa47677257[1113]: 10.0.1.14 - - [15/Apr/2025 16:07:21] "POST / HTTP/1.1" 200 -				
☐	🔍	🕒	Apr 15, 2025 @ 23:07:21.310	<27>Apr 15 16:07:21 3546c2f00eac[1113]: 10.0.1.14 - - [15/Apr/2025 16:07:21] "GET /32 HTTP/1.1" 200 -				
☐	🔍	🕒	Apr 15, 2025 @ 23:07:21.105	<27>Apr 15 16:07:21 7da80ff5da4b[1113]: INFO: __main__:5 units of work done, updating hash counter				
☐	🔍	🕒	Apr 15, 2025 @ 23:07:21.102	<27>Apr 15 16:07:21 1ffa47677257[1113]: 10.0.1.14 - - [15/Apr/2025 16:07:21] "POST / HTTP/1.1" 200 -				
☐	🔍	🕒	Apr 15, 2025 @ 23:07:21.096	<27>Apr 15 16:07:21 3546c2f00eac[1113]: 10.0.1.14 - - [15/Apr/2025 16:07:21] "GET /32 HTTP/1.1" 200 -				
☐	🔍	🕒	Apr 15, 2025 @ 23:07:20.888	<27>Apr 15 16:07:20 1ffa47677257[1113]: 10.0.1.14 - - [15/Apr/2025 16:07:20] "POST / HTTP/1.1" 200 -				
☐	🔍	🕒	Apr 15, 2025 @ 23:07:20.883	<27>Apr 15 16:07:20 3546c2f00eac[1113]: 10.0.1.14 - - [15/Apr/2025 16:07:20] "GET /32 HTTP/1.1" 200 -				

- Các yếu tố bên trong log
 - + **IP:** 10.0.1.14
 - + **Thời gian:** 15/Apr/2025:16:07:21
 - + Method: POST
 - + Path: /
 - + HTTP version: HTTP/1.1
 - + Status code: 200 (OK)

- Thống kê :

Phân tích tỉ lệ GET POST

Phân tích tỉ lệ các dịch vụ rng, hasher, worker, webui



4.13 Hiện thị các thông tin metric trên cụm như mục trên bằng lệnh CLI

4.13.1 CPU, RAM (active or inactive memory), disk usage

CPU: `top -bn1 | head -n 5`

```
osboxes@vps0:~$ top -bn1 | head -n 5
top - 13:37:46 up 1:04, 3 users, load average: 9.71, 13.04, 18.02
Tasks: 163 total, 6 running, 157 sleeping, 0 stopped, 0 zombie
%Cpu(s): 31.6 us, 21.1 sy, 0.0 ni, 36.8 id, 5.3 wa, 0.0 hi, 5.3 si, 0.0 st
MiB Mem : 3403.8 total, 132.9 free, 3111.8 used, 414.8 buff/cache
MiB Swap: 4096.0 total, 1945.8 free, 2150.2 used. 292.0 avail Mem
```

- Tổng mức sử dụng CPU: 68.09% user, 14.89% system
- CPU còn rảnh (idle): 13.83%
- Load trung bình (1/5/15 phút): 9.71 / 13.04 / 18.02

Ram: `free -h`

```
osboxes@vps0:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          3.3Gi        2.9Gi        210Mi        6.0Mi        434Mi        388Mi
Swap:          4.0Gi        2.2Gi        1.8Gi
```

- Tổng RAM: 3.3 GiB, đã dùng: 2.9 GiB (~88%), còn trống: 210 MiB
- Tổng Swap: 4.0 GiB, đã dùng: 2.2 GiB, còn trống: 1.8 GiB

Disk Usage: `df -h`

```
rsboxes@vps0:~$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	341M	29M	313M	9%	/run
/dev/mapper/ubuntu--vg-ubuntu--lv	98G	18G	76G	19%	/
tmpfs	1.7G	0	1.7G	0%	/dev/shm
tmpfs	5.0M	0	5.0M	0%	/run/lock
tmpfs	1.0M	0	1.0M	0%	/run/credentials/systemd-journald.service
tmpfs	1.0M	0	1.0M	0%	/run/credentials/systemd-tmpfiles-setup-dev-early.service
tmpfs	1.0M	0	1.0M	0%	/run/credentials/systemd-udev-load-credentials.service
tmpfs	1.0M	0	1.0M	0%	/run/credentials/systemd-sysctl.service
tmpfs	1.0M	0	1.0M	0%	/run/credentials/systemd-tmpfiles-setup-dev.service
tmpfs	1.7G	0	1.7G	0%	/tmp
/dev/sda2	2.0G	253M	1.6G	14%	/boot
tmpfs	1.0M	0	1.0M	0%	/run/credentials/systemd-tmpfiles-setup.service
tmpfs	1.0M	0	1.0M	0%	/run/credentials/systemd-networkd.service
tmpfs	1.0M	0	1.0M	0%	/run/credentials/systemd-resolved.service
tmpfs	1.0M	0	1.0M	0%	/run/credentials/getty@tty1.service
tmpfs	341M	12K	341M	1%	/run/user/1000

- Dung lượng ổ chính (/): 98 GB, đã dùng: 18 GB (19%)
- Phân vùng boot (/boot): 2.0 GB, đã dùng: 253 MB

4.13.2 Tất cả các process và state của chúng

```

ps -eo pid,user,stat,%cpu,%mem,cmd --sort=-%cpu
c6b0e6990:~$ ps -eo pid,user,stat,%cpu,%mem,cmd --sort=-%cpu
  PID USER      STAT  %CPU  %MEM     CMD
249657 osboxes  R+    300   0.1  ps -eo pid,user,stat,%cpu,%mem,cmd --sort=-%cpu
11102 osboxes  S1   13.5  16.6  /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddr 2832 osboxes Ssl 10.3 20.5 /usr/share/logstash/
jdk/bin/java -Xms1g -Xmx1g -Djava.awt.headless=true -Dfile.encoding= 249588 root S1 9.9 1.7 libnetwork-setkey -exec-root=/var/run/docker f96381192345d4352bc348cce70
cab74f060e82244 3265 osboxes S1 9.8 19.2 /usr/share/logstash/jdk/bin/java -Xms1g -Xmx1g -Djava.awt.headless=true -Dfile.encoding= 4291 osboxes S1 7.3 12.0 /usr
/share/kibana/bin/../node/glibc-217/bin/node /usr/share/kibana/bin/../src/cli/dist - 1057 root Ssl 6.9 2.9 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/c
containerd.sock
53646 root Ss 4.2 0.2 python worker.py
11454 osboxes Ss 2.7 0.2 /usr/lib/systemd/systemd --user
1 root Ss 2.7 0.3 /sbin/init
249549 root S1 1.9 0.3 runc --root /var/run/docker/runtime-runc/moby --log /run/containerd/io.containerd.runtim 351 root Rs 1.4 0.1 /usr/lib/systemd/sys
temd-udev
53926 root Ss 1.3 0.2 python hasher.py
2716 472 Ssl 1.3 2.1 grafana server --homepath=/usr/share/grafana --configs/etc/grafana/grafana.ini --packagi 54032 root Ss 1.2 0.2 python rng.py
225695 root I< 1.2 0.0 [kworker@0:1H:kblockd]
249531 root S1 1.1 0.3 /usr/bin/containerd-shim-runc-v2 -namespace moby -id f96381192345d4352bc348cce70cab74f06 3192 999 Ssl 1.0 0.0 redis-server *:6379
162963 root S 0.9 0.1 (udev-worker)
249592 root Ssl 0.9 0.3 runc init
162919 root S 0.8 0.1 (udev-worker)
3812 nobody Ssl 0.7 0.4 /bin/node_exporter --path.procfs=/host/proc --path.sysfs=/host/sys --path.rootfs=/rootfs 172516 root S 0.7 0.1 (udev-worker)
17 root I 0.6 0.0 [rcu_preempt]
906 message+ S 0.6 0.1 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation -sys 2850 nobody Ssl 0.6 1.3 /bin/prometheus --co
nfig_files=/etc/prometheus/prometheus.yml --storage.tsdb.path=/promet 525 systemd+ Ss 0.5 0.2 /usr/lib/systemd/systemd-networkd
204904 root I< 0.5 0.0 [kworker@0:2H:kblockd]
964 root Ssl 0.4 0.6 /usr/bin/containerd
162921 root S 0.4 0.1 (udev-worker)
46 root S 0.4 0.0 [kworker@0:1H:kblockd]
922 root Ssl 0.4 0.2 /usr/libexec/udisks2/udisksd
4365 osboxes S1 0.3 0.2 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server
233 root S 0.3 0.0 [jbd2/dm-0-8]
548 systemd+ S 0.3 0.1 /usr/lib/systemd/systemd-resolved
226534 root I 0.3 0.0 [kworker@0:2-mm_percpu_wq]
302 root S<s 0.3 1.5 /usr/lib/systemd/systemd-journald
46 root S 0.2 0.0 [ksppad0]
162966 root R 0.2 0.1 (udev-worker)
163198 root I 0.2 0.0 [kworker@0-tgroup_destroy]
1033 root S<l 0.1 0.1 (/usr/sbin/ModemManager)

```

Các process và state của chúng: Prometheus: Ss1, Grafana: Ss1 , Logstash: Ss1, Elasticsearch: S1, Kibana: S1, Node Exporter: Ss1,

4.13.3 Số lượng file đang mở, sockets và trạng thái của chúng

```
osboxes@vps0:~$ lsof | wc -l
```

Số lượng file đang mở: 63806

```

osboxes@vps0:~$ ss -s
Total: 311
TCP:    83 (estab 5, closed 62, orphaned 0, timewait 0)

Transport Total      IP        IPv6
RAW        1          0         1
UDP        5          4         1
TCP        21         9         12
INET       27        13        14
FRAG       0          0         0

```

Có total: 311 tức có 311 socket đang được mở

```

osboxes@vps0:~$ ss -ant | sort | uniq -c
  1 ESTAB 0      0      192.168.56.101:53858 192.168.56.101:5000
  1 ESTAB 0      0      192.168.56.101:53868 192.168.56.101:5000
  1 ESTAB 0      0      192.168.56.101:53872 192.168.56.101:5000
  1 ESTAB 0      0      192.168.56.101:53876 192.168.56.101:5000
  1 ESTAB 0     36      192.168.56.101:22      192.168.56.1:64736
  1 LISTEN 0     4096      0.0.0.0:22            0.0.0.0:*
  1 LISTEN 0     4096      0.0.0.0:9100          0.0.0.0:*
  1 LISTEN 0     4096    127.0.0.53:53         0.0.0.0:*
  1 LISTEN 0     4096    127.0.0.54:53         0.0.0.0:*
  1 LISTEN 0     4096      [::]:22              [::]:*
  1 LISTEN 0     4096      *:2377               *:.*
  1 LISTEN 0     4096      *:3000               *:.*
  1 LISTEN 0     4096      *:5000               *:.*
  1 LISTEN 0     4096      *:5601               *:.*
  1 LISTEN 0     4096      *:7946               *:.*
  1 LISTEN 0     4096      *:8000               *:.*
  1 LISTEN 0     4096      *:8001               *:.*
  1 LISTEN 0     4096      *:8002               *:.*
  1 LISTEN 0     4096      *:9090               *:.*
  1 LISTEN 0     4096      [::]:9100            [::]:*
  1 LISTEN 0     4096      *:9200               *:.*
  1 State Recv-Q Send-Q Local Address:Port Peer Address:Port

```

5 kết nối **ESTABLISHED** (Đã kết nối, đang truyền dữ liệu.)

Nhiều socket **LISTEN** (Đang chờ kết nối đến (server).)

62 kết nối đã **CLOSED** (đã đóng).

4.13.4 Các hoạt động của I/O (disk, network), trên thao tác hoặc dung lượng (volume)

- Theo dõi I/O trên Disk (Block I/O) và theo dõi Network I/O (in/out) của container: dùng lệnh `sudo docker stats`

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O
7f05						
cbfad3b1c3f210	webui.1.jum8f585pvtxgusr7jkua8h78	0.00%	1.176MiB / 3.324GiB	0.03%	10.1kB / 2.92kB	43.5MB / 23.8MB
d6b58547ab531	rng.1.cs7hc8qqgpk8i2tviemk83g6x	1.55%	6.262MiB / 3.324GiB	0.18%	8.89MB / 8.14MB	6.91MB / 19MB
ca6d5e35b1861	hasher.1.xfe78cnoy7860vr9y68xgzktc	2.11%	7.406MiB / 3.324GiB	0.22%	11.3MB / 8.88MB	25.1MB / 18.4MB
3617cfd7a0881	worker.1.6j8lk2114qmd1jmddepj5pkzq	4.54%	9.848MiB / 3.324GiB	0.29%	17.4MB / 20.9MB	38.4MB / 17.9MB
16de34e40cf8	monitoring_prometheus.1.wb47z2x70kumd98uvkpguwp55	0.03%	42MiB / 3.324GiB	1.23%	7.61MB / 1.49MB	196MB / 21.9MB
f2e04ec8e1b68	monitoring_node-exporter.3vbq0z00a67h98lvzvov51n8.y8drjj7u1lvogtopk35d0dgyk	0.00%	9.941MiB / 3.324GiB	0.29%	265kB / 6.02MB	15.1MB / 1.39MB
a1a8da00078316	kibana.1.vmfyx6ojm2e5mi0vxwoudwt7e	7.54%	445.2MiB / 3.324GiB	13.08%	8.48MB / 68.2MB	1.28GB / 152MB
93c0ec23fd4812	logstash.1.fmqmqm1mmtttvbotmij31kkce	3.47%	553.4MiB / 3.324GiB	16.26%	7.79MB / 12.7MB	509MB / 481MB
0b691f2c77be43	grafana.1.j9zqs6aplqccg6kphkkhymble	0.04%	59.05MiB / 3.324GiB	1.73%	5.42MB / 7.35MB	455MB / 82.5MB
5b4ecc3bfff0298	elasticsearch.1.qe7nnkv9wc6fkr860numiqck4	3.33%	591.9MiB / 3.324GiB	17.39%	90.1MB / 17.2MB	1.15GB / 2.88GB
3d0e4636603c42	logstash.2.yrppv22m8x160w7r1ubsj7jv5	2.84%	513.2MiB / 3.324GiB	15.08%	6.88MB / 11.7MB	406MB / 526MB

- Theo dõi Volume trong Swarm

```
osboxes@vps0:~$ sudo docker volume ls
DRIVER      VOLUME NAME
local       1fa7ab46dd54b0f9d9e558ffbb7ed56c77574a2dd10bfe354e7c658d0595d37a
local       4cda4cee4507d3db97e814bbe2300a5d874e38d12ecdc1756f3314ad2be2db4f
local       67c5f5aee515ec7c14b0669615010bea24316e4ac99a4fc55e65442be051f65c
local       87b0a6c72fa8e5d83c0f230be5f6b4d9a991bf7b11ac84bcef8644da592075b8
local       7253d9b8d58f4e1dccc486dc06532b710848e637e32ba225618f7fbc438d8e8
local       90132dc3ba0506c65b46698c4e874516dde101e3fcb3c79f9ea90de897f2b1a5
local       b5cea3e0db5e7f4be77589c2ae46afed80628f06be9887a3fe4c43ddb79dcd83
local       bbed1cf869c7990ff85b165359183b86f10395c415e4f0762031f9f70947dbe4
local       d51df2c5d2a228eed4716e4b49d32d6480e5f4d29dc097cf33d1432445c9d54f
local       d266d372d8cc82c626acbc18ccb6020ea9ef8c50073224655f3b97d81d0e5c9a
local       dc9639e7abff50fc99ea076bdd9776956b2dfa14647458f9dd8077e8ff2130b42
```

- Xem chi tiết volume

```
osboxes@vps0:~$ sudo docker volume inspect 1fa7ab46dd54b0f9d9e558ffbb7ed56c77574a2dd10bfe354e7c658d0595d37a
[
  {
    "CreatedAt": "2025-02-26T12:33:36Z",
    "Driver": "local",
    "Labels": {
      "com.docker.volume.anonymous": ""
    },
    "Mountpoint": "/var/lib/docker/volumes/1fa7ab46dd54b0f9d9e558ffbb7ed56c77574a2dd10bfe354e7c658d0595d37a/_data",
    "Name": "1fa7ab46dd54b0f9d9e558ffbb7ed56c77574a2dd10bfe354e7c658d0595d37a",
    "Options": null,
    "Scope": "local"
  }
]
```

4.14 Trình diễn cơ chế tạo pipeline thu thập và lưu trữ các metric với

Prometheus với CLI

- Prometheus là hệ thống giám sát và cảnh báo mã nguồn mở được sử dụng rộng rãi để thu thập và lưu trữ dữ liệu dạng time series.
- Node Exporter là một datasource được viết bằng Go nhưng nó sẽ thiên về việc lấy các dữ liệu phần cứng từ hệ thống của chúng ta hơn, như các thông số về CPU.
- **Cài đặt Node Exporter:** docker service create --name node-exporter --mode global --network monitoringnet -p 9100:9100 prom/node-exporter:v1.8.1

- cAdvisor là 1 dự án Open source của Google với mục đích để phân tích mức độ sử dụng, hiệu năng, và rất nhiều thông số khác từ các ứng dụng Container, cung cấp cho người dùng cái nhìn tổng quan về toàn bộ các container đang chạy.
- **Cài đặt cAdvisor:** `docker service create --name cadvisor --mode global --network monitoringnet --mount type=bind,source=/,destination=/rootfs:ro --mount type=bind,source=/var/run,destination=/var/run:ro --mount type=bind,source=/sys,destination=/sys:ro --mount type=bind,source=/var/lib/docker/,destination=/var/lib/docker:ro --publish mode=host,target=8080,published=8080 gcr.io/cadvisor/cadvisor:v0.47.2`

```
root@vps0:/home/osboxes# docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
69teyrtqiyqj0	cadvisor	global	2/2	gcr.io/cadvisor/cadvisor:v0.47.2	
3b9fs1v1i0jc	elasticsearch	replicated	1/1	elasticsearch:8.16.5	*:9200->9200/tcp
ifmrkuc5hykm	grafana	replicated	1/1	grafana/grafana:10.4.1	*:3000->3000/tcp
7o7cv00b6meh	hasher	replicated	1/1	vutran123/hasher:coinswarm	*:8002->80/tcp
qnoav9pp6ern	influxdb	replicated	1/1	influxdb:2	*:8086->8086/tcp
l5tj4wsi9ui7	kibana	replicated	1/1	kibana:8.16.5	*:5601->5601/tcp
at1l2yov8t8e	logstash	replicated	1/1	logstash:8.16.5	*:5000->5000/tcp
nrcjr0dlfok2	node-exporter	global	2/2	prom/node-exporter:v1.8.1	*:9100->9100/tcp
n9cfg2z2b6um	prometheus	replicated	1/1	prom/prometheus:v2.52.0	*:9090->9090/tcp
znt01zb709wn	redis	replicated	1/1	redis:latest	
u8uzk8mz3au9	rng	replicated	1/1	vutran123/rng:coinswarm	*:8001->80/tcp
phqcdbr1l6wr	webui	replicated	1/1	vutran123/webui:coinswarm	*:8000->80/tcp
ubc810z0d5m9	worker	replicated	2/2	vutran123/worker:coinswarm	

- Tạo một file với tên là prometheus.yml với nội dung như sau:

```
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'node_exporters'
    static_configs:
      - targets: ['192.168.56.101:9100', '192.168.56.102:9100']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['192.168.56.101:8080', '192.168.56.102:8080']

remote_write:
  - url: "http://influxdb:8086/api/v1/prom/write"
```

- Cài đặt Prometheus:
- `docker config create prometheus_config ./prometheus.yml`

- docker service create --name prometheus --config source=prometheus_config,target=/etc/prometheus/prometheus.yml --network monitoringnet --publish 9090:9090 prom/prometheus:v2.52.0

```
root@vps0:/home/osboxes# docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
69teyrtqiyqj0    cadvisor            global              2/2                 gcr.io/cadvisor/cadvisor:v0.47.2
3b9fs1vli0jc     elasticsearch       replicated          1/1                 elasticsearch:8.16.5
ifmrkuc5hykm     grafana             replicated          1/1                 grafana/grafana:10.4.1
7o7cv00b6meh     hasher             replicated          1/1                 vutran123/hasher:coinswarm
qnoav9pp6ern     influxdb            replicated          1/1                 influxdb:2
l5tj4wsiu17      kibana              replicated          1/1                 kibana:8.16.5
at1l2yov8t8e     logstash            replicated          1/1                 logstash:8.16.5
nrcjr0dlfok2     node-exporter       global              2/2                 prom/node-exporter:v1.8.1
n9cfg2z2b6um     prometheus          replicated          1/1                 prom/prometheus:v2.52.0
znt01zb709wn     redis               replicated          1/1                 redis:latest
u8uzk8mz3au9     rng                 replicated          1/1                 vutran123/rng:coinswarm
phqcdbr116wr     webui               replicated          1/1                 vutran123/webui:coinswarm
ubc810z0d5m9     worker              replicated          2/2                 vutran123/worker:coinswarm
```

Targets

All scrape pools ▾ All Unhealthy Collapse All 🔍 Filter by endpoint or labels

cadvisor (2/2 up) [Show logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.56.101:8080/metrics	UP	Instance="192.168.56.101:8080" Job="cadvisor"	750.000ms ago	15.511ms	
http://192.168.56.102:8080/metrics	UP	Instance="192.168.56.102:8080" Job="cadvisor"	-421.000ms ago	12.369ms	

node_exporters (2/2 up) [Show logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.56.101:9100/metrics	UP	Instance="192.168.56.101:9100" Job="node_exporters"	2.961s ago	52.248ms	
http://192.168.56.102:9100/metrics	UP	Instance="192.168.56.102:9100" Job="node_exporters"	2.995s ago	44.767ms	

- Dùng CLI (curl, promtool, http) để kiểm tra pipeline:
- Kiểm tra cAdvisor: curl http://localhost:8080/metrics

```
root@vps0:/home/osboxes# curl http://192.168.56.101:9091/metrics
curl: (7) Failed to connect to 192.168.56.101 port 9091 after 0 ms: Could not connect to server
root@vps0:/home/osboxes# curl http://192.168.56.101:8080/metrics
# HELP cadvisor_version_info A metric with a constant '1' value labeled by kernel version, OS version, docker version
, cadvisor version & cadvisor revision.
# TYPE cadvisor_version_info gauge
cadvisor_version_info{cadvisorRevision="1a19abb0",cadvisorVersion="v0.47.2",dockerVersion="",kernelVersion="6.11.0-19
-generic",osVersion="Alpine Linux v3.16"} 1
# HELP container_blkio_device_usage_total counter
# TYPE container_blkio_device_usage_total counter
container_blkio_device_usage_total{device="/dev/mapper/ubuntu--vg-ubuntu--lv",id="/",major="252",minor="0",operation=
"Read"} 3.0900224e+07 1744720411451
container_blkio_device_usage_total{device="/dev/mapper/ubuntu--vg-ubuntu--lv",id="/",major="252",minor="0",operation=
"Write"} 0 1744720411451
container_blkio_device_usage_total{device="/dev/sda",id="/",major="8",minor="0",operation="Read"} 3.0900224e+07 17447
20411451
container_blkio_device_usage_total{device="/dev/sda",id="/",major="8",minor="0",operation="Write"} 0 1744720411451
```

- Kiểm tra node exporter: curl http://localhost:9100/metrics

```

root@vps0:/home/osboxes/monitoring/prometheus# curl http://192.168.56.101:9100/metrics | head
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           Dload  Upload  Total   Spent    Left     Speed

  0      0    0     0    0     0      0      0      0      0      0  0:00:00  0:00:00 --:--:--    0# HELP go_gc_duration_seconds A summary
of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 2.4014e-05
go_gc_duration_seconds{quantile="0.25"} 3.1902e-05
go_gc_duration_seconds{quantile="0.5"} 3.7517e-05
go_gc_duration_seconds{quantile="0.75"} 4.9981e-05
go_gc_duration_seconds{quantile="1"} 0.001611379
go_gc_duration_seconds_sum 0.075564753
go_gc_duration_seconds_count 1174
# HELP go_goroutines Number of goroutines that currently exist.
100 8192    0 8192    0    276k    0 --:--:-- --:--:-- --:--:-- 285k
curl: (23) Failure writing output to destination, passed 2048 returned 0
root@vps0:/home/osboxes/monitoring/prometheus#

```

4.15 Đồng bộ hóa dữ liệu từ Prometheus với InfluxDB

Prometheus là một monitoring toolkit thu thập time-series metrics từ các node trong Swarm thông qua Node Exporter, trong khi InfluxDB là time-series database lưu trữ metrics dài hạn. Đồng bộ hóa được thực hiện bằng remote_write, cho phép Prometheus đẩy metrics trực tiếp đến InfluxDB qua HTTP endpoint, sử dụng overlay network monitoringnet để đảm bảo giao tiếp an toàn và scalable.

Phân Tích Cấu Hình

Lệnh 1: Tạo Overlay Network (ssh vào máy manager vps0 và thực hiện các lệnh sau)

```
sudo docker network create --scope=swarm --attachable -d overlay --  
opt encrypted monitoringnet
```

- Mục đích: Tạo network monitoringnet để kết nối các service (Prometheus, InfluxDB, Grafana, Node Exporter).
- Chi tiết:
 - --scope=swarm: Network hoạt động trong Swarm mode, hỗ trợ multi-host communication.
 - --attachable: Cho phép container ngoài stack kết nối, tăng tính linh hoạt.
 - -d overlay: Driver overlay hỗ trợ giao tiếp giữa node trong Swarm.
 - --opt encrypted: Mã hóa traffic, bảo vệ dữ liệu metrics khi truyền giữa node.

Lệnh 2: xác nhận Network đã thiết lập

```
docker network ls
```

```

osboxes@vps0:~$ sudo docker network ls
[sudo] password for osboxes:
Sorry, try again.
[sudo] password for osboxes:
NETWORK ID          NAME                DRIVER              SCOPE
174ba3926d82        bridge              bridge              local
f936c2qhtl1s        coinswarmnet        overlay             swarm
03ddeacc2c12        docker_gwbridge     bridge              local
1b8902a8b9a5        host                host                local
7g729dgjbk2o        ingress             overlay             swarm
s28h6sga95dr        loggingnet          overlay             swarm
523x346rq3vf        monitoringnet       overlay             swarm
ba2a96473de1        none                null                local
pwrr64gh0pef        proxy               overlay             swarm
osboxes@vps0:~$

```

Lệnh 3: Tạo File Cấu Hình Prometheus, mục đích để tạo và chỉnh sửa file prometheus.yml nhằm cấu hình scrape targets và remote_write.

touch prometheus.yml

nano prometheus.yml

Nội

dung

file:

```

GNU nano 8.1 prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'node1'
    static_configs:
      - targets: ['192.168.56.101:9100']

  - job_name: 'node2'
    static_configs:
      - targets: ['192.168.56.102:9100']

remote_write:
  - url: "http://influxdb:8086/api/v1/prom/write?db=metrics"

```

Chú thích file:

- scrape_interval: 15s: Thu thập metrics mỗi 15 giây.
- scrape_configs: Thu thập từ:

Service node-exporter (port 9100) chạy global trên mọi node.

Node cụ thể: VPS1 (192.168.56.101:9100), VPS2 (192.168.56.102:9100).

- remote_write: Đẩy metrics sang InfluxDB database metrics.

Lệnh 4: Deploy InfluxDB

```
docker service create --name prometheus --network monitoringnet --publish published=9090,target=9090 --mount type=bind,source=/home/osboxes/prometheus.yml,target=/etc/prometheus/prometheus.yml prom/prometheus
```

Lệnh 5: Deploy InfluxDB

```
docker service create --name influxdb --network monitoringnet --publish published=8086,target=8086 --mount type=volume,source=influxdb-data,target=/var/lib/influxdb -e INFLUXDB_DB=metrics influxdb:1.8
```

Mục đích: Chạy InfluxDB để nhận metrics từ Prometheus và lưu trữ trong database metrics.

Chi tiết:

- --name influxdb: Tên service.
- --network monitoringnet: Kết nối overlay network.
- --publish published=8086,target=8086: Map port 8086 cho HTTP API.
- --mount type=volume,...: Persistent volume để lưu dữ liệu.
- -e INFLUXDB_DB=metrics: Tạo database metrics.
- influxdb:1.8: Image InfluxDB phiên bản 1.8.

Lệnh 6: Triển khai Node Exporter trên toàn bộ các node trong Swarm

```
docker service create --name node-exporter --mode global --network monitoringnet --mount type=bind,src=/proc,dst=/host/proc --mount type=bind,src=/sys,dst=/host/sys --mount type=bind,src=/,dst=/rootfs,readonly --mount type=bind,src=/etc/hostname,dst=/etc/host_hostname,readonly --publish mode=host,target=9100,published=9100,protocol=tcp prom/node-exporter --path.procfs=/host/proc --path.sysfs=/host/sys --path.rootfs=/rootfs --no-collector.arp
```

Kết quả: Metrics từ Swarm node được Prometheus thu thập và đồng bộ liên tục vào InfluxDB database metrics. Cấu hình đảm bảo fault tolerance

(persistent volume) và scalability (overlay network), sẵn sàng cho truy vấn và visualization.

```
osboxes@vps0:~$ sudo docker service ls
[sudo] password for osboxes:
ID                NAME                MODE                REPLICAS    IMAGE                                  PORTS
ljslykhyj8xk     coinswarm-proxy     replicated          1/1         nginx:latest                        *:8080->80/tcp
9yqmf70tqj8s     elasticsearch        replicated          1/1         elasticsearch:8.16.5               *:9200->9200/tcp
ybdapscs318r     grafana              replicated          1/1         grafana/grafana:latest              *:3000->3000/tcp
y1pr8k14mk1j     hasher               replicated          1/1         trinhnamtrung/hasher:coinswarm     *:8002->80/tcp
qctfcg58tq8c     influxdb             replicated          1/1         influxdb:1.8                        *:8086->8086/tcp
inrmupz2qhd0     kibana               replicated          1/1         kibana:8.16.5                       *:5601->5601/tcp
5xhwm6zre3zo     logstash             replicated          1/1         logstash:8.16.5                     *:5000->5000/tcp
bt5lkkrv6nfa     node-exporter        global              3/3         prom/node-exporter:latest
tezreydtu03x     prometheus           replicated          1/1         prom/prometheus:latest              *:9090->9090/tcp
c4btw2jcps01     redis                replicated          1/1         redis:latest
ark9ktqktfni     rng                  replicated          1/1         trinhnamtrung/rng:coinswarm         *:8001->80/tcp
mgo7qc2hug4k     webui                replicated          1/1         trinhnamtrung/webui:coinswarm       *:8000->80/tcp
z8olp2g23iyk     worker               replicated          2/2         trinhnamtrung/worker:coinswarm
```

4.16 Truy vấn dữ liệu metrics được thu thập được lưu trữ trong InfluxDB chạy trên swarm

InfluxDB lưu trữ metrics từ Prometheus trong database metrics, hỗ trợ truy vấn bằng InfluxQL (InfluxDB 1.8) để phân tích hiệu suất Swarm node (CPU, memory, disk, v.v.). InfluxDB chạy như service trên Swarm.

Truy vấn từ máy ảo chứa InfluxDB

Phương pháp 1: InfluxDB CLI

Mục đích: Chạy InfluxQL trực tiếp trong container InfluxDB để truy vấn chi tiết.

Thực hiện:

Lệnh 1: ssh vào máy chứa InfluxDB

ssh osboxes@192.168.56.102

Lệnh 2: Tìm container InfluxDB

docker ps

Lệnh 3: Truy cập container (Thay <container-id> bằng ID thực tế.)

docker exec -it <container-id> bash

Lệnh 4: Chạy Influx CLI

Influx

Lệnh 5: thực hiện truy vấn

SHOW DATABASES;

```
osboxes@vps2:~$ sudo docker exec -it 92f9bd8f4915 bash
root@92f9bd8f4915:/# influx
Connected to http://localhost:8086 version 1.8.10
InfluxDB shell version: 1.8.10
> SHOW DATABASES;
name: databases
name
----
metrics
_internal
```

Lệnh 6: Truy vấn

USE metrics

SELECT * FROM "node_memory_MemAvailable_bytes" WHERE
time >= now() - 30m

Lệnh 7: Lấy memory available trong 30 phút.

SELECT MEAN(value) FROM "node_cpu_seconds_total" WHERE time >=
now() - 1h GROUP BY time(10s)

Lệnh 8: Thoát

Thoát CLI: exit.

Thoát container: exit.

Phương pháp 2: HTTP API

Mục đích: Gửi truy vấn qua HTTP từ VPS2, không cần vào container.

Thực hiện:

Lệnh 1: Cài đặt và kiểm tra curl

sudo apt-get update && sudo apt-get install curl

curl -version

```
osboxes@vps0:~$ curl --version
curl 8.9.1 (x86_64-pc-linux-gnu) libcurl/8.9.1 OpenSSL/3.3.1 zlib/1.3.1 brotli/1.1.0 zstd/1.5.6 libidn2/2.3.7 libpsl/0.
1.2 libssh2/1.11.0 nghttp2/1.62.1 librtmp/2.3 OpenLDAP/2.6.8
Release-Date: 2024-07-31, security patched: 8.9.1-2ubuntu2.2
Protocols: dict file ftp ftps gopher gophers http https imap imaps ipfs ipns ldap ldaps mqtt pop3 pop3s rtmp rtsp scp s
tp smb smbs smtp smtps telnet tftp ws wss
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL
hreadsafe TLS-SRP UnixSockets zstd
osboxes@vps0:~$
```

Lệnh 2: Chạy truy vấn

```
curl -G http://192.168.56.102:8086/query --data-urlencode
"db=metrics" --data-urlencode "q=SELECT * FROM
\"node_cpu_seconds_total\" LIMIT 5"
```

```
osboxes@vps0:~$ curl -G http://192.168.56.102:8086/query --data-urlencode "db=metrics" --data-urlencode "q=SELECT * FROM
\"node_cpu_seconds_total\" LIMIT 5"
{"results": [{"statement_id": 0, "series": [{"name": "node_cpu_seconds_total", "columns": ["time", "_name_", "cpu", "instance",
"job", "value"], "values": [{"2025-04-14T16:57:30.062Z", "node_cpu_seconds_total", "0", "node-exporter:9100", "node", "idl
e", 1343.57}, {"2025-04-14T16:57:30.062Z", "node_cpu_seconds_total", "0", "node-exporter:9100", "node", "iowait", 117.96}, {"2025
-04-14T16:57:30.062Z", "node_cpu_seconds_total", "0", "node-exporter:9100", "node", "irq", 0}, {"2025-04-14T16:57:30.062Z", "nod
e_cpu_seconds_total", "0", "node-exporter:9100", "node", "nice", 0}, {"2025-04-14T16:57:30.062Z", "node_cpu_seconds_total", "0",
"node-exporter:9100", "node", "softirq", 504.42}]}]}]}
osboxes@vps0:~$
```

Lấy 5 record của node_cpu_seconds_total.

```
osboxes@vps0:~$ curl -G http://192.168.56.102:8086/query --data-urlencode "db=metrics" --data-urlencode "q=SELECT MEAN(v
alue) FROM \"node_cpu_seconds_total\" WHERE time >= now() - 1h GROUP BY time(10s)"
{"results": [{"statement_id": 0, "series": [{"name": "node_cpu_seconds_total", "columns": ["time", "mean"], "values": [{"2025-04-1
5T15:13:50Z", null}, {"2025-04-15T15:14:00Z", 86.545625}, {"2025-04-15T15:14:10Z", 87.9875}, {"2025-04-15T15:14:20Z", null}, {"2
025-04-15T15:14:30Z", 89.43125}, {"2025-04-15T15:14:40Z", 90.860625}, {"2025-04-15T15:14:50Z", null}, {"2025-04-15T15:15:00Z",
92.32}, {"2025-04-15T15:15:10Z", 93.67875}, {"2025-04-15T15:15:20Z", null}, {"2025-04-15T15:15:30Z", 94.908125}, {"2025-04-15T1
5:15:40Z", 96.42375000000001}, {"2025-04-15T15:15:50Z", null}, {"2025-04-15T15:16:00Z", 97.93}, {"2025-04-15T15:16:10Z", 99.438
12499999999}, {"2025-04-15T15:16:20Z", null}, {"2025-04-15T15:16:30Z", 100.92000000000002}, {"2025-04-15T15:16:40Z", 102.39875
}, {"2025-04-15T15:16:50Z", null}, {"2025-04-15T15:17:00Z", 103.890625}, {"2025-04-15T15:17:10Z", 105.41749999999999}, {"2025-0
4-15T15:17:20Z", null}, {"2025-04-15T15:17:30Z", 106.923125}, {"2025-04-15T15:17:40Z", 108.42437499999999}, {"2025-04-15T15:17
:50Z", null}, {"2025-04-15T15:18:00Z", 109.88874999999999}, {"2025-04-15T15:18:10Z", 111.36749999999999}, {"2025-04-15T15:18:2
0Z", null}, {"2025-04-15T15:18:30Z", 112.82687499999999}, {"2025-04-15T15:18:40Z", 114.290625}, {"2025-04-15T15:18:50Z", null},
{"2025-04-15T15:19:00Z", 115.754375}]}]}]}
osboxes@vps0:~$
```

Tính CPU usage trung bình mỗi 10 giây trong 1 giờ.

```
osboxes@vps0:~$ curl -G http://192.168.56.102:8086/query --data-urlencode "db=metrics" --data-urlencode "q=SELECT * FROM
\"node_memory_MemAvailable_bytes\" WHERE time >= now() - 30m"
{"results": [{"statement_id": 0, "series": [{"name": "node_memory_MemAvailable_bytes", "columns": ["time", "_name_", "instance",
"job", "value"], "values": [{"2025-04-15T15:44:31.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 140
9478656}, {"2025-04-15T15:44:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409478656}, {"2025-04-
15T15:44:46.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409490944}, {"2025-04-15T15:44:47.617Z",
"node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409499136}, {"2025-04-15T15:45:01.722Z", "node_memory_MemAv
ailable_bytes", "192.168.56.101:9100", "node1", 1409556480}, {"2025-04-15T15:45:02.614Z", "node_memory_MemAvailable_bytes", "n
ode-exporter:9100", "node", 1409560576}, {"2025-04-15T15:45:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100",
"node1", 1409691648}, {"2025-04-15T15:45:17.622Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409695744},
{"2025-04-15T15:45:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409720320}, {"2025-04-15T15:
45:32.624Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409724416}, {"2025-04-15T15:45:46.726Z", "node_m
em_Available_bytes", "192.168.56.101:9100", "node1", 1409749856}, {"2025-04-15T15:45:47.614Z", "node_memory_MemAvailable_byt
es", "node-exporter:9100", "node", 1409753952}, {"2025-04-15T15:46:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100",
"node1", 1409779392}, {"2025-04-15T15:46:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409783488}, {"2025-04-15T15:46:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409808928}, {"2025-04-15T15:46:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409813024}, {"2025-04-15T15:46:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409838368}, {"2025-04-15T15:46:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409842464}, {"2025-04-15T15:46:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409863808}, {"2025-04-15T15:46:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409867904}, {"2025-04-15T15:47:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409889248}, {"2025-04-15T15:47:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409893344}, {"2025-04-15T15:47:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409914592}, {"2025-04-15T15:47:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409918688}, {"2025-04-15T15:47:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409940032}, {"2025-04-15T15:47:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409944128}, {"2025-04-15T15:47:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409965376}, {"2025-04-15T15:47:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409969472}, {"2025-04-15T15:48:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1409990720}, {"2025-04-15T15:48:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1409994816}, {"2025-04-15T15:48:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410016064}, {"2025-04-15T15:48:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410020160}, {"2025-04-15T15:48:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410041408}, {"2025-04-15T15:48:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410045504}, {"2025-04-15T15:48:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410066752}, {"2025-04-15T15:48:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410070848}, {"2025-04-15T15:49:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410092096}, {"2025-04-15T15:49:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410096192}, {"2025-04-15T15:49:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410117440}, {"2025-04-15T15:49:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410121536}, {"2025-04-15T15:49:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410142784}, {"2025-04-15T15:49:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410146880}, {"2025-04-15T15:49:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410168128}, {"2025-04-15T15:49:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410172224}, {"2025-04-15T15:50:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410193472}, {"2025-04-15T15:50:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410197568}, {"2025-04-15T15:50:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410218816}, {"2025-04-15T15:50:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410222912}, {"2025-04-15T15:50:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410244160}, {"2025-04-15T15:50:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410248256}, {"2025-04-15T15:50:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410269504}, {"2025-04-15T15:50:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410273600}, {"2025-04-15T15:51:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410290752}, {"2025-04-15T15:51:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410294848}, {"2025-04-15T15:51:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410316096}, {"2025-04-15T15:51:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410320192}, {"2025-04-15T15:51:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410341440}, {"2025-04-15T15:51:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410345536}, {"2025-04-15T15:51:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410366784}, {"2025-04-15T15:51:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410370880}, {"2025-04-15T15:52:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410392128}, {"2025-04-15T15:52:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410396224}, {"2025-04-15T15:52:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410417472}, {"2025-04-15T15:52:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410421568}, {"2025-04-15T15:52:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410442816}, {"2025-04-15T15:52:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410446912}, {"2025-04-15T15:52:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410468160}, {"2025-04-15T15:52:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410472256}, {"2025-04-15T15:53:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410489312}, {"2025-04-15T15:53:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410493408}, {"2025-04-15T15:53:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410514560}, {"2025-04-15T15:53:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410518656}, {"2025-04-15T15:53:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410539808}, {"2025-04-15T15:53:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410543904}, {"2025-04-15T15:53:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410565152}, {"2025-04-15T15:53:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410569248}, {"2025-04-15T15:54:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410586304}, {"2025-04-15T15:54:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410590400}, {"2025-04-15T15:54:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410607456}, {"2025-04-15T15:54:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410611552}, {"2025-04-15T15:54:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410628608}, {"2025-04-15T15:54:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410632704}, {"2025-04-15T15:54:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410649760}, {"2025-04-15T15:54:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410653856}, {"2025-04-15T15:55:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410671008}, {"2025-04-15T15:55:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410675104}, {"2025-04-15T15:55:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410692256}, {"2025-04-15T15:55:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410696352}, {"2025-04-15T15:55:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410713408}, {"2025-04-15T15:55:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410717504}, {"2025-04-15T15:55:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410734560}, {"2025-04-15T15:55:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410738656}, {"2025-04-15T15:56:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410755712}, {"2025-04-15T15:56:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410759808}, {"2025-04-15T15:56:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410776960}, {"2025-04-15T15:56:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410781056}, {"2025-04-15T15:56:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410798112}, {"2025-04-15T15:56:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410802208}, {"2025-04-15T15:56:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410819264}, {"2025-04-15T15:56:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410823360}, {"2025-04-15T15:57:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410840416}, {"2025-04-15T15:57:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410844512}, {"2025-04-15T15:57:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410861568}, {"2025-04-15T15:57:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410865664}, {"2025-04-15T15:57:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410882816}, {"2025-04-15T15:57:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410886912}, {"2025-04-15T15:57:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410904064}, {"2025-04-15T15:57:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410908160}, {"2025-04-15T15:58:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410925216}, {"2025-04-15T15:58:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410929312}, {"2025-04-15T15:58:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410946368}, {"2025-04-15T15:58:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410950464}, {"2025-04-15T15:58:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410963520}, {"2025-04-15T15:58:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410967616}, {"2025-04-15T15:58:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410980672}, {"2025-04-15T15:58:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1410984768}, {"2025-04-15T15:59:01.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1410997728}, {"2025-04-15T15:59:02.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411001824}, {"2025-04-15T15:59:16.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411014880}, {"2025-04-15T15:59:17.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411018976}, {"2025-04-15T15:59:31.722Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411032032}, {"2025-04-15T15:59:32.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411036128}, {"2025-04-15T15:59:46.724Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411049088}, {"2025-04-15T15:59:47.614Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411053184}, {"2025-04-15T15:59:59.999Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411066240}, {"2025-04-15T16:00:00.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411070336}, {"2025-04-15T16:00:01.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411083392}, {"2025-04-15T16:00:02.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411087488}, {"2025-04-15T16:00:03.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411096512}, {"2025-04-15T16:00:04.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411101536}, {"2025-04-15T16:00:05.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411110560}, {"2025-04-15T16:00:06.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411115584}, {"2025-04-15T16:00:07.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411124608}, {"2025-04-15T16:00:08.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411129632}, {"2025-04-15T16:00:09.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411138656}, {"2025-04-15T16:00:10.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411143680}, {"2025-04-15T16:00:11.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411152704}, {"2025-04-15T16:00:12.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411157728}, {"2025-04-15T16:00:13.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411166752}, {"2025-04-15T16:00:14.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411171776}, {"2025-04-15T16:00:15.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411180800}, {"2025-04-15T16:00:16.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411185824}, {"2025-04-15T16:00:17.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411194848}, {"2025-04-15T16:00:18.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411199872}, {"2025-04-15T16:00:19.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411208896}, {"2025-04-15T16:00:20.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411213920}, {"2025-04-15T16:00:21.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411222944}, {"2025-04-15T16:00:22.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411227968}, {"2025-04-15T16:00:23.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411236992}, {"2025-04-15T16:00:24.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411242016}, {"2025-04-15T16:00:25.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411251040}, {"2025-04-15T16:00:26.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 1411256064}, {"2025-04-15T16:00:27.000Z", "node_memory_MemAvailable_bytes", "192.168.56.101:9100", "node1", 1411265088}, {"2025-04-15T16:00:28.000Z", "node_memory_MemAvailable_bytes", "node-exporter:9100", "node", 141127011
```

Thành công: Prometheus thu thập metrics từ Node Exporter trên VPS1 và VPS2, đẩy sang InfluxDB database metrics qua remote_write, sử dụng monitoringnet để đảm bảo scalability và security.

Hướng dẫn lệnh: Các bước tạo network, cấu hình Prometheus, và deploy service được thực hiện chi tiết, xử lý lỗi như file missing, port conflict.

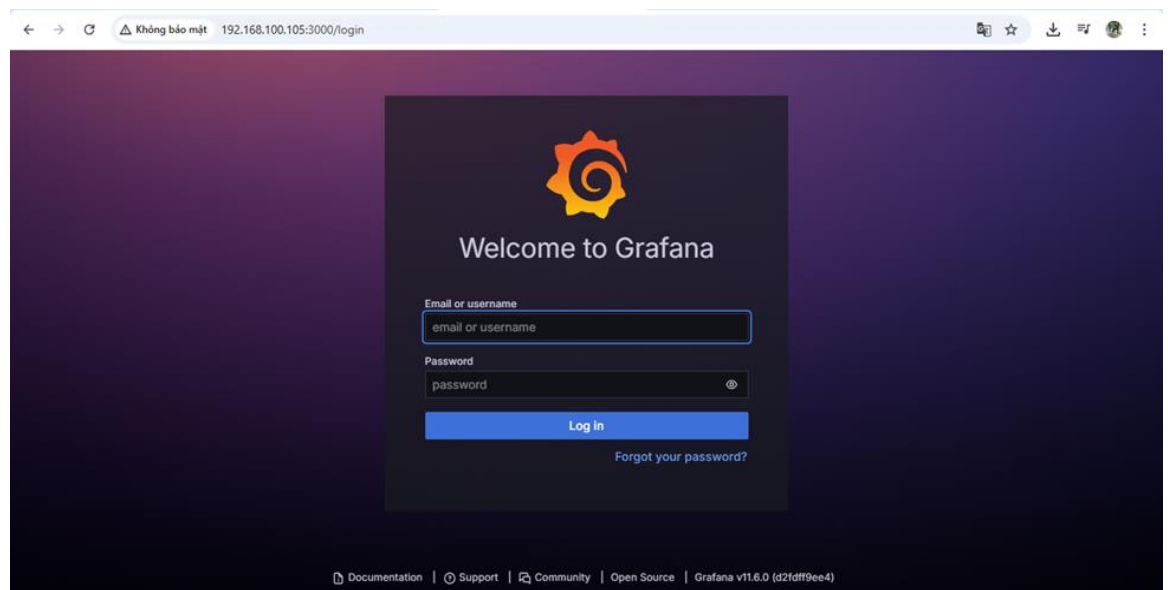
4.17 Tạo Dashboard sử dụng Prometheus/Grafana để phân tích , giám sát hiệu suất swarm

Đầu tiên kiểm tra các dịch vụ Grafana và Prometheus dùng để giám sát swarm có đang chạy hay không bằng câu lệnh : **sudo docker service ls**

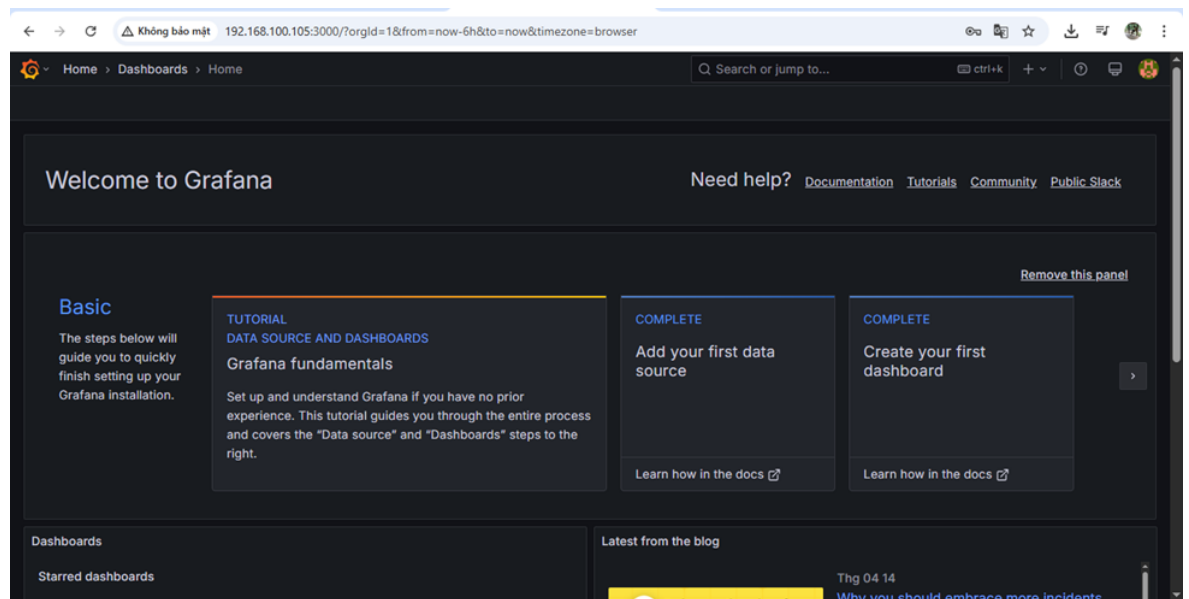
```
osboxes@vps0:~$ sudo docker service ls
ID                NAME                MODE                REPLICAS        IMAGE                PORTS
5c43me7o7pb      coinswarm-proxy      replicated          1/1             nginx:latest        *:8080->80/tcp
ts833ehwhehw     elasticsearch        replicated          1/1             elasticsearch:8.16.5 *:9200->9200/tcp
q3idc7jmpovk     grafana              replicated          1/1             grafana/grafana:latest *:3000->3000/tcp
fqcki1q009iu     hasher              replicated          1/1             ngokhai/hasher:coinswarm *:8002->80/tcp
i15epol6bg4g     influxdb             replicated          1/1             influxdb:1.8         *:8086->8086/tcp
nhbghusrkhq8     kibana              replicated          1/1             kibana:8.16.5        *:5601->5601/tcp
za2s98s205hx     logstash            replicated          2/2             logstash:8.16.5      *:5000->5000/tcp
epl5w53qv7hc     node-exporter        global             1/1             prom/node-exporter:latest
rmhz751ggrvx     prometheus           replicated          1/1             prom/prometheus:latest *:9090->9090/tcp
mhgds69kg51     redis               replicated          1/1             redis:latest
ygd0n13d6i4y     rng                 replicated          1/1             ngokhai/rng:coinswarm *:8001->80/tcp
vqi5ct4n1b3     webui               replicated          1/1             ngokhai/webui:coinswarm *:8000->80/tcp
leje4gi8921d     worker              replicated          1/1             ngokhai/worker:coinswarm
```

Có thể thấy ở 2 dịch vụ grafana và prometheus đều có Replicas là 1/1 -> chứng tỏ dịch vụ đã được khởi động .

Tiếp theo truy cập vào grafana trên trình duyệt bằng địa chỉ IP của máy ảo chạy dịch vụ grafana thông qua port 3000 (đường dẫn truy cập ở đây là : <http://192.168.100.105:3000>)



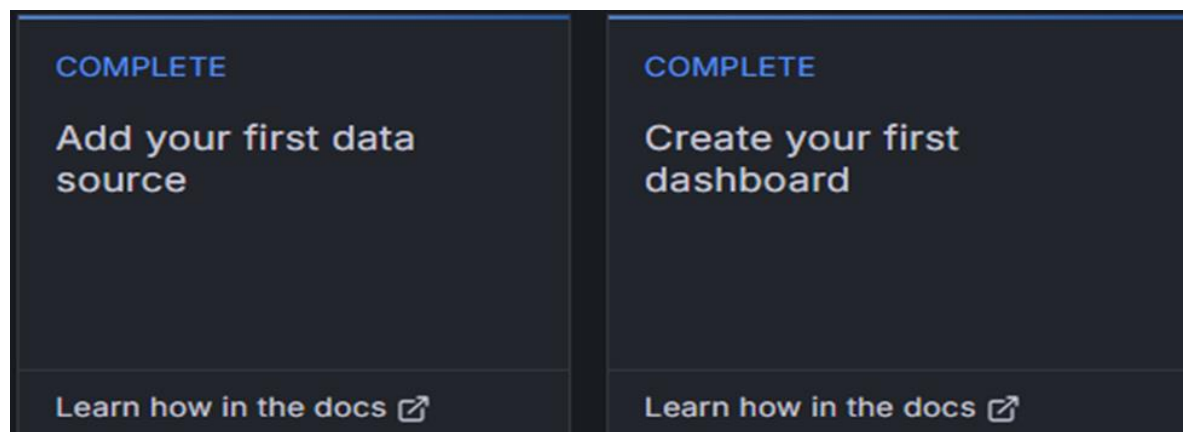
Ở đây cần phải nhập username và password để đăng nhập vào giao diện của grafana. Ở đây cài đặt username = admin và password = admin



Đây là giao diện trang chủ của Grafana

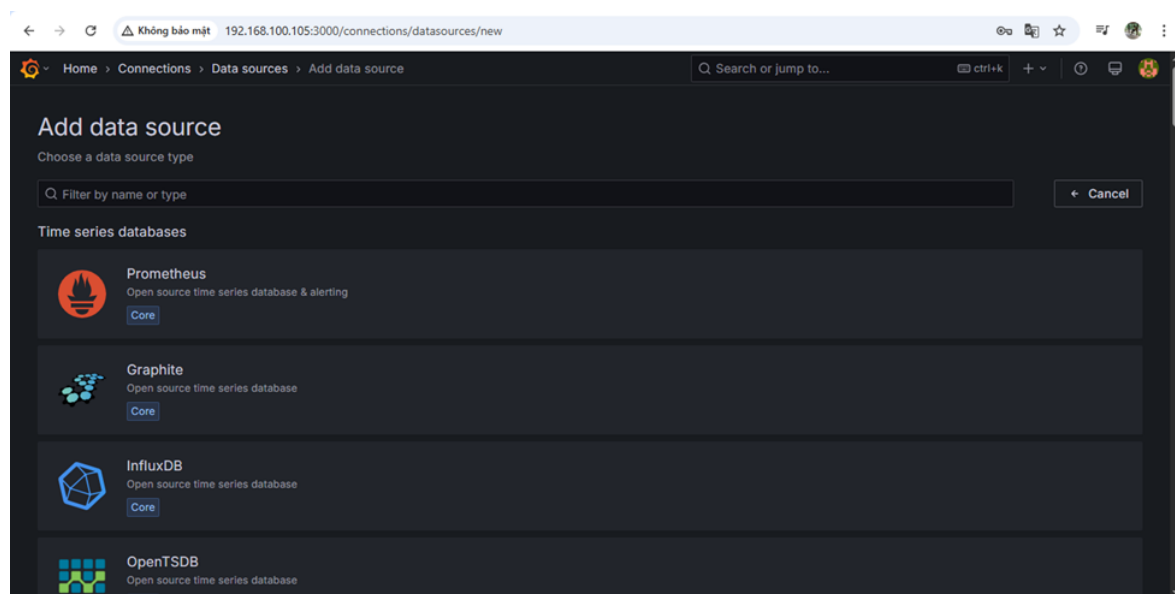
Tiếp theo cần phải import data source cho grafana để có thể trực quan hóa trên dashboard dùng để phân tích . Chúng ta dùng dữ liệu thu được từ dịch vụ Prometheus để giám sát các thành phần chính trong Docker Swarm như:

- +CPU, RAM, Disk, Network của từng **node**
- +Trạng thái và hiệu suất của các **service** và **container**
- +Số lượng container đang chạy, bị lỗi
- +Hiệu năng của Docker daemon



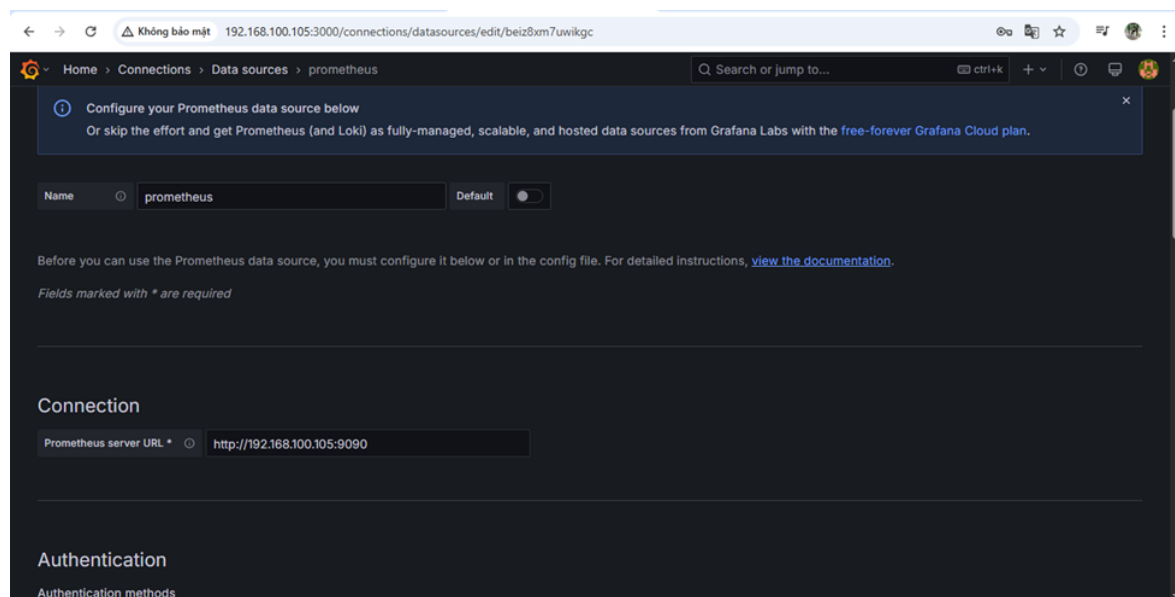
Ở 2 mục này , mục đầu tiên dùng để nhập data source chính là prometheus , mục thứ 2 dùng để tạo một dashboard để trình bày các sơ đồ biểu hiện sự giám sát thông qua các biểu đồ.

Đầu tiên nhấn vào Add your first data source ở mục đầu tiên



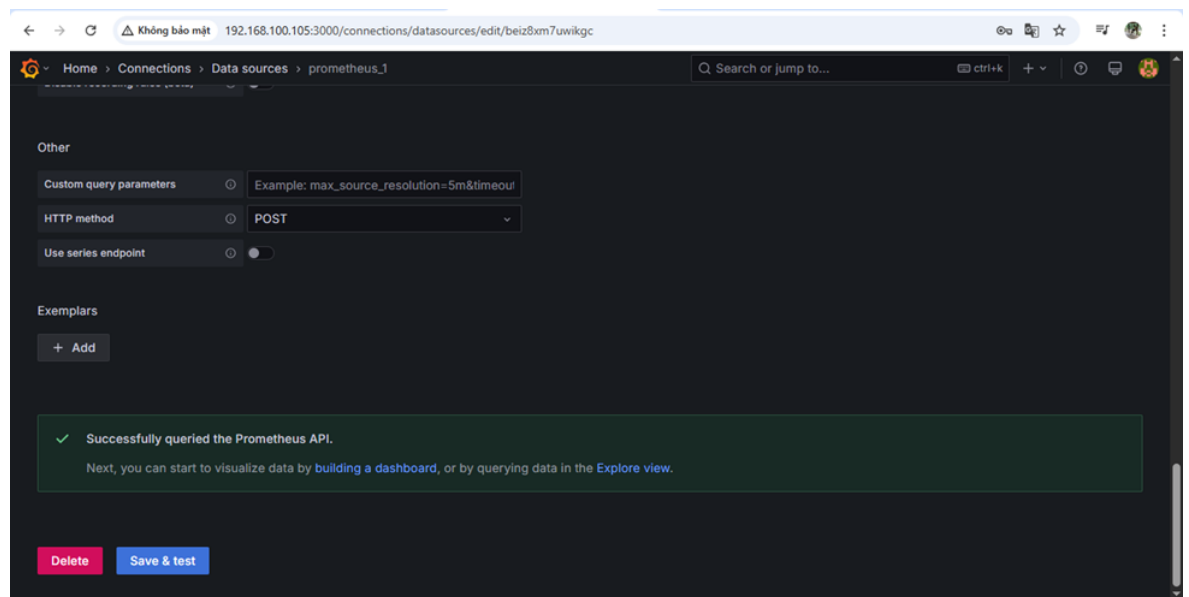
Chọn

Prometheus



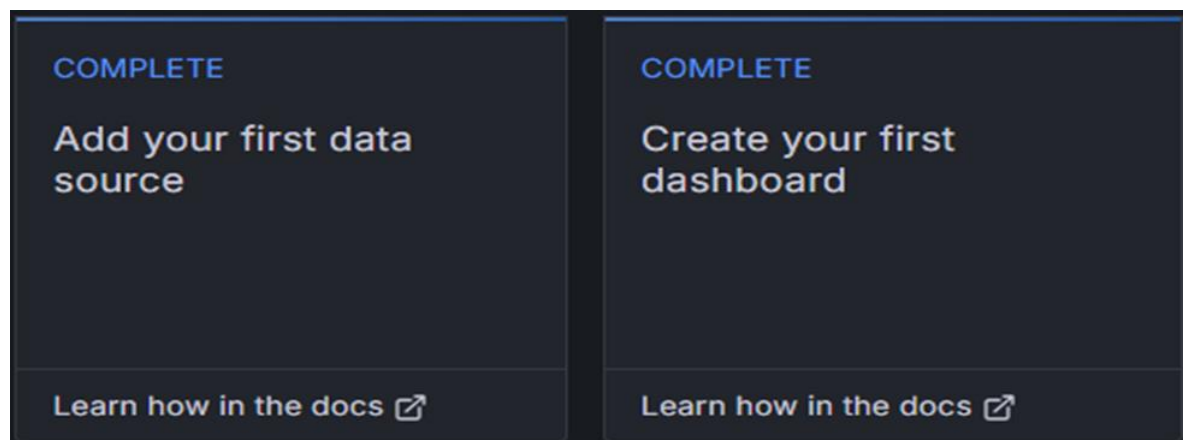
Ở đây nhập tên là prometheus , còn dữ liệu thu được sẽ lấy từ <http://192.168.100.105:9090> (9090 là port đã cài đặt cho prometheus , còn địa

chỉ ip thì sẽ nhập vào tùy vào từng máy ảo đã cài đặt)

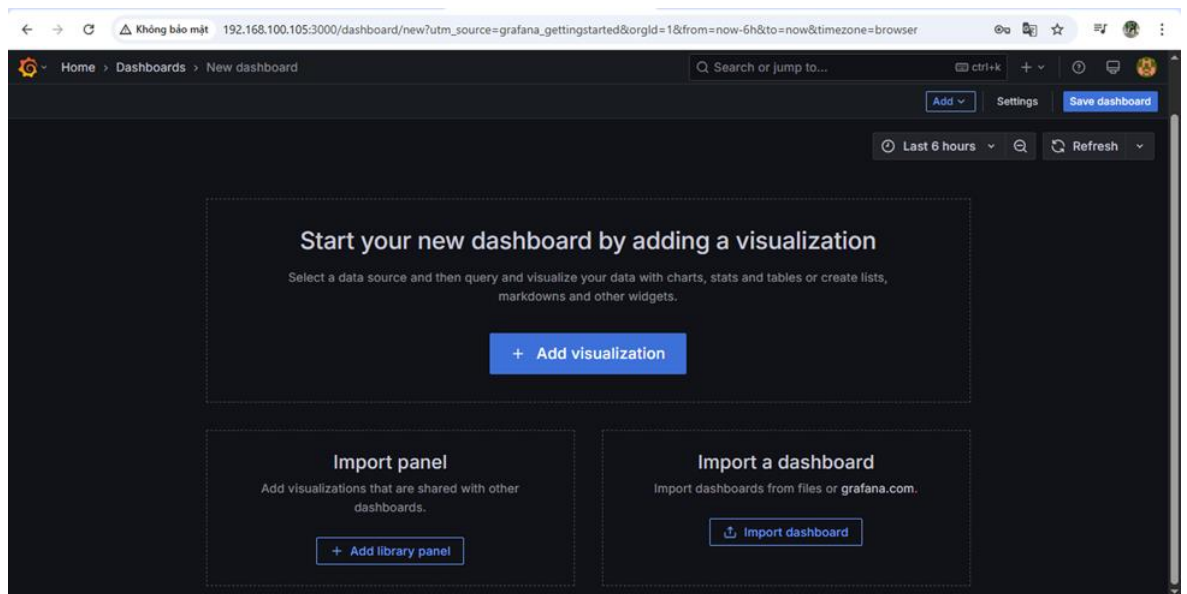


Sau đó nhấn vào Save & test , nếu thông báo hiện lên Successfully thì là dữ liệu đã được import thành công.

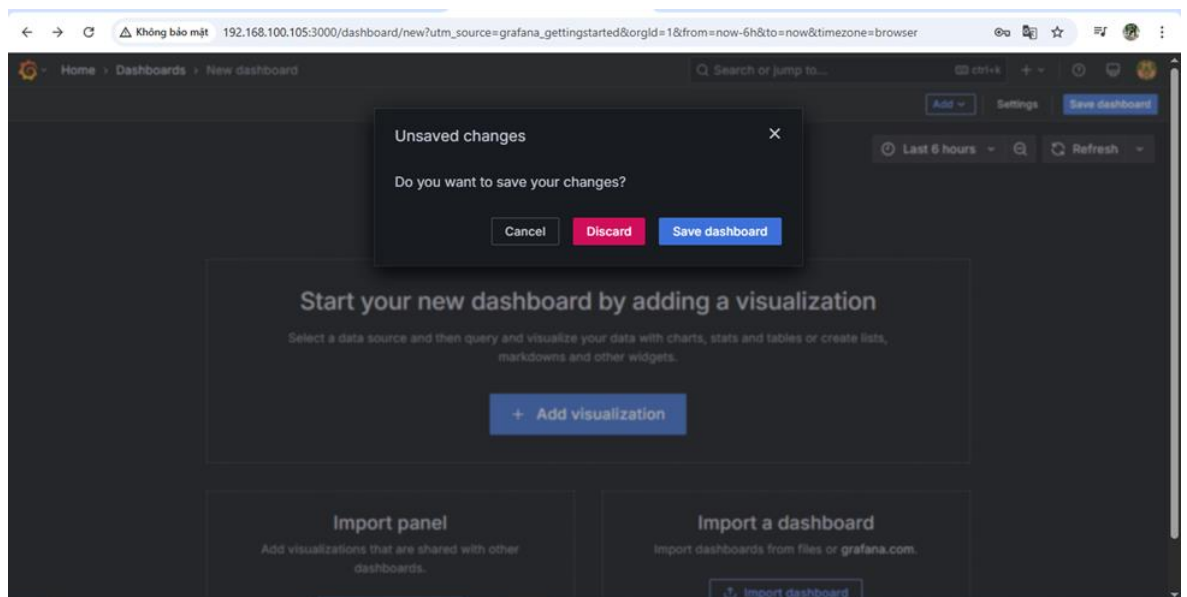
Sau đó quay lại trang chủ của grafana



Vẫn ở 2 mục này , tiếp theo chọn vào Create your first dashboard ở mục thứ 2 để tạo dashboard

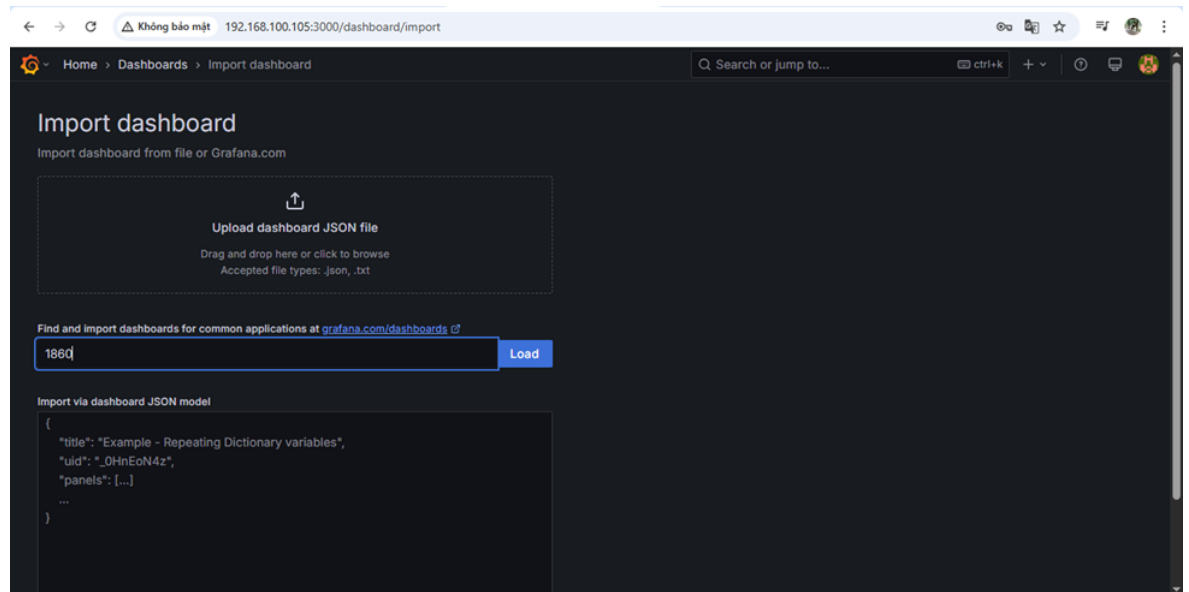


Chọn Import dashboard ở mục Import a dashboard



Chọn

Discard



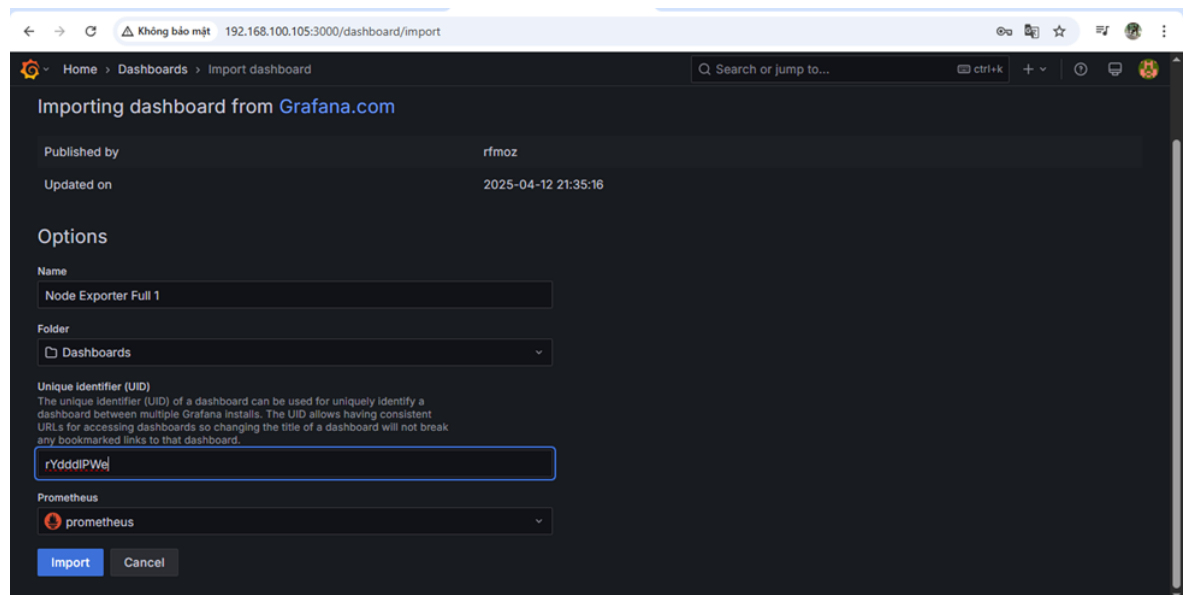
Ở giao diện này nhập vào ô tìm kiếm dashboard 1860 để tìm ID của các dashboard đã được tạo sẵn , 1860 là dashboard được sử dụng phổ biến trong grafana sử dụng prometheus có tên là Node Exporter Full

Sau

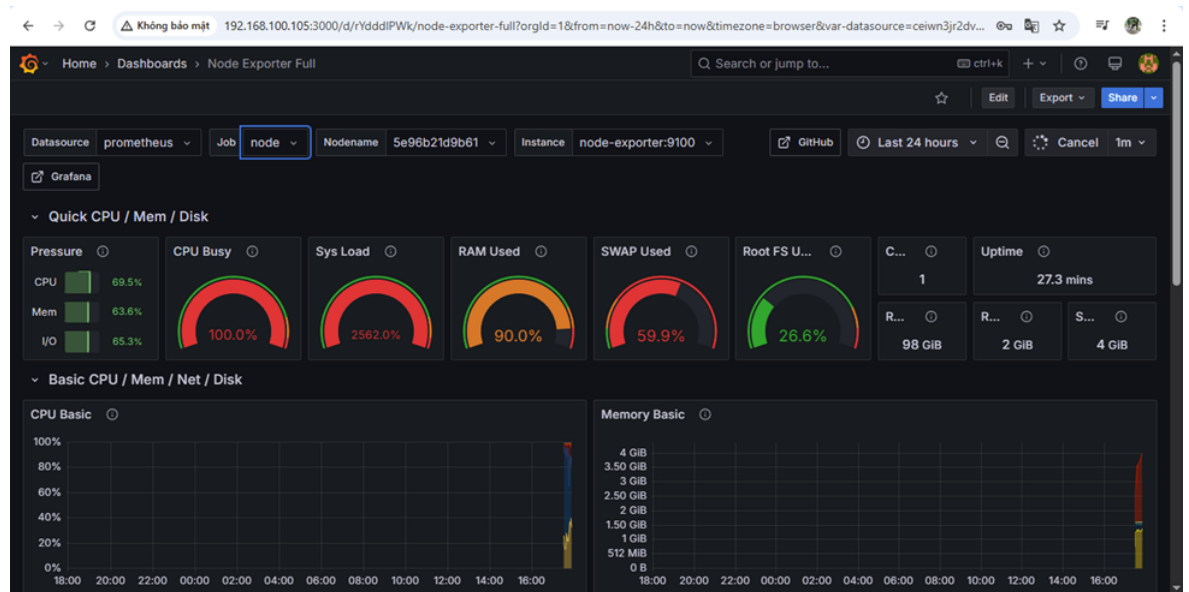
đó

chọn

Load



Tiếp theo ở đây đặt tên cho Dashboard là Node Exporter Full , chọn prometheus là dữ liệu prometheus lúc này đã tạo , sau đó chọn import



Cuối cùng đây là giao diện dashboard thu được từ các dữ liệu của prometheus đã được trực quan hóa

Nhìn vào các số liệu trên biểu đồ có thể phân tích như sau :

-CPU Pressure = 69.5% → cho thấy CPU đang bị sử dụng nặng và có thể bị nghẽn

-CPU Busy = 100.0% → Cho thấy CPU đang bị sử dụng tối đa, rất có thể gây nghẽn nếu kéo dài

-Sys Load = 2562.0%

+ System Load Average đo theo phần trăm:

- Nếu 100% = 1 core full load
- 2562% tương đương ~25.6 core đang bị tải cao (rất nặng)

-RAM Used = 90.0%

+Bộ nhớ vật lý đã sử dụng đến 90% → cần chú ý để tránh out-of-memory

-SWAP Used = 59.9%

+Phân vùng SWAP đã sử dụng hơn 50% → dấu hiệu RAM đang không đủ, phải đẩy sang SWAP (kém hiệu quả)

-Root FS Used = 26.6%

+Phân vùng root (/) của ổ cứng chỉ sử dụng ~26.6% → an toàn

-Uptime thể hiện thời gian khởi động của dịch vụ

-Core =1 thể hiện số lượng core CPU mà node này có

-> Nhận định chung:

-Hệ thống hiện tại đang bị quá tải CPU và RAM:

+CPU sử dụng tối đa (100%)

+Load average quá cao (~25 lần core thực tế)

+RAM gần đầy (90%)

+SWAP hoạt động mạnh → hiệu suất sẽ giảm

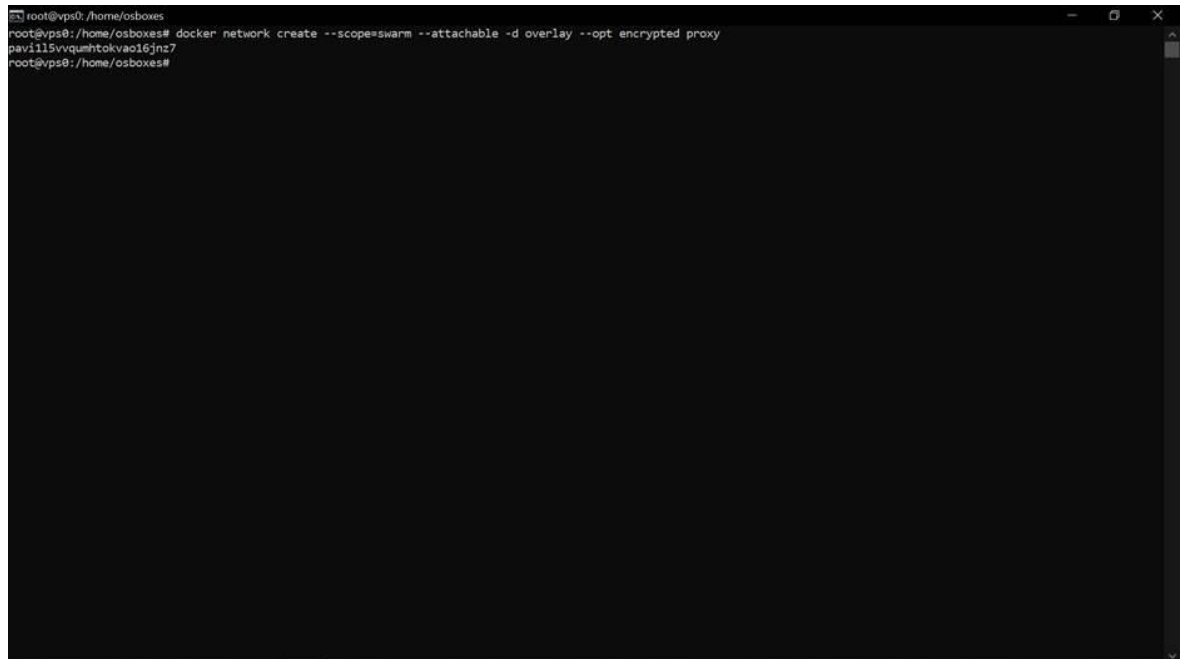
+Rất cần xem lại workload hoặc tăng tài nguyên cho node này

4.18 Cấu hình URL routing trong reverse proxy với nginx trên cụm swarm cho các dịch vụ

Cấu hình Reverse Proxy với NGINX trong Docker Swarm

Bước 1: Tạo mạng overlay proxy cho dịch vụ nginx chạy

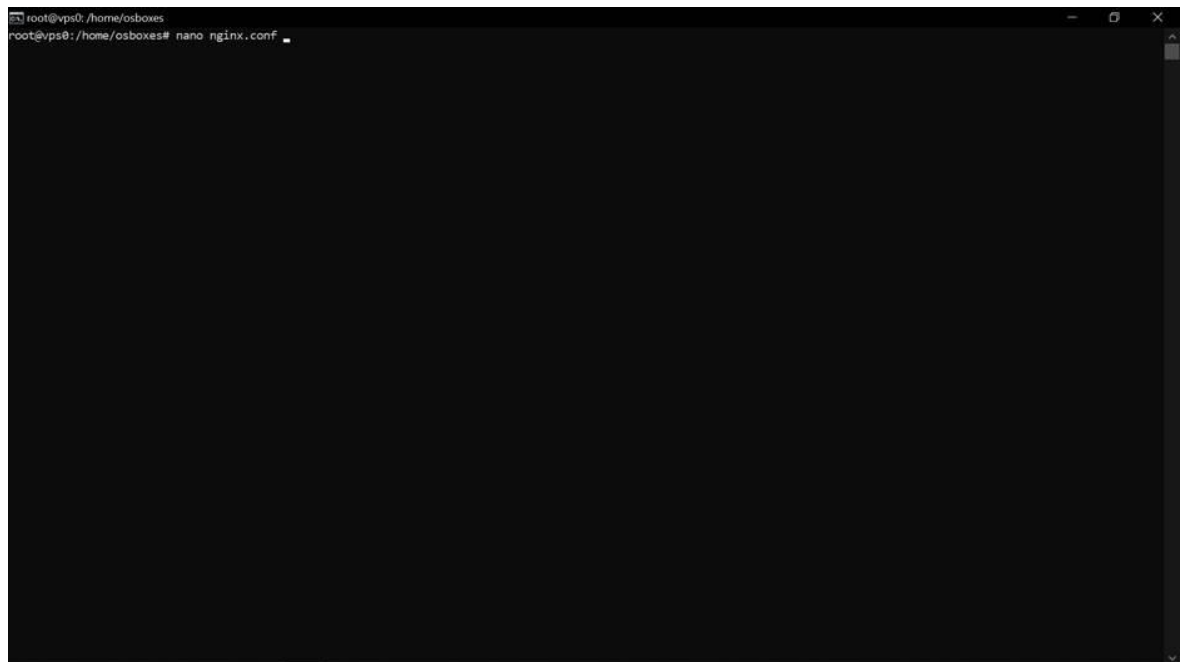
```
docker network create --scope=swarm --attachable -d overlay --opt encrypted proxy
```



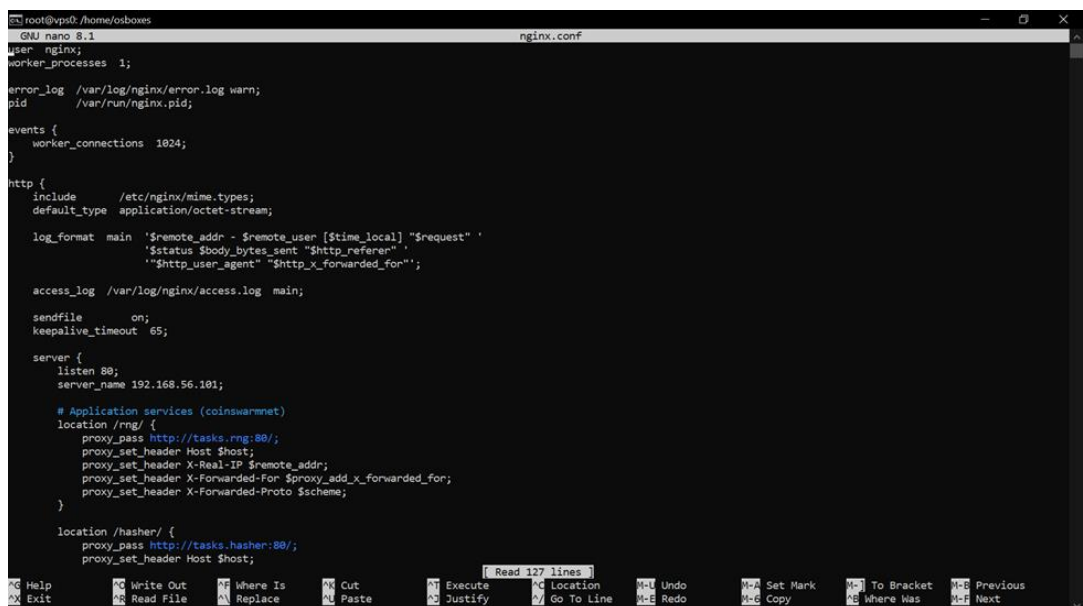
```
root@vps0: /home/osboxes
root@vps0: /home/osboxes# docker network create --scope=swarm --attachable -d overlay --opt encrypted proxy
pavill15vqumhtokvao16jnz7
root@vps0: /home/osboxes#
```

Bước 2: Tạo và cấu hình file nginx.conf

- Dùng lệnh nano nginx.conf



- Gắn nội dung file nginx.conf:



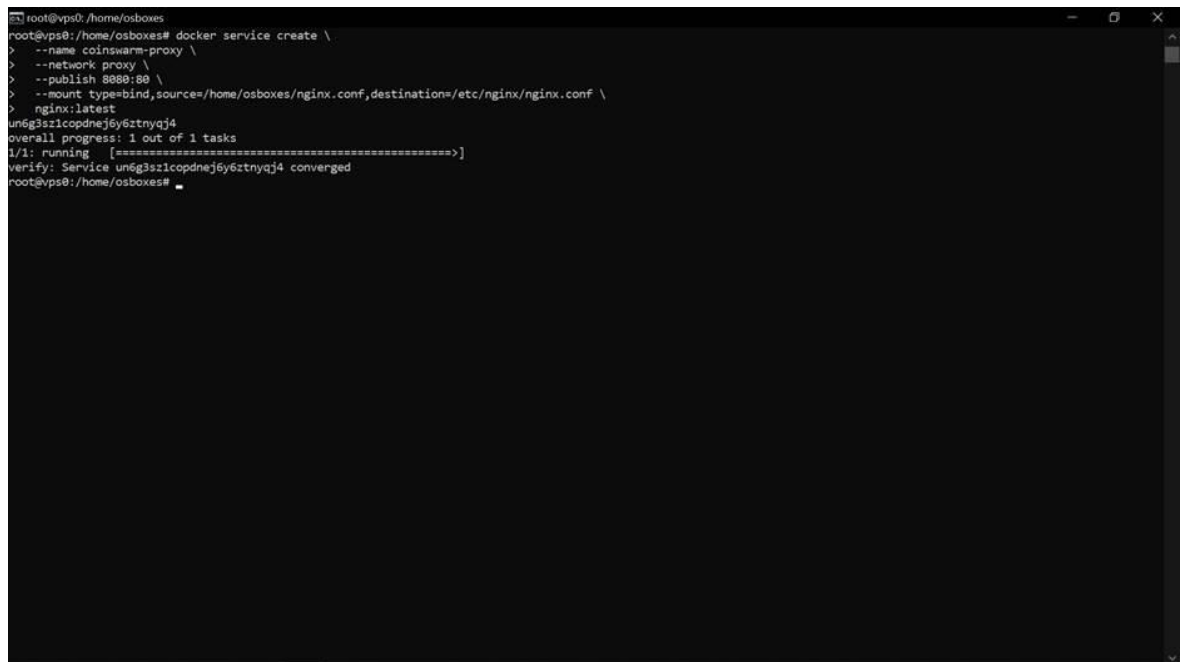
Bước 3: Tạo service reverse proxy với tên coinswarm-proxy

```
docker service create \  
  --name coinswarm-proxy \  
  --network proxy \  
  --
```

```
--publish 8080:80 \

--mount
type=bind,source=/home/osboxes/nginx.conf,destination=/etc/nginx/nginx.co
nf \

nginx:latest
```



```
root@vps0: /home/osboxes
root@vps0:/home/osboxes# docker service create \
> --name coinswarm-proxy \
> --network proxy \
> --publish 8080:80 \
> --mount type=bind,source=/home/osboxes/nginx.conf,destination=/etc/nginx/nginx.conf \
> nginx:latest
un6g3sz1copdnej6y6ztngj4
overall progress: 1 out of 1 tasks
1/1: running [=====]
verify: Service un6g3sz1copdnej6y6ztngj4 converged
root@vps0:/home/osboxes#
```

Bước 4: Kết nối thêm các mạng overlay khác để nginx kết nối được với các service đang chạy trên mạng khác dùng lệnh sau:

```
docker service update \

--network-add coinswarmnet \

--network-add loggingnet \

--network-add monitoringnet \

coinswarm-proxy
```

```
root@vps0:/home/osboxes
root@vps0:/home/osboxes# docker service update \
> --network-add coinswarmnet \
> --network-add loggingnet \
> --network-add monitoringnet \
> coinswarm-proxy
coinswarm-proxy
overall progress: 1 out of 1 tasks
1/1: running [=====>]
verify: Service coinswarm-proxy converged
root@vps0:/home/osboxes#
```

Bước 5: Kiểm tra cấu hình dịch vụ

- Xem thông tin chi tiết dịch vụ và mạng:
docker service inspect coinswarm-proxy --pretty

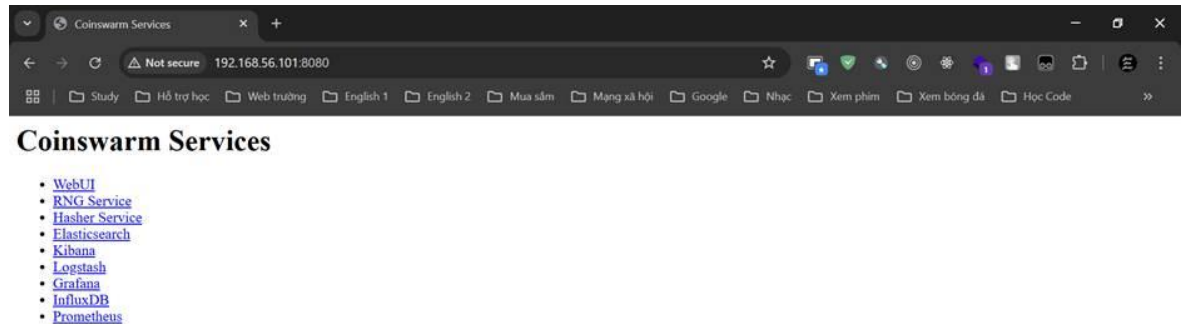
```
Select root@vps0:/home/osboxes
Status: Error: no such service: --pretty, Code: 1
root@vps0:/home/osboxes# docker service inspect coinswarm-proxy --pretty

ID:                un6g3szlcpdnej6y6tynyqj4
Name:              coinswarm-proxy
Service Mode:      Replicated
Replicas:          1
UpdateStatus:
  State:           completed
  Started:         About a minute ago
  Completed:       About a minute ago
  Message:         update completed
Placement:
UpdateConfig:
  Parallelism:     1
  On failure:      pause
  Monitoring Period: 5s
  Max failure ratio: 0
  Update order:    stop-first
RollbackConfig:
  Parallelism:     1
  On failure:      pause
  Monitoring Period: 5s
  Max failure ratio: 0
  Rollback order:  stop-first
ContainerSpec:
  Image:           nginx:latest@sha256:09369da6b18306312cd908661320086bf87fbae1b6b0c49a1f50ba531fef2eab
  Init:            false
Mounts:
  Target:          /etc/nginx/nginx.conf
  Source:          /home/osboxes/nginx.conf
  ReadOnly:        false
  Type:            bind
Resources:
Networks: coinswarmnet monitoringnet proxy loggingnet
Endpoint Mode: vip
Ports:
  PublishedPort = 8080
  Protocol = tcp
  TargetPort = 80
  PublishMode = ingress
root@vps0:/home/osboxes#
```

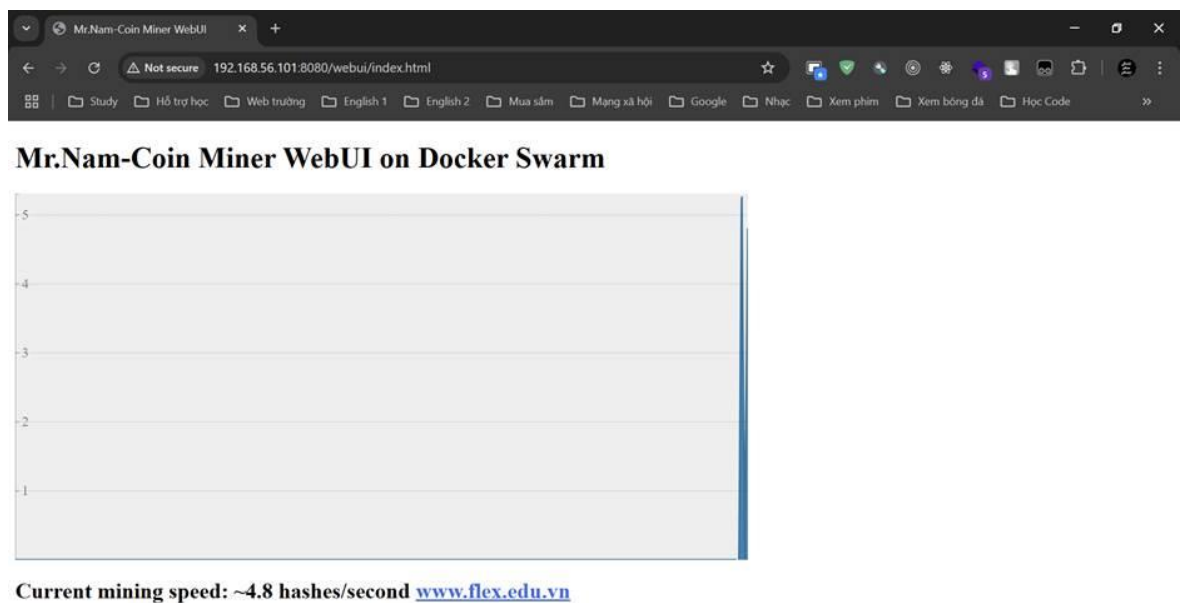
- Ở đây ta thấy được dịch vụ đã được kết nối với các mạng khác

4.19 Cấu hình URL routing trong reverse proxy với nginx trên cụm swarm cho các dịch vụ

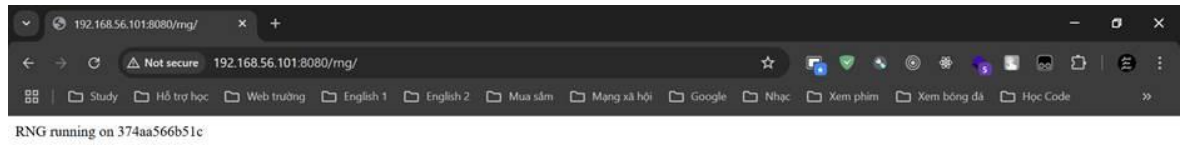
Vào cổng chung 8080 để kiểm tra các dịch vụ đã được kết nối



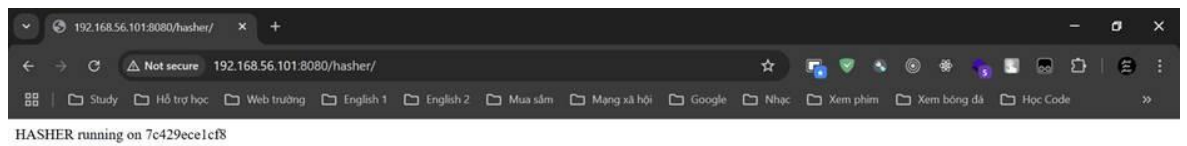
-
- Ở đây ta thấy danh sách các dịch vụ được liệt kê, ta sẽ vào thử vài dịch vụ để kiểm tra
 - WebUI



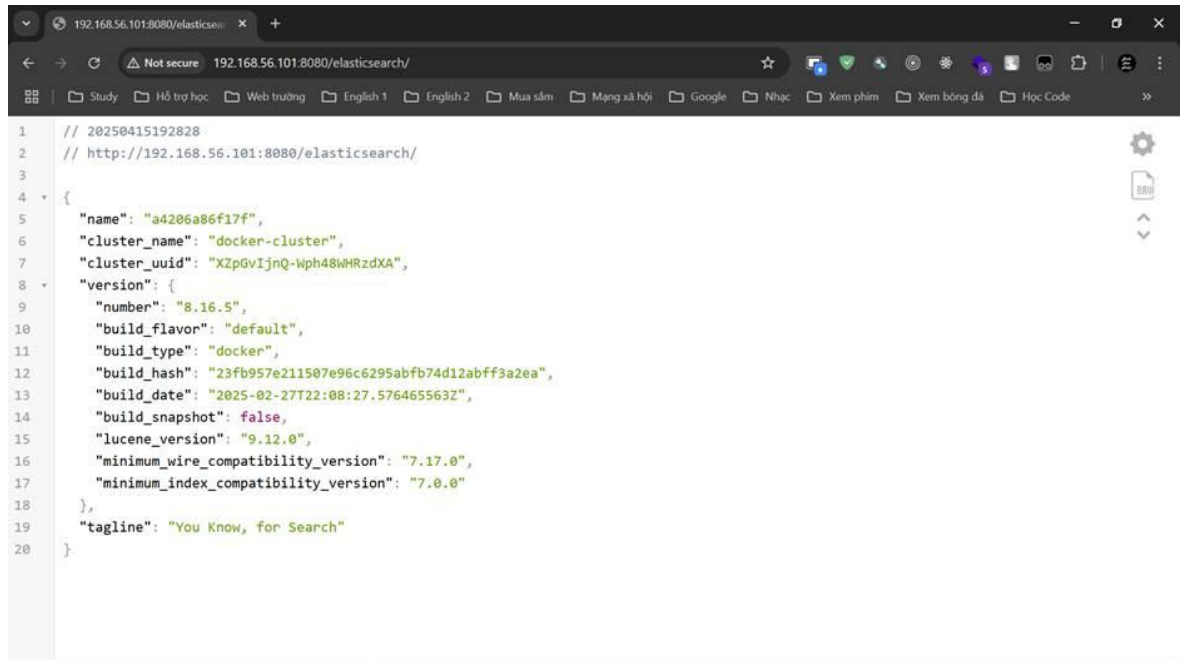
– RNG



– Hasher

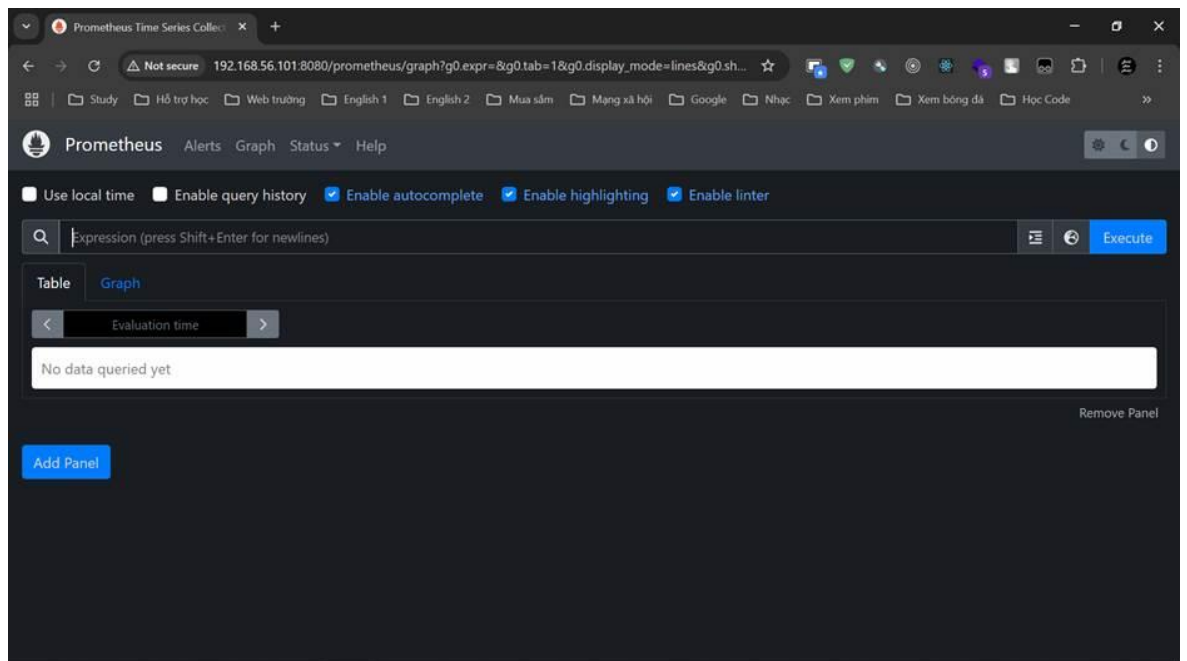


– ElasticSearch



```
1 // 20250415192828
2 // http://192.168.56.101:8080/elasticsearch/
3
4 {
5   "name": "a4206a86f17f",
6   "cluster_name": "docker-cluster",
7   "cluster_uuid": "XZpGvIjnQ-Wph48wHRzdXA",
8   "version": {
9     "number": "8.16.5",
10    "build_flavor": "default",
11    "build_type": "docker",
12    "build_hash": "23fb957e211507e96c6295abfb74d12abff3a2ea",
13    "build_date": "2025-02-27T22:08:27.576465563Z",
14    "build_snapshot": false,
15    "lucene_version": "9.12.0",
16    "minimum_wire_compatibility_version": "7.17.0",
17    "minimum_index_compatibility_version": "7.0.0"
18  },
19   "tagline": "You Know, for Search"
20 }
```

– Prometheus



CHƯƠNG 5: : KẾT LUẬN

5.1 Kết quả đạt được

5.1.1 Kết quả triển khai hệ thống Microservices trên Docker Swarm

- Cluster Docker Swarm hoạt động ổn định:
- Triển khai thành công cụm (cluster) gồm nhiều node (manager & worker) để đảm bảo tính sẵn sàng cao (High Availability - HA).
- Cân bằng tải (Load Balancing) tự động giữa các dịch vụ nhờ Docker Swarm's built-in **Ingress Load Balancer**.
- Triển khai Microservices:
- Các dịch vụ được đóng gói trong **Docker containers**, dễ dàng mở rộng (scale) theo nhu cầu.
- Sử dụng **Docker Compose** hoặc **Stack Deploy** để quản lý các service.
- Reverse Proxy với Nginx:
- Cấu hình Nginx làm **API Gateway** để định tuyến (routing) request tới các microservices.
- Hỗ trợ **SSL/TLS termination** để bảo mật kết nối.

5.1.2 Kết quả giám sát hệ thống với Prometheus-Grafana-InfluxDB

- Prometheus:
- Thu thập metrics từ các microservices, container, và hệ thống (CPU, RAM, Network, Disk I/O).
- Hỗ trợ **Service Discovery** để tự động phát hiện các dịch vụ mới trong Swarm.
- Grafana:
- Xây dựng dashboard trực quan hóa hiệu suất hệ thống (ví dụ:
 - Tốc độ phản hồi API (Latency).
 - Tỷ lệ lỗi (Error Rate).
 - Mức độ sử dụng tài nguyên (Resource Utilization).
- InfluxDB (nếu sử dụng):
- Lưu trữ dữ liệu time-series dài hạn, hỗ trợ phân tích xu hướng.

- Cảnh báo (Alerting):
- Thiết lập cảnh báo qua **Grafana Alert** hoặc **Prometheus Alertmanager** khi hệ thống vượt ngưỡng (ví dụ: CPU > 80%, dịch vụ down).

5.1.3 Kết quả giám sát log tập trung với ELK Stack (Elasticsearch, Logstash, Kibana)

- Log tập trung:
- Các microservices gửi log về **Elasticsearch** thông qua **Logstash**.
- **Kibana** giúp tìm kiếm, phân tích log nhanh chóng (phát hiện lỗi, debug).
- Phân tích lỗi:
- Xác định nguyên nhân sự cố nhanh nhờ log được tổng hợp từ nhiều dịch vụ.
- Hỗ trợ truy vấn theo thời gian thực.

5.1.4 Kết quả tối ưu hóa hệ thống

- Tối ưu hiệu suất:
- Điều chỉnh **replica** của các dịch vụ dựa trên tải (auto-scaling với Prometheus metrics).
- Tối ưu **Nginx caching** để giảm tải cho backend.
- Bảo mật:
- Sử dụng **Docker Secrets** để quản lý thông tin nhạy cảm (mật khẩu, API keys).
- **Network Segmentation**: Cô lập mạng giữa các microservices với Docker Swarm overlay networks.
- Giảm thời gian downtime:
- Rolling updates trong Docker Swarm giúp cập nhật dịch vụ không gây gián đoạn.

5.2 Hướng phát triển

5.2.1 Mở Rộng Quy Mô Hệ Thống (Scaling & High Availability)

- Tự động mở rộng (Auto-scaling):
 - Tích hợp **Docker Swarm** với **Kubernetes (K8s)** để tận dụng khả năng auto-scaling dựa trên tải (CPU/RAM).

- Sử dụng công cụ như **Prometheus** + **Alertmanager** để kích hoạt scaling khi vượt ngưỡng.
- Tăng cường HA:
 - Triển khai **multi-region Swarm cluster** để đảm bảo khả năng chịu lỗi (disaster recovery).
 - Áp dụng **GlusterFS hoặc Ceph** để đồng bộ dữ liệu giữa các node.

5.2.2 Tăng tính bảo mật hệ thống:

Thiết lập HTTPS với chứng chỉ SSL thông qua Let's Encrypt kết hợp với NGINX.

- Giới hạn quyền truy cập vào các service thông qua firewall, VPN hoặc mạng riêng (internal network).
- Thêm các cơ chế xác thực như Basic Auth hoặc OAuth2 cho các dashboard như Grafana, Kibana.

5.2.3 CI/CD & DevOps Automation

- Tích hợp pipeline:
 - Tự động build/deploy ứng dụng lên Swarm qua **GitLab CI/CD hoặc GitHub Actions**.
 - Kiểm thử tự động với **Jenkins + Selenium** trước khi deploy.
- Infrastructure as Code (IaC):
 - Quản lý cấu hình Swarm bằng **Terraform/Ansible**.
 - Versioning cấu hình với **Docker Compose v3 hoặc Kustomize**.

5.2.4 Phân tích log nâng cao:

- Sử dụng thêm công cụ như Filebeat hoặc Fluentd để tối ưu quá trình thu thập log.
- Áp dụng Machine Learning để phát hiện bất thường trong log (Anomaly Detection) hỗ trợ phát hiện sớm lỗi hệ thống.

5.2.5 Triển khai thực tế trên nền tảng Cloud:

- Đưa hệ thống lên các nền tảng đám mây như AWS, Azure hoặc Google Cloud để kiểm tra tính khả dụng và độ ổn định trong môi trường thực tế.

CHƯƠNG 6: TÀI LIỆU THAM KHẢO

Các tài liệu từ Internet

- [1]. <https://github.com/mrhuynhnam/bigdata>